INDEX

S. No	Topic	Page No.
	Week 1	
1	Biological Neuron	1
2	From Spring to Winter of AI	8
3	The Deep Revival	18
4	From Cats to Convolutional Neural Networks	25
5	Faster, higher, stronger	29
6	The Curious Case of Sequences	31
7	Beating humans at their own games (literally)	38
8	The Madness (2013-)	41
9	(Need for) Sanity	50
10	Motivation from Biological Neurons	54
11	McCulloch Pitts Neuron, Thresholding Logic	62
12	Perceptrons	73
13	Error and Error Surfaces	82
14	Perceptron Learning Algorithm	85
15	Proof of Convergence of Perceptron Learning Algorithm	98
	Week 2	
16	Deep Learning(CS7015): Linearly Separable Boolean Functions	108
17	Deep Learning(CS7015): Representation Power of a Network of Perceptrons	113
18	Deep Learning(CS7015): Sigmoid Neuron	124
19	Deep Learning(CS7015): A typical Supervised Machine Learning Setup	136
20	Deep Learning(CS7015): Learning Parameters: (Infeasible) guess work	146
21	Deep Learning(CS7015): Learning Parameters: Gradient Descent	157
22	Deep Learning(CS7015): Representation Power of Multilayer Network of Sigmoid Neurons	179
	Week 3	
23	Feedforward Neural Networks (a.k.a multilayered network of neurons)	209
24	Learning Paramters of Feedforward Neural Networks (Intuition)	222
25	Output functions and Loss functions	229
26	Backpropagation (Intuition)	244
27	Backpropagation: Computing Gradients w.r.t. the Output Units	255
28	Backpropagation: Computing Gradients w.r.t. Hidden Units	267
29	Backpropagation: Computing Gradients w.r.t. Parameters	279
30	Backpropagation: Pseudo code	287

31	Derivative of the activation function	292
32	Information content, Entropy & cross entropy	294

Week 4

33	Recap: Learning Parameters: Guess Work, Gradient Descent	312
34	Contours Maps	324
35	Momentum based Gradient Descent	333
36	Nesterov Accelerated Gradient Descent	346
37	Stochastic And Mini-Batch Gradient Descent	355
38	Tips for Adjusting Learning Rate and Momentum	368
39	Line Search	377
40	Gradient Descent with Adaptive Learning Rate	383
41	Bias Correction in Adam	404

Week 5

42	Eigenvalues and Eigenvectors	412
43	Linear Algebra : Basic Definitions	424
44	Eigenvalue Decompositon	434
45	Principal Component Analysis and its Interpretations	442
46	PCA : Interpretation 2	465
47	PCA : Interpretation 3	477
48	PCA : Interpretation 3 (Contd.)	482
49	PCA : Practical Example	484
50	Singular Value Decomposition	491

Week 6

51	Introduction to Autoncoders	510
52	Link between PCA and Autoencoders	543
53	Regularization in autoencoders (Motivation)	556
54	Denoising Autoencoders	564
55	Sparse Autoencoders	582
56	Contractive Autoencoders	590

Week 7

57	Bias and Variance	599
58	Train error vs Test error	608
59	Train error vs Test error (Recap)	624
60	True error and Model complexity	631
61	L2 regularization	639
62	Dataset augmentation	656

63	Parameter sharing and tying	660
64	Adding Noise to the inputs	662
65	Adding Noise to the outputs	667
66	Early stopping	671
67	Ensemble Methods	679
68	Dropout	686

Week 8

69	A quick recap of training deep neural networks	700
70	Unsupervised pre-training	706
71	Better activation functions	726
72	Better initialization strategies	752
73	Batch Normalization	774

Week 9

74	One-hot representations of words	786
75	Distributed Representations of words	793
76	SVD for learning word representations	802
77	SVD for learning word representations (Contd.)	803
78	Continuous bag of words model	814
79	Skip-gram model	839
80	Skip-gram model (Contd.)	846
81	Contrastive estimation	856
82	Hierarchical softmax	861
83	GloVe representations	871
84	Evaluating word representations	877
85	Relation between SVD and Word2Vec	885

Week 10

86	The convolution operation	888
87	Relation between input size, output size and filter size	907
88	Convolutional Neural Networks	919
89	Convolutional Neural Networks (Contd.)	935
90	CNNs (success stories on ImageNet)	951
91	CNNs (success stories on ImageNet) (Contd.)	972
92	Image Classification continued (GoogLeNet and ResNet)	974
93	Visualizing patches which maximally activate a neuron	989
94	Visualizing filters of a CNN	993
95	Occlusion experiments	999

96	Finding influence of input pixels using backpropagation	1003
97	Guided Backpropagation	1008
98	Optimization over images	1012
99	Create images from embeddings	1022
100	Deep Dream	1032
101	Deep Art	1039
102	Fooling Deep Convolutional Neural Networks	1044

Week 11

103	Sequence Learning Problems	1050
104	Recurrent Neural Networks	1057
105	Backpropagation through time	1067
106	The problem of Exploding and Vanishing Gradients	1080
107	Some Gory Details	1089
108	Selective Read, Selective Write, Selective Forget - The Whiteboard Analogy	1095
109	Long Short Term Memory(LSTM) and Gated Recurrent Units(GRUs)	1103
110	How LSTMs avoid the problem of vanishing gradients	1126
111	How LSTMs avoid the problem of vanishing gradients (Contd.)	1133

Week 12

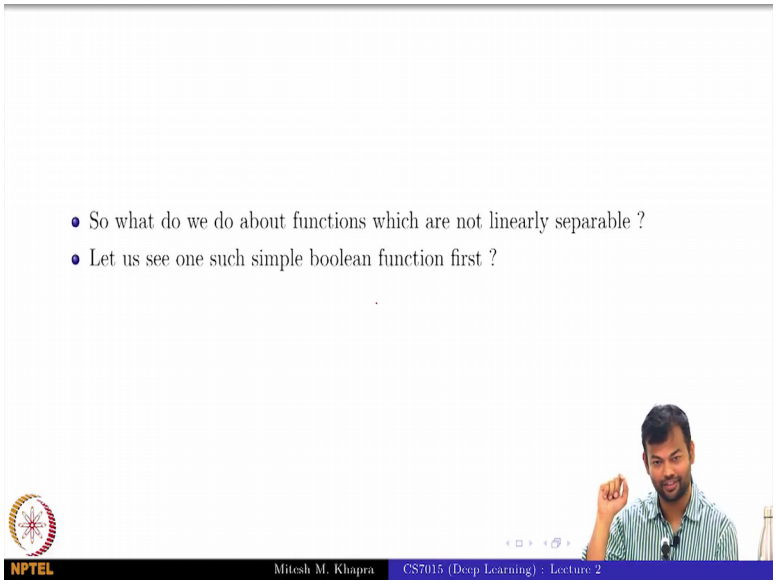
112	Introduction to Encoder Decoder Models	1152
113	Applications of Encoder Decoder models	1169
114	Attention Mechanism	1185
115	Attention Mechanism (Contd.)	1205
116	Attention over images	1207
117	Hierarchical Attention	1216

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module - 2.7
Lecture - 02
Linearly Separable Boolean Functions

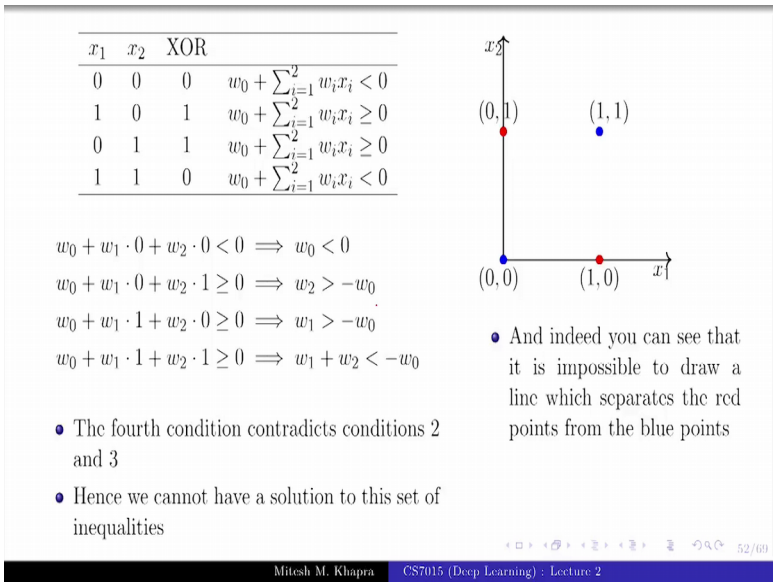
So, in this module, we look at Linearly Separable Boolean Functions again and we will try to make some more statements about them.

(Refer Slide Time: 00:20)



So, what do we have do; so, the guiding question that we have is what do we do about functions which are not linearly separable and let us see one such very simple function. Can you guess what function I am going to talk about? All of you are paying attention in the first lecture.

(Refer Slide Time: 00:36)

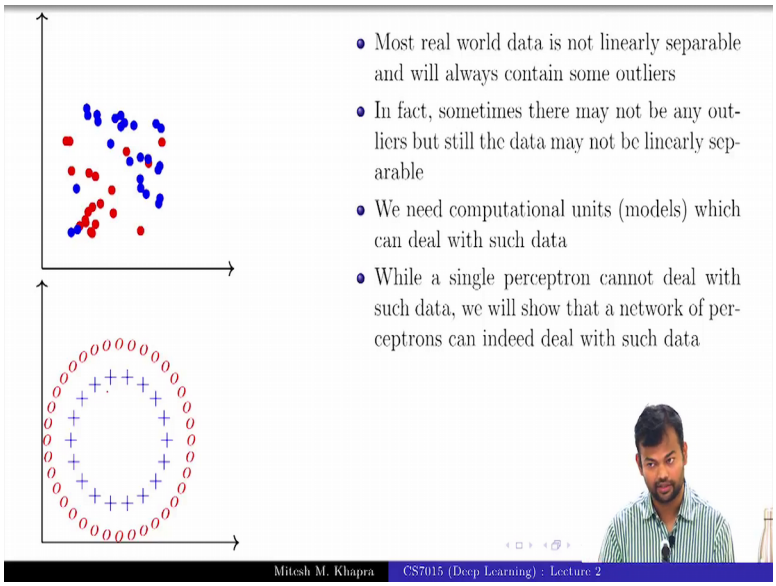


So, here is the XOR function. Now, these are the set of inequalities that result from XOR function. I hope right. Now, let us see the first condition implies that w naught should be less than 0, second condition implies this, third condition implies this, fourth condition implies this. Just looking at this can you tell me, can you find a configuration for w naught w1, w2, such that these inequalities can be satisfied together. No, right because 2 and 3 want it to be greater than minus 1 minus w naught and when you take an addition of that, it has to be less than minus 1.

So, that is not going to happen. So, you see a contradiction. So, this is a simple Boolean function which the perceptron cannot handle because it is not linearly separable. It is not linearly separable. There does not exist a line. If there does not exist a line, you cannot find the line. In fact, you can look at it visually.

So, these are the red points for which the output should be 0 or 1 and the blue points are the points for which the output should be 0. If we need to change this, I think we were using blue as positive and red as negative and you cannot just draw a line. There is no way you can draw a line such that the blue points lie on one side and the red points lie on the other side. So, it is a simple two input function. So, it is not that I have taken a very contrived example.

(Refer Slide Time: 01:57)

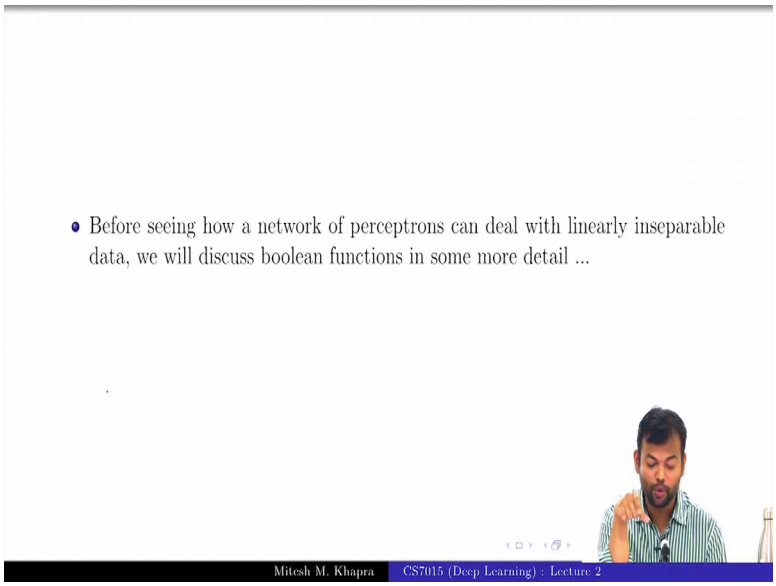


Most real world data is not linearly separable and it always contains some outliers, right. So, here maybe you have some data where you are trying to say that people which live in this part of the world belong to a certain or maybe people who live or work here have a certain qualification, people who work in this company may have a certain different qualification, and there might be some outliers, right. It is not that is always going be very clean. So, now what do I mean? .And it is not necessary that the points will only be outliers.

In fact, there could be a clear case where there are no outliers, but still you cannot find a line such that you separate the positive from the negative. Can you think of such an example? Good right. This is clear data. There is no outliers here as well. I mean it is just saying that everyone who lies within this boundary has a certain characteristic and outside that boundary people have a different characteristic, right and there is no outlier here, but you cannot separate this data with a line. So, all functions that you deal with will not go or are not going to be linearly separable.

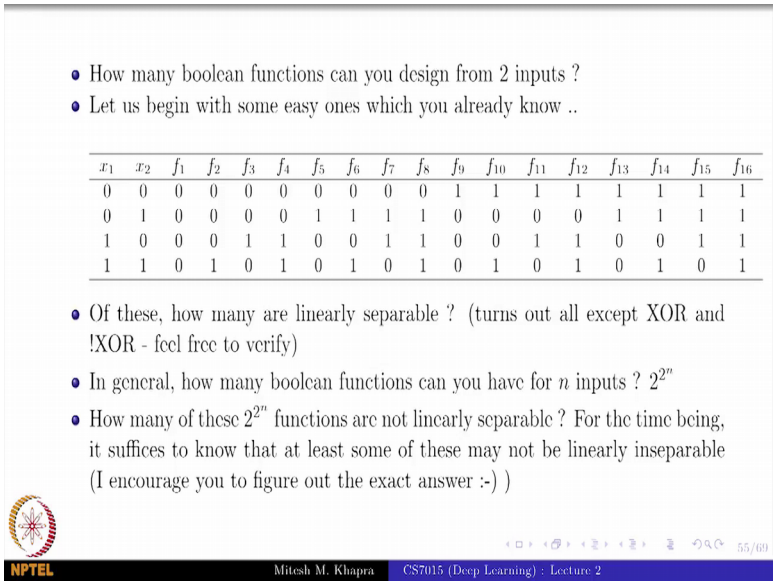
So, we have to work around those, right and while a single perceptron cannot deal with this, we will show that a network of perceptron's can indeed deal with such data. So, that is where we are headed.

(Refer Slide Time: 03:16)



So, before going there we will discuss some more Boolean functions in more detail. And I will try to see what kind of non-linearly separable Boolean functions are there.

(Refer Slide Time: 03:26)



x_1	x_2	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}	f_{16}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

So, first of all how many Boolean functions can you design from two inputs? How many can you design? 16 looks like a good number from 3 inputs 256. How many if you understand this? Let us see. So, let us begin with some easy ones that you already know, right. So, these are two inputs x1, x2. What is this function always off? The other extreme is always on and I have already given you the answer f 16.

So, then you have the AND function and OR function then some other functions, right. So, why did you reach 16? Actually because with two inputs we will have these four values to take care of and each of these are again binary. So, you actually have 2 raise to 2 raise to n, right. So, for three inputs 2 raise to 2 raise to 3 would be 256. Now, that is the easy part of these. How many are linearly separable? I will have to do any actually stare it in and seriously try to find the answer when you cannot really do that.

So, turns out all of them except XOR and in, not of XOR, ok; so for the two input cases, there are two functions which are not linearly separable. For n inputs how many functions would be not linearly separable? It is an arbitrary. N is not the answer. You are not going to disappoint me, not n ok, but what is the answer. So, for n inputs, we will have two raise two n functions of these we do not know how many are going to be not linearly separable. That is not a solved problem, although I encourage you to go and find the answer.

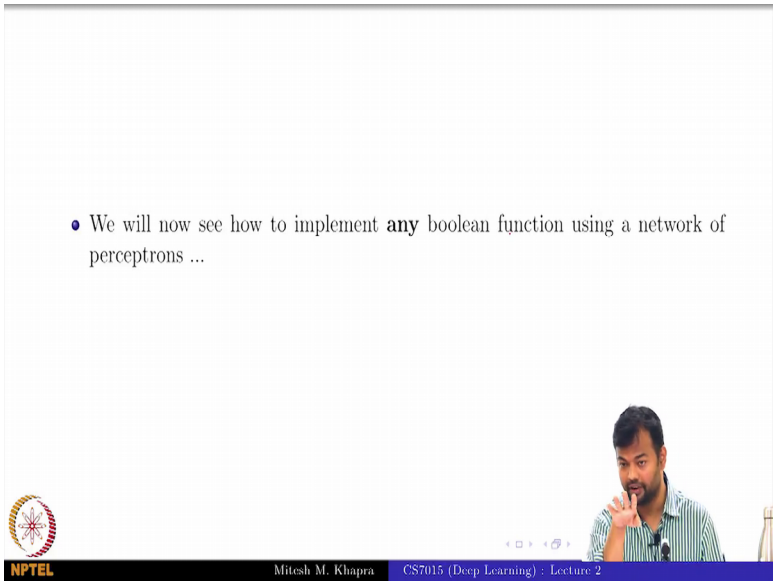
I am looking for a good will hunting kind of a moment, but all it suffices to know is that there exists some which are not linearly separable, and that everyone agrees that there exists some, right and as n grows probably that number will increase and so on, but it is not known exactly. You cannot write it as a function. So, what we have done so far is looked at Boolean functions, how many Boolean functions can exist and of that we just have concluded that there would be some which are not linearly separable.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering Indian Institute of Technology, Madras

Module – 2.8
Lecture – 02
Representation Power of a Network of Perceptrons

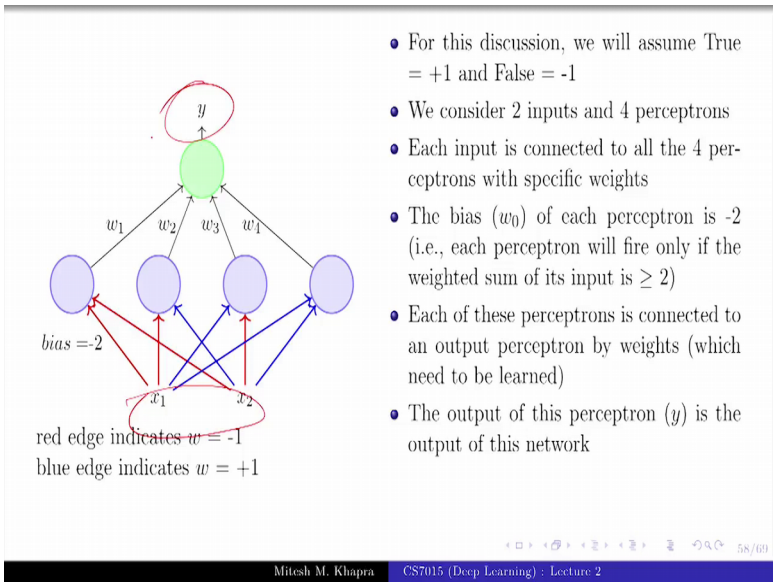
We will go to the next module, where we talk about a Network of Perceptrons and then we talk about the representation power of a network of perceptrons. So, this module should have been titled as Network of Perceptrons. So now, in particular what we are going to see is how any Boolean function whether linearly separable or not can be represented using a network of perceptrons.

(Refer Slide Time: 00:25)



Now, what do I mean by represented during a network of perceptrons? What it means is that I will give you a network of perceptrons, you take any Boolean function feed any value of x1 to xn and the network will give you the same y as it is expected from the truth table, ok, that is what representation means just to put it out clearly.

(Refer Slide Time: 00:57)



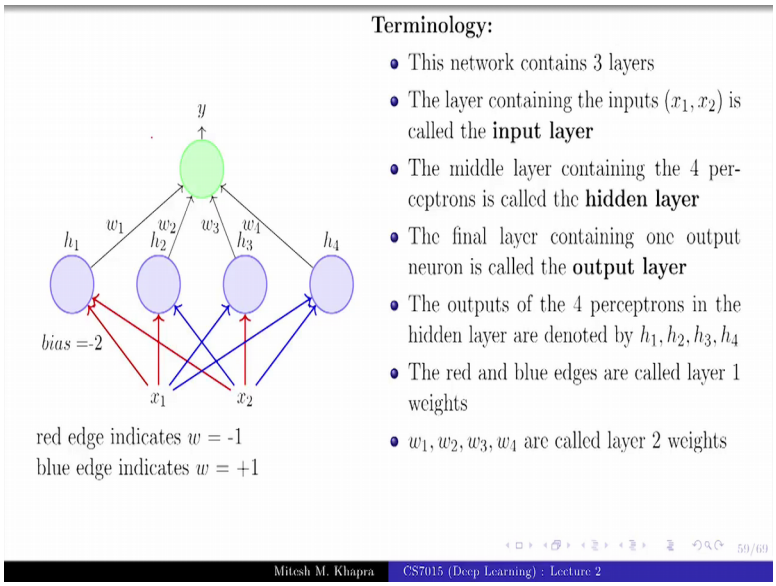
And now I am going to again do a setup, I am not giving you the solution, I am just making some set up and then we will discuss the solution.

So, for this discussion we will assume that true equal to plus 1 and false equal to minus 1. So, instead of 0 and 1, we will assume minus 1 and plus 1. And these are your inputs x1 and x2 we are taking the 2 input case. And I will have 4 perceptrons first, I will have 4 perceptrons; and I will also have very specific weights connected to form the inputs to these perceptrons. So, red means minus 1 and blue means plus 1, right. So, the first 2 inputs are connected with a weight of minus 1. The next 2 inputs with minus 1 plus 1, plus 1 minus 1 and the last would be?

Plus 1. Now, once I have this, I will set the bias of all these perceptrons to minus 2. So, that will; that means, they will fire only if they are weighted sum of the inputs is greater than 2. Now after this I will have one more perceptron. So, I had 2 inputs, I converted that to 4 values, these 4 values are now going to feed into one more perceptron.

And these weights I will not fix them, these are the weights that I am going to learn ok, these and the final output of this perceptron which is the green perceptron is the output of the network, right. So now, coming back to what I said that it can represent any function, what I mean is that you take any function, feed in any combination of x1, x2, this network will give you y and I am telling you that it will match the truth table of that function.

(Refer Slide Time: 02:43)

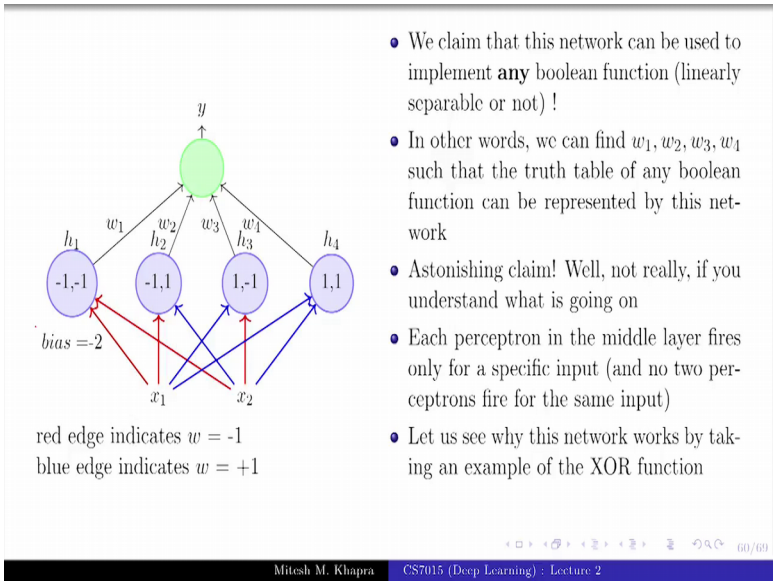


Now, let us define some terminology this will also stay with us for the rest of the course.

So, this network contains 3 layers. The layer containing the inputs it is called the input layer; very creative, the middle layer containing the 4 perceptrons is called the hidden layer. And the output layer which gives you the output is called the output layer, output perceptron which gives you the output is called the output layer right. So, you have a input layer, a hidden layer and an output layer.

And the outputs of the 4 perceptrons I am going to call them as h1, h2, h3 and h4. And the red and blue edges are called the weights for the layer 1, which we have not learned we have actually set them by hand. And the weights for w1, w2, w3, w4 are called the weights for the second layer, other the layer 2 weights and these are the weights that we want to learn.

(Refer Slide Time: 03:34)

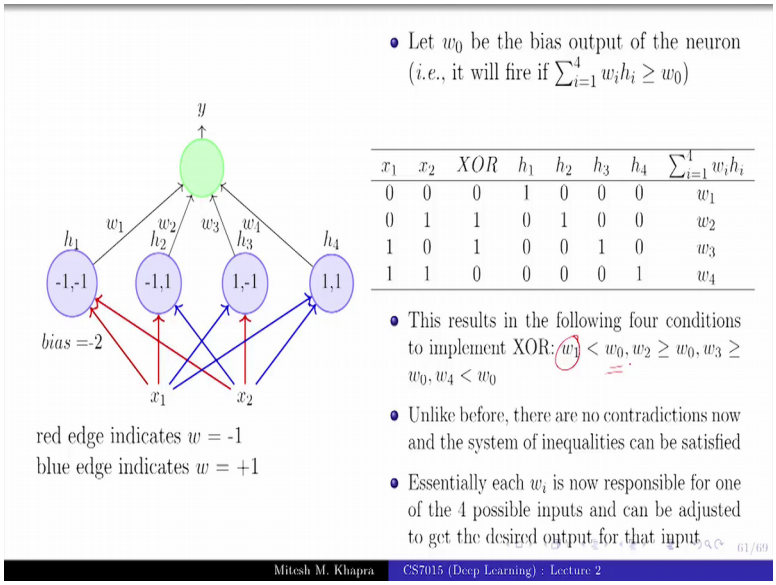


Now I make this claim that this network, it can take any Boolean function, it can implement any Boolean function. So, this same network can implement any Boolean function; that means, if I take this network, and if I try to learn the values of w1, w2, w3, w4, for any Boolean function whether it was originally linearly separable or not; I will be able to implement it.

So, isn't this an astonishing claim? Any Boolean function, do you think this is an astonishing claim? Well not really if you really understand what is happening here right. So, let us see what exactly is happening here. So, when will perceptron 1 fire? When the input is false, false 0, 0; will it fire for any other input? When will perceptron 2 fire? Any other input? Same for the next perceptron? Same for the next perceptron?

So, you start getting an intuition of what is happening here? You do, ok let us see. So now, for this particular network now that I have given you some intuition of what is happening. Basically every node or every neuron in the hidden layer is catering to one of the inputs, and it will fire only for that input, it will not fire for anyone else.

(Refer Slide Time: 04:59)



So now let us try to implement the XOR function, and see what are the. So now, let us try to implement the XOR function, and see what are the set of inequalities that result from this. Earlier when we try to look at the set of inequalities, we ended up with a contradiction, let us see if that happens now. So, this is x1, x2 this is your XOR function. So, that is just like any truth table then I am noting down the intermediate values. And then my final input to the green perceptron is going to be summation of these, and it will fire if this summation is greater than equal to 0, or else it will not fire.

Now, for the first case when the input is 0 comma 0 what is h1 going to be? 1 and everything else is going to be 0, that is exactly what we saw in the previous slide. So, what is the summation going to be? Just w1 right, so, it is w1 h1 plus w2 h2 so on, but h2 to h4 are 0. So, only thing that remains is w1; for the second case, w2; for the third case, w3 for the 4th case w4.

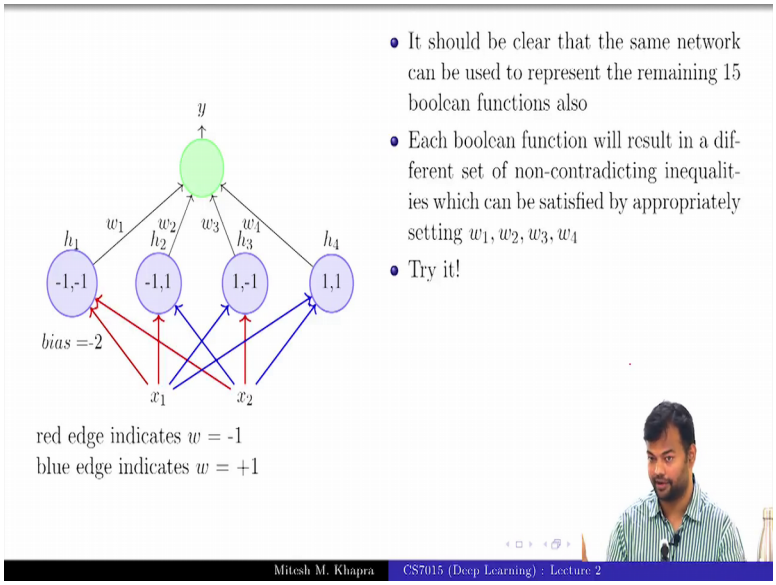
So, is it clear now what is happening? Let us go a bit more into detail right. So now, for the XOR condition, what are the conditions that we need? w1 should be less than w0. Because this should not fire, w2 should be greater than equal to 0, w3 should be greater than equal to sorry w naught not w is not 0, and w4 should not fire. So, w4 should be less than w0. Are there any contradictions here? By design no right. So, we have made sure that for the final layer only one of these guys feeds to it.

So, it does not matter what the remaining outputs are. They do not interfere with each other, unlike earlier where we had conditions like w1 should be something, w2 should be something, and then w1 plus w2 should be something. There are no such contradictions here. Because we have made sure that every neuron in the middle layer actually caters to one specific input, and now the weights in the final layer can be adjusted so that we get the desired output for that input.

So, I can set whatever value of w1 I need to set; so, that I can fire the neuron. In fact, I could just fix w0 as 0, and then I can adjust the weights of w1, w2, w3, w4. And I can implement the XOR function. So, are you convinced that this can be used to implement any Boolean function? How many if you are not convinced? So, the negative question never works, how many if you are convinced? Sure.

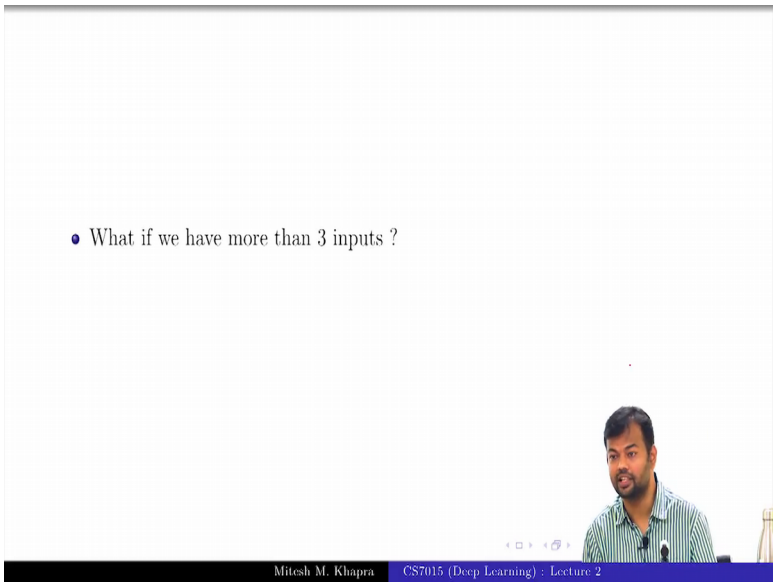
Now, what if we had 3 inputs?

(Refer Slide Time: 07:40)



Before that it should be clear that the same network can be used for any of the remaining 15 functions. And for each of these functions we will end up with a different value of w1, w2, w3, w4, but you will be able to satisfy the truth table, right and you can go home and try it, which I am sure you will do.

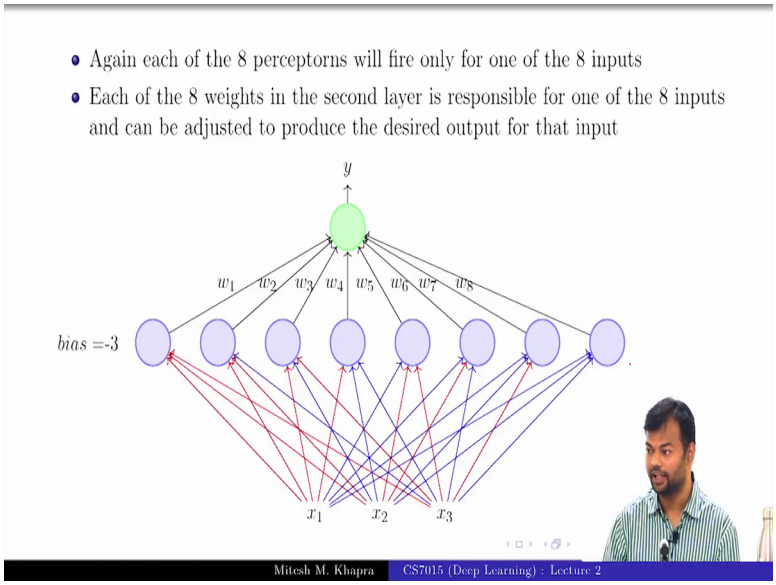
(Refer Slide Time: 07:56)



Ah So, what if we have a function of 3 inputs? 256, what is 256? No.

8. Fine, sure.

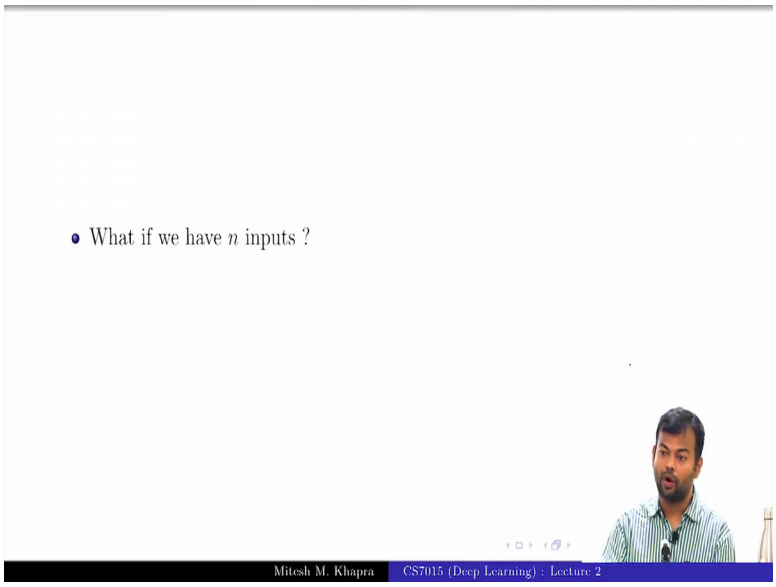
(Refer Slide Time: 08:11)



So, this is what it will look like, and anything specific about the weights of the initial layer, can you tell me what the weights would be? Just tell me red red red red blue blue whatever colours you like this thing. First perceptron what would the weight colours be red, red, red, then?

Enough so, this is how it will look right, right and now this same thing will work with the same logic. For any Boolean function of 3 inputs, you will get these 8 inequalities, and they will not interfere with each other, and you can set the values of w1 to w8 so that you can satisfy it ok, fine.

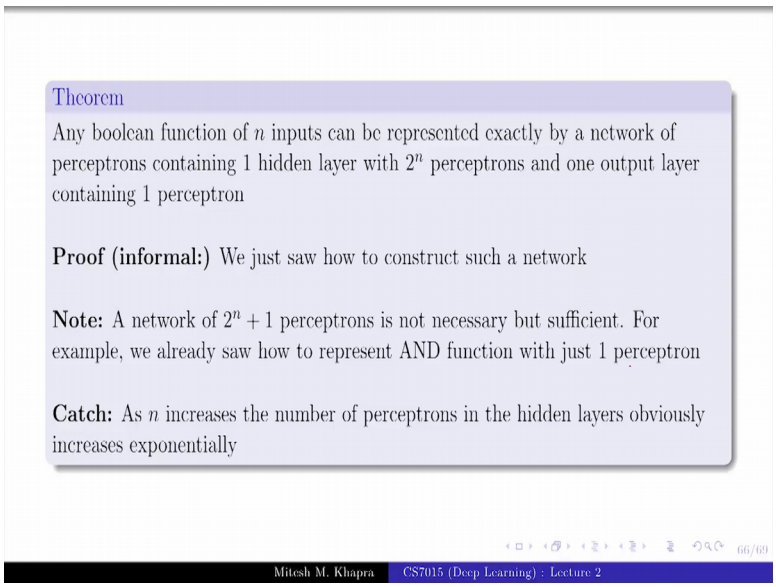
(Refer Slide Time: 08:47)



So, what if we have n inputs?

2 power n perceptrons in the middle layer, right, ok?

(Refer Slide Time: 08:55)

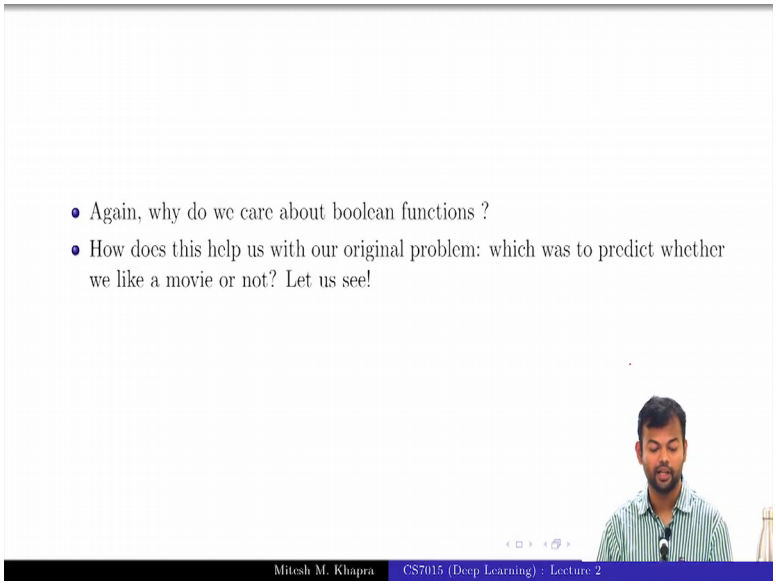


So now here is the theorem, any Boolean function of n inputs can be represented exactly by a network of perceptrons containing one hidden layer with 2 raised to n perceptrons, and one output layer containing 1 perceptron. We just saw an informal proof of this, we just constructed I just gave you the answer it this is how you will get it. Now note that a network of 2 raised to n plus 1 perceptron, is it sufficient or necessary? Or both? Sufficient yes that is what it says, is it necessary?

No, we already saw the AND function which we can just represent using a single perceptron, right. So, it is not necessary, but it is sufficient. So, this is a very powerful theorem if you think of it right. So now, this whole objection right remember this history, and when we have the c I winter when people showed that perceptron cannot handle the XOR function that is for a single perceptron.

If you have a network of perceptrons you can actually have any Boolean function, but what is the catch? As the value of n increases the number of neurons increases exponentially right, but still in theory you could have a solution.

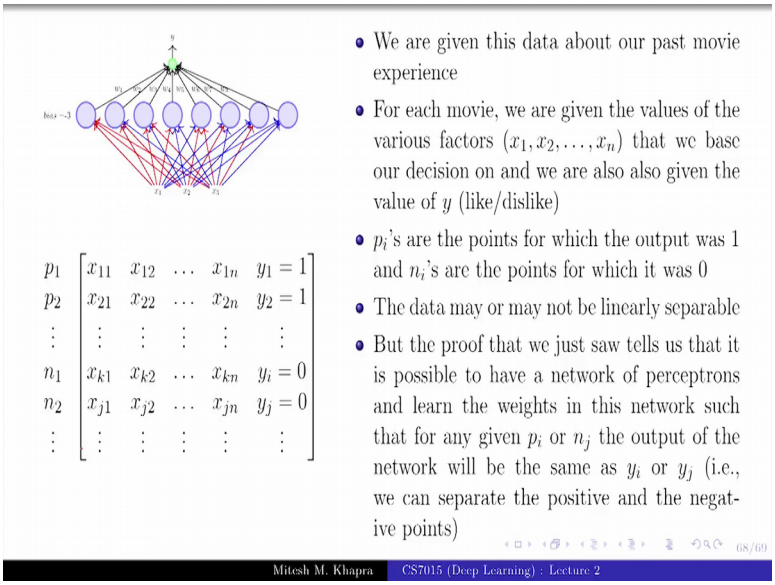
(Refer Slide Time: 10:01)



Now again why do we care about Boolean functions, I keep coming back to this. Why do we care about Boolean functions, because you took this and so, the question that I the question that I want to answer is, how does this relate back to our original problem? Right we know any Boolean function can be implemented, how do we go back to our original problem; which is whether we like a movie or not, right.

And you could see that there is a whole family of problems there, right, whether we like a movie or not, whether we like a product or not, whether I want to go home today or not, yes no any kind of a yes no problem, it is a whole family of problems there.

(Refer Slide Time: 10:34)



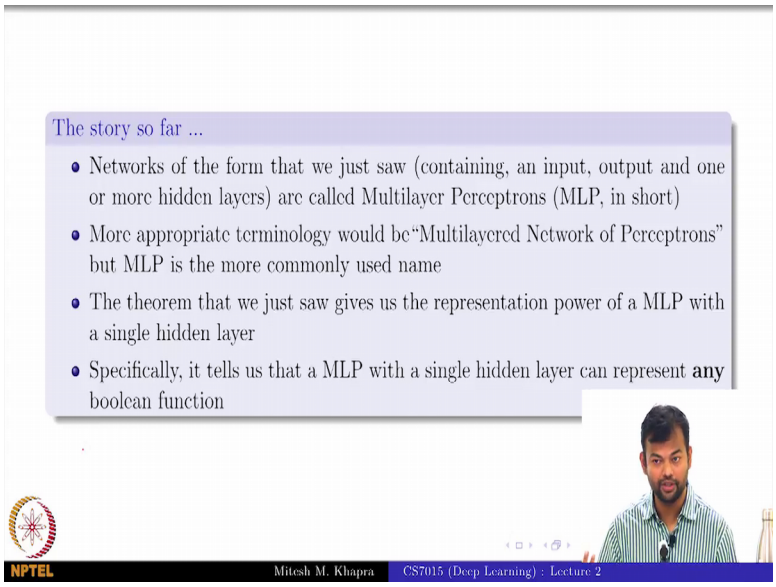
So, let us see so, we are given this data from our past experience, right. So, we are told that this is what the movie looks like. These are the actor's, director's, joiners everything; we also know whether we like these or not.

So, we have a set of positive points and we have a set of negative points, right. And now we want to have a computational model which can satisfy this data; that means, once the model is trained, once whatever algorithm I algorithm I use has converged, it should be able to give me the correct output for a given input. That is what we are interested in, and that is a real classification problem that we are interested in. Now for each movie we are given these factors as well as the decision.

And I said pi's and ni's are positive and negative points. The data may or may not be linearly separable. It is not necessary that the data is linearly separable, those were the goody cases it, but in general that may not happen. But do we worry about it now? No what the previous theorem told us is that irrespective of whether your data is linearly separable or not, I can give you a network which will be able to solve this problem modulo; that it might be very expensive in the number of neurons in the middle layer. But if you keep that aside, I have a solution for this.

And that is why we care about Boolean functions, because many problems we could actually cast to it in a simplistic way, if we ignore the real inputs. And if you even think of the real inputs. Suppose it could take all values between 0 to 1, you can always make it binary, you could say that is the value between 0 and 0.1 is the value between 0.1 and 0.2. And you could make it as small the scale as small as possible right. So, that is why we care about this.

(Refer Slide Time: 12:09)



So, the story so far has been that the network of networks of the form that we just saw, it which contain one input layer, output layer and one or more hidden layers. These are known as multilayer perceptrons, but a more appropriate terminology would be multi-layered network of perceptrons.

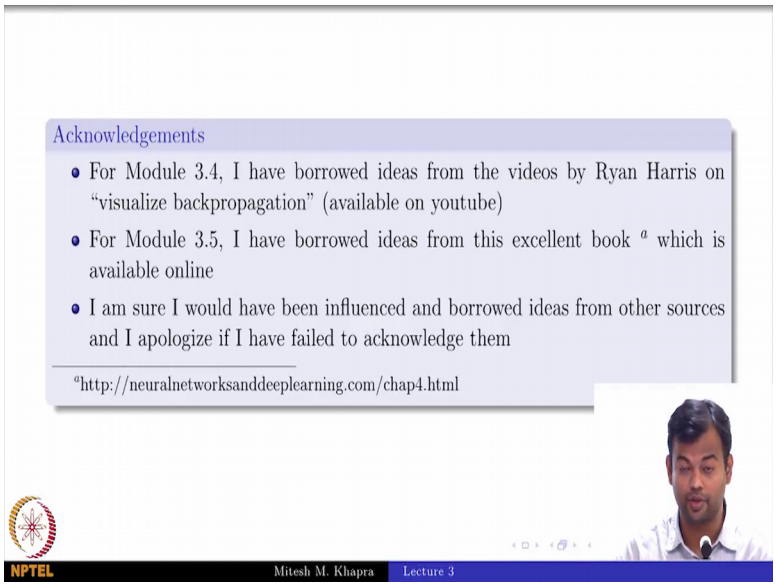
Because the perceptron is not multi-layered; you have a network of perceptrons and that network has many layers right. But generally there is abuse of notation; we always call it MLP which is multi-layered perceptrons. And the theorem that we just saw gives us the representation power of an MLP, and basically tells us that it can represent any Boolean function that we want to deal with. So, that is where we will end this class, and in the next class we will talk about sigmoid neurons.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture – 03
Sigmoid Neurons, Gradient Descent, Feedforward Neural Networks, Representation Power of Feedforward Neural Networks

We are in lecture 3 of CS7015. And today we are going to cover the following modules, we are going to talk about Sigmoid Neurons, Gradient Descent, Feedforward Neural Networks, Representation power of Feedforward Neural Networks.

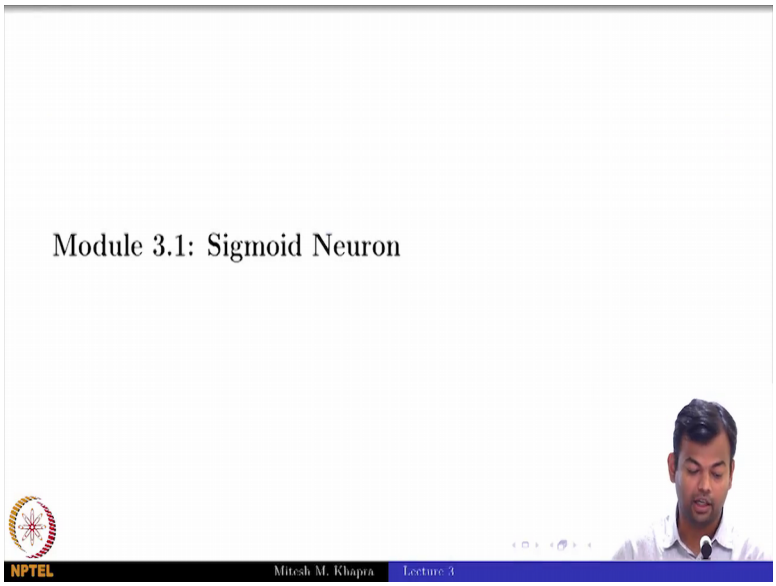
(Refer Slide Time: 00:31)



So, let us start; so, here are some acknowledgments. So, for one of the modules I have borrowed ideas from the videos of Ryan Harris on "visualize back propagation" they are available on YouTube, you can have a look if you want. For module 3.5, I have borrowed ideas from this excellent book which is available, online it is the URL as mentioned in the footnote.

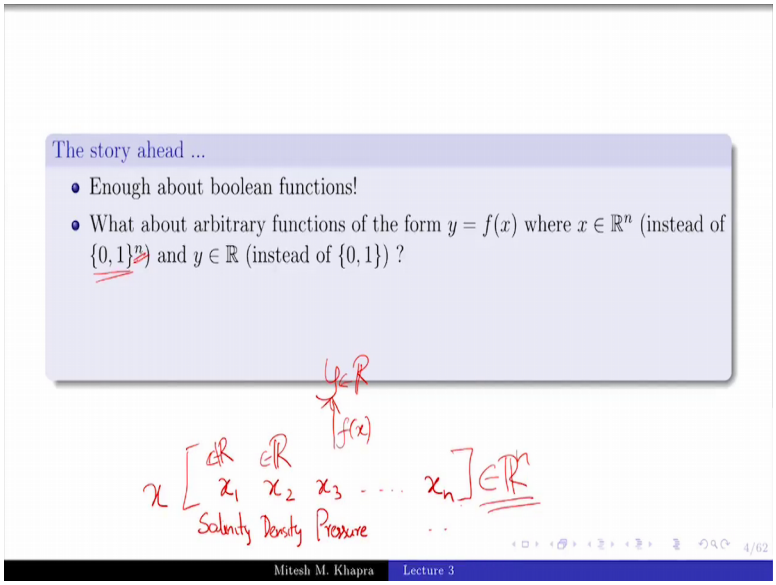
And I am sure I would have been influenced in borrowed ideas from other places and I apologize if I am not acknowledge them probably properly. If you think there are some other sources from which I have taken ideas and let me know I will put them in the acknowledgments.

(Refer Slide Time: 01:02)



So, with that we will start with module 3.1 which is on sigmoid neurons. So, the story I had is that it is enough about Boolean functions.

(Refer Slide Time: 01:10)



Now, we have done a lot of Boolean functions, but now we want to move on to arbitrary functions of the form y is equal to f of x; where x could belong to Rn and y could belong to R. So, what do I mean by this? So, let me just explain this with the help of an example. So, I will again go back to our oil mining example oil drilling example; where we are given a particular location say in the ocean and we are interested in finding how

much oil could I drill from this place, and that is what I would base my decision alright whether I want to actually invest in this location or not.

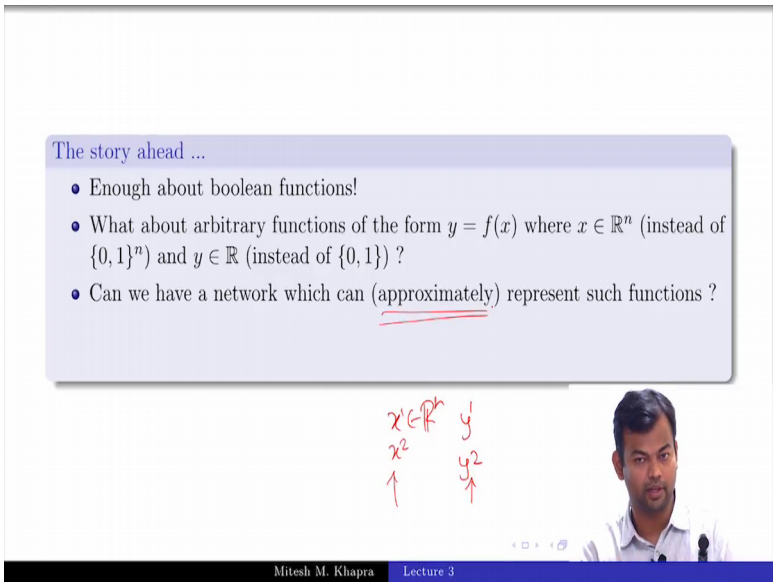
And then what we are saying is that this could depend on several factors. So, we could have x1, x2, x3 up to xn, right where this could be the salinity of the water at that location. So, this could be a real number, this could be the density of the water it is average density. This could be the pressure on the surface of the ocean bed and so on and so forth.

So, each of these values independently belongs to the set of real numbers. So, each of this is a real number and we have n of these. So, together they belong to Rn. So, I can read that I have n such real numbers, and I could just put them in a vector and say that I have a input x which belongs to R raised to n.

So, we have this x which we can say belongs to Rn. And in this particular case, we want to predict y, we want to take this as an input and predictor y. And what is y in this case? You want to predict the quantity of oil that we could mine. So, what does Ry belong to again a set of real numbers, and it could be some gallons or litres or kill of water. So, this again belongs to R. So, these are the kind of functions that we are interested in now.

We want a function which takes us from I am having this x, which belongs to Rn right it is a vector of dimension n, and takes us to a value belonging to R. So, you clearly see that this is different from the case when we had n variables each of this was just Boolean. So, these were only 0 one inputs now we have real inputs, and these are the kind of functions that we are interested in.

(Refer Slide Time: 03:35)

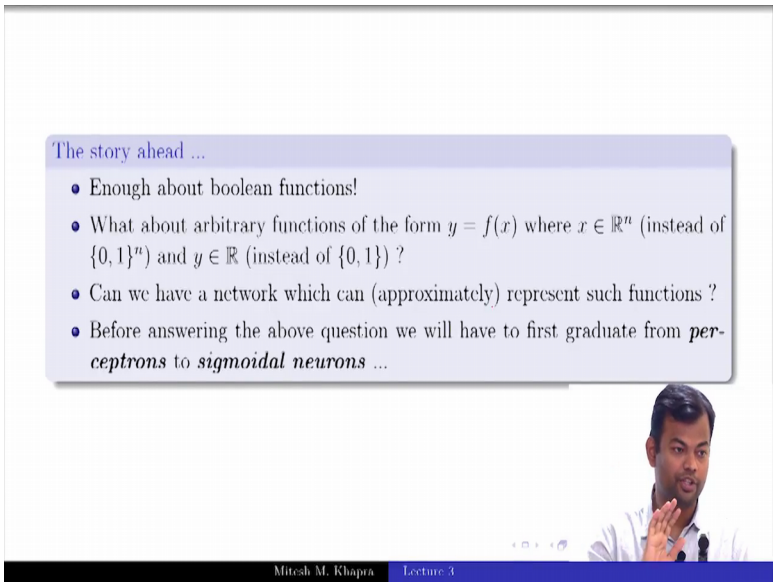


Now, can we have a network which can represent such functions? Now, what do I mean by represent such functions? We already spoke about this when we were doing Boolean functions. So, what do we mean by representing the function? We mean that if I am given a lot of training data, right so, I am given these x1, x2 each of these belongs to Rn. And I am also given the corresponding labels. Now I want a network which should be able to give me the same predictions as is are there in my training data.

So, it should be able to take any of these x's as input, and it should give me the same y I corresponding to it. And I am saying approximately which means I am with some error rate, whether if it is within some to with as long as it is close to the actual value I am fine with it. So, that is what I mean by a network which can represent such functions, is that working definition of representing clear? Right, so, that is a very similar to the definition that we were used for Boolean functions. We had said that we should be exactly be able to get the truth table the network should be able to represent the truth table exactly.

So, that is very similar to the definition that I am using here.

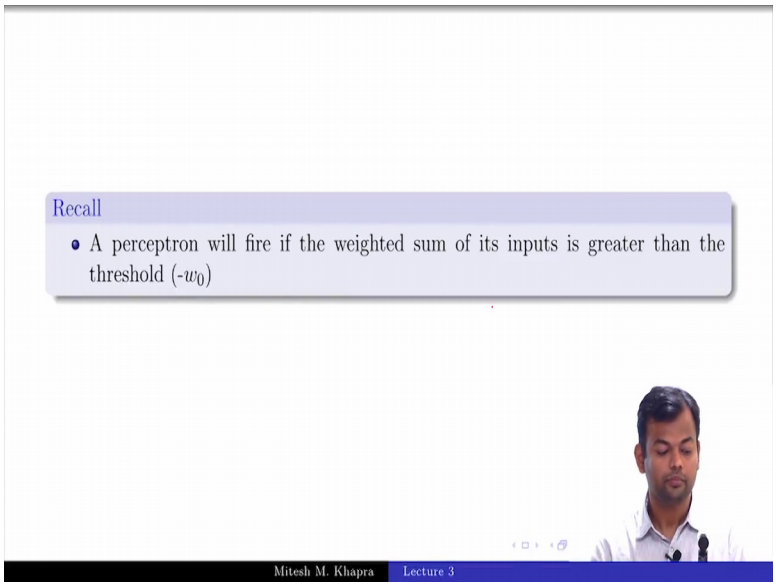
(Refer Slide Time: 09:45)



And then before we do this, right before we come up with a network which can do this for arbitrary functions, we have to graduate from perceptron's to something known as Sigmoid neurons. So, please remember this overall context that we dealt with a lot of Boolean functions, we analyze them carefully and we saw that we could come up with these networks which could represent arbitrary Boolean functions.

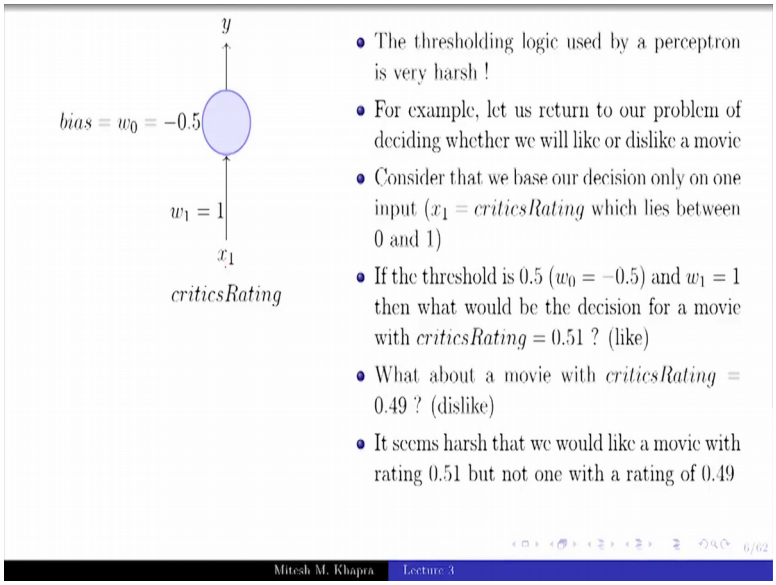
And they could represent them exactly as long as we have one hidden layer. Of course, the catch was that that hidden layer could grow exponentially. Now we want to graduate from Boolean to real functions; that means, you have a real input of n variables, and one or more outputs and you should be able to represent this exactly. So, that is where the transition is where so, that is the story that we are looking for.

(Refer Slide Time: 05:30)



So, let us start so, recall that a perceptron will fire, if the weighted sum of it is inputs is greater than the threshold. Just recall that fine.

(Refer Slide Time: 05:38)

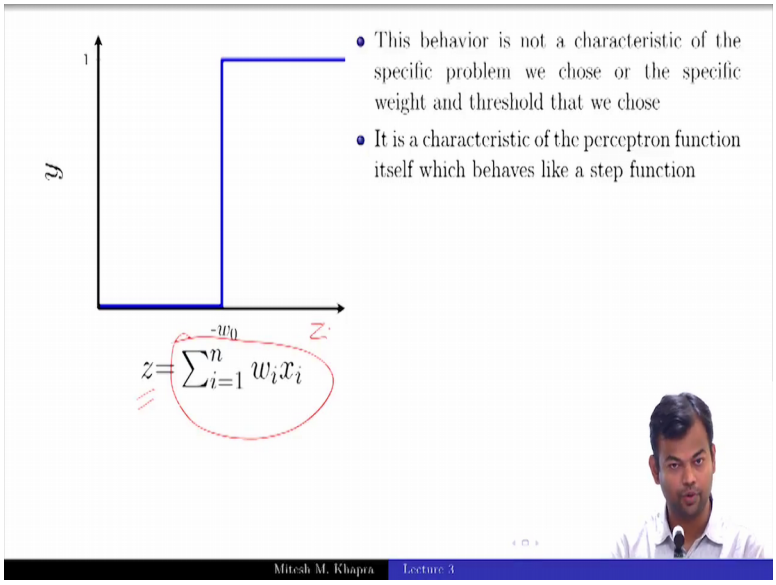


So now, I claim that the thresholding logic which is used by a perceptron is actually very harsh. Now what do I mean by that? Let us see. So, let us return to a problem of deciding whether we like or dislike a movie. That is the same problem that we have been dealing with. And now consider that we base our decisions only on one input; which is the critics rating which lies between 0 to 1. And this is what my model looks like. It takes the input

as the critics rating, I have learned some weight for it, and my threshold is 0.5. What does this mean? It means that if for a given movie the rating is 0.51 will it predict like or dislike like. So, then I should go and watch the movie, what about a movie for which the critics rating is 0.49, dislike. So now, you see what I mean by harsh.

So, both these values are very close to each other, but for one I say I like it, for the other I say that I would not like it. So, it is not how we make decisions, right you would have probably said something equal for both the movies, you would have not given such a drastic decision.

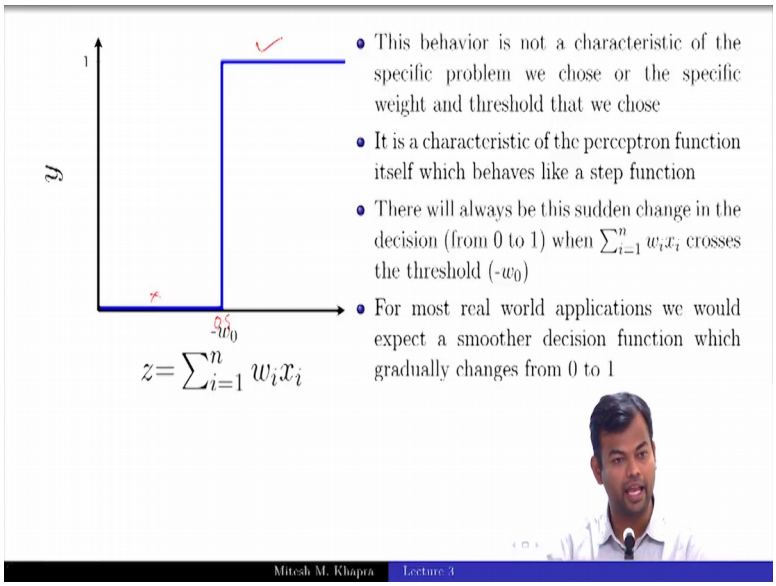
(Refer Slide Time: 06:52)



So, why is this happening? So, you might say oh this is a characteristic of a problem that you have picked up, maybe that is the critics rating which is between 0 to 1 or something, but I want to convince you that this is not a characteristic of the problem that I have picked up. But this is something to do with the perceptron function itself. So, this is what the perceptron function looks like. So, this sum of all the inputs the weighted sum of all the inputs I am calling it by a quantity z. And this is what I am going to plot on the this axis, so, this is my z axis.

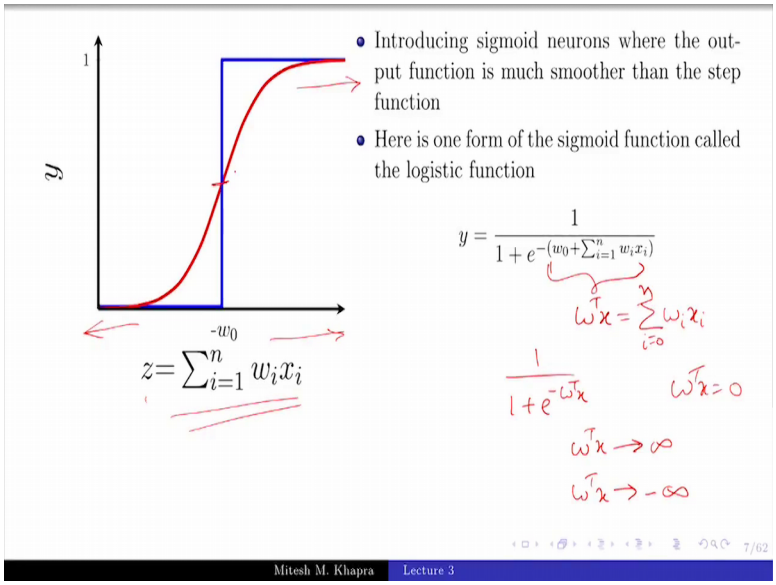
Now, what does the perceptron say that? When this value of z becomes greater than w naught or minus of w naught it will fire, and when it is less than minus of w naught, it will not fire that is what it says. So, this is a characteristic of the perceptron function itself it is going to have this.

(Refer Slide Time: 07:43)



Sharp decision boundary that whenever your sum crosses this threshold you will say 1, and whenever your sum does not cross this threshold you will say 0. So, in this toy example over the movie critics it just happened that this was 0.5. And so, it was saying yes for 0.51, and it was saying no for 0.49. So, this will happen for any problem that you pick up.

(Refer Slide Time: 08:06)



So, to counter this we introduce something known as sigmoid neurons, and this is just a smoother function or a smoother version of the step function, you see that?

How many if you know what a sigmoid function, what is the formula for a sigmoid function? Quite a few good, and here is one such sigmoid function which is called the logistic function. So, remember that sigmoid is a family of functions, these are functions which have this s shaped, logistic function which I have shown here is one such function and the other function that we will see in this course is something known as the tanh function. So, let me just get into a bit more detail with this logistic function.

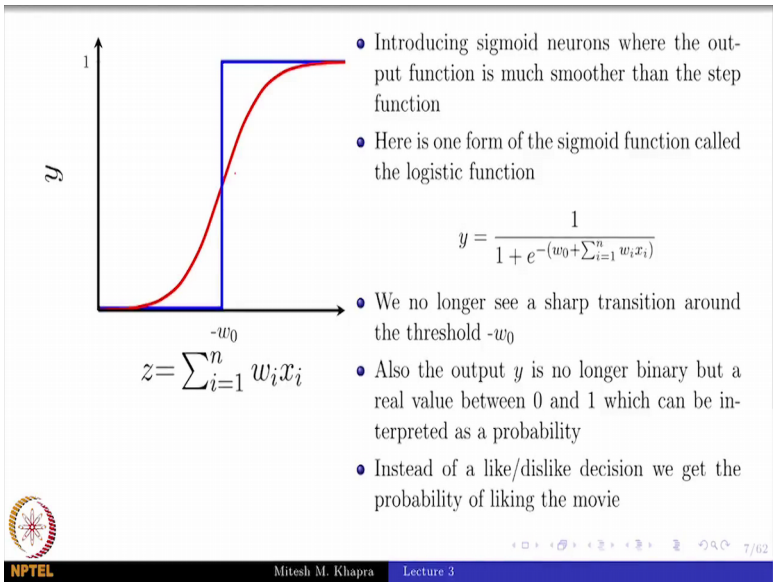
I just want you to understand it properly. So, this quantity here remember we were writing it as w transpose x. Which was summation i equal to 0 to n, wi xi remember this. So now, I am just going to consider this to be 1 over 1 plus e raise to minus w transpose x. Now I am going to ask you some questions and try answering those.

What happens when w transpose x tends to infinity. What happens to the sigmoid function?

One and that is exactly what is happening here as this tends to infinity as this keeps growing. So, remember this axis is z which is the same as w transpose x, right this is w transpose x. So, as it tends to infinity, your sigmoid goes to 1, what happens if w transpose x is minus infinity.

0 and that is exactly what is happening here. And what happens when w transpose x is equal to 0, half. So, this is that value corresponding to half, is that clear?. so, that is how a sigmoid function behaves fine.

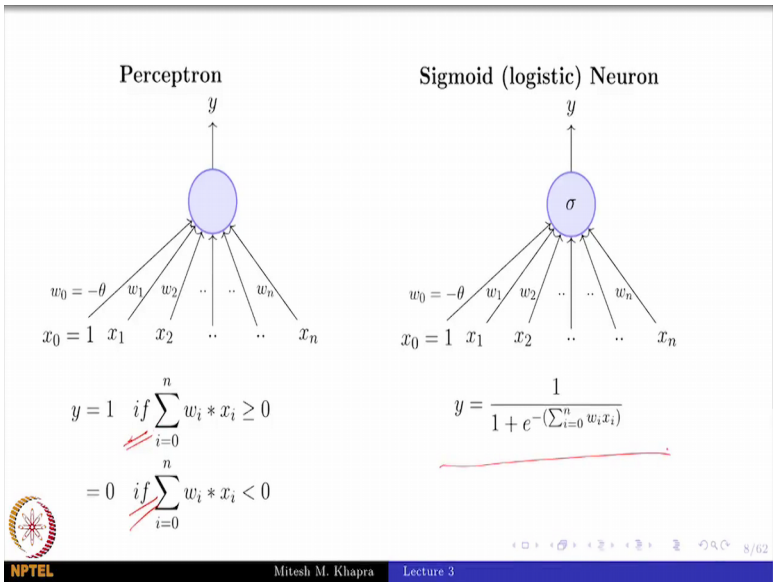
(Refer Slide Time: 10:09)



Now, we no longer see a sharp transition, it is a very smooth function, and the sigmoid function lies between the values produced by the sigmoid function rate, what is the range that they lie between?

0 to 1, what is another quantity of interest that you know which lies between 0 to 1? Probability; so, that is one advantage of sigmoid functions. So now, you can interpret the value given by a sigmoid function as a probability. So, what does it mean in our movie example again? So, it just tells me in those 2 cases, that with 50 one percent probability I like the movie or with 49 percent probability I like the movie. So now, this is not very drastic or very harsh, right I am not saying yes or no I am not committing myself, I am just giving you a number which is proportional to how much I like the movie. So, it can be interpreted as a probability.

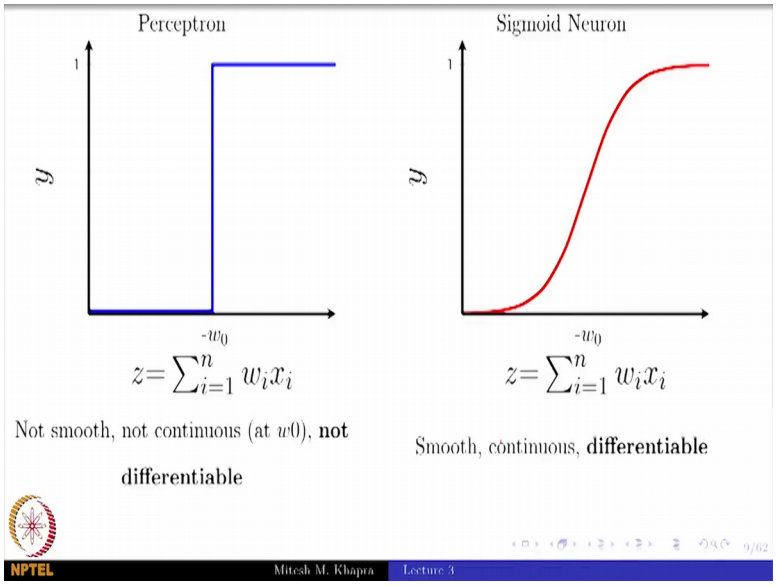
(Refer Slide Time: 10:59)



Now here's the overall picture it. So, this is the difference between the perceptron function and the sigmoid function. So, notice that here we had this if else condition, right which was leading to that sharp boundary.

Now, here we do not have that defence condition, we just have a function which is a smooth function.

(Refer Slide Time: 11:18)



And here is another picture, so, this is not smooth, not continuous and not differentiable, everyone agrees with that? It is not smooth here, right it is not differentiable here

Whereas this is smooth continuous and differentiable. And the contents that we covered today it will be very important to deal with functions; which are smooth continuous and differentiable.

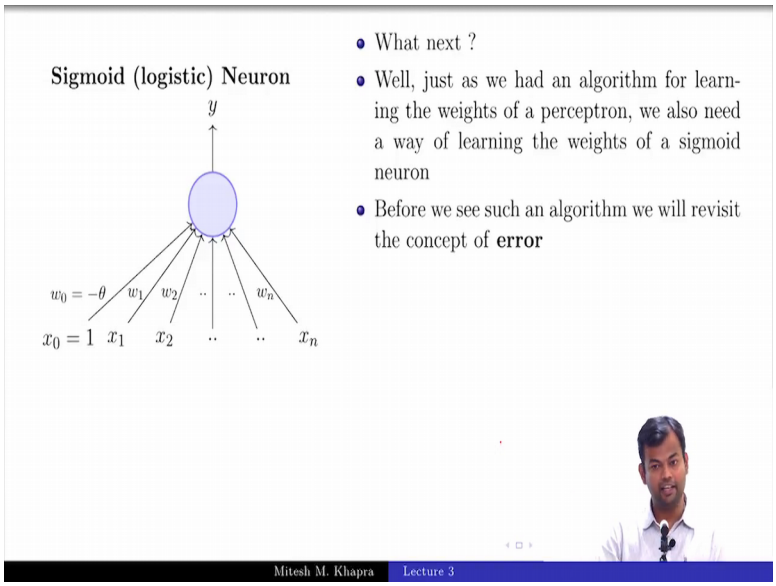
So, for lot of this course calculus is going to be the hero of the course lot of the things that we do will be based on calculus. And in calculus always if you have smooth and continuous and differentiable functions they are always good. So, that is why we want to deal with such functions.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 3.2
Lecture – 03
A typical Supervised Machine Learning Setup

We will start module 2 which brings us to a typical Supervised Machine Learning Setup this is a very important module please pay attention.

(Refer Slide Time: 00:21)



So now we have a sigmoid neuron, we have taken care of the fact that the perceptron was a very harsh function. So, we have a smooth function so, things are fine. Now what next? Where do we go from here? What is my next topic going to be? Yes, a lot of you are giving the right answers; we need to learn these weights. It does not help just to define the function, this function depends on certain weights, and now I need to give you an algorithm which will help you to learn these weights.

Now remember when I talked about perceptrons before giving you an algorithm what did I revisit? What did I talk about the error surfaces. And then I had motivated from there that our goal is to find a set of weights which give us close to give us 0 error in that case. Or in general's speaking generally they should give us a minimum error. They should

help us to minimize the error rate. So, I need to set up that similar story here. So, we will again revisit the concept of error.

(Refer Slide Time: 01:13)

So now in the case of perceptron I had shown you this figure which they were this data is not linearly separable which is obvious. And I told you that perceptron cannot handle this data. But what do I mean by it cannot handle this data? It cannot give 0 error. But what would happen if I run the perceptron algorithm on this, take a guess, what does the perceptron algorithm do?

Fine and I could convergences my condition. I could make that condition a bit loose. What is a valid convergence condition that you would lose here use here? Till almost all my points are separated properly. So, instead of aiming for 100 percent separation, I could have a threshold which says as long as 90 percent of the points are separated; I am fine with it. That looks like a reasonable thing to do.

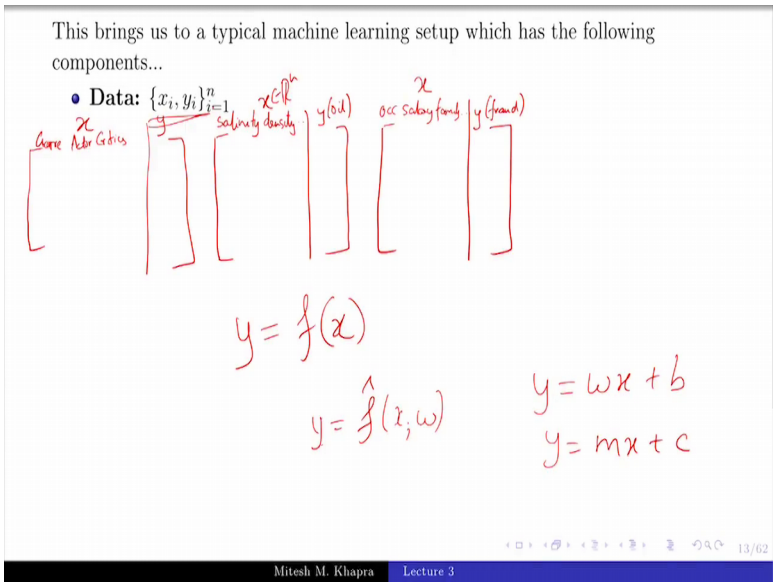
So now, if I run the perceptron algorithm with that condition, what do you think will I get as a decision boundary? Everyone has a picture in mind, ok, let us see, does this match what you had in mind roughly? Of course, there many things possible, but it will basically pass through this. Now, what is happening here? What is the problem? There are 3 blue points which are wrongly classified and 3 red points which are wrongly classified.

But in most real world applications we will find that this line is not too bad, you could live with this error. This is probably 3 out of 30 on both sides which is roughly 10 percent error, unless you are using it when some mission critical applications or in health care where it is a life and death situation or something, in most cases you could live with this right? So, if you are trying to predict whether people will vote for a particular party, if you make this kind of error it would be largely ok unless it is a very close election, but largely it would be ok.

So, we could live with this kind of errors in most cases. So, from now on we are not going to be too optimistic, and if you are going to say that there would be some error, but

my job is to find the weights such that my error is minimized.I want the minimum possible error that I could get is that fine? So, again whatever weights we want to learn, we are going to be driven by some error function and we would want to minimize that error function.

(Refer Slide Time: 03:14)



So, this brings us to a typical machine learning setup which has the following components. So, this perhaps is the most important slide in the course, and I will say this at least for 100 other slides in the course, but at least for now this is the most important.

So, you are given some data xi, yi and you are given n such elements right. So, let me just elaborate on this, and give me I will give you some instances of this. Let me give you some instances of this right; so, one thing we say I already told you was this; so, this is my x and this is my y. So, one example which I gave was about movies. So, this was genre, actor and critics rating and so on. This is one instantiation of this problem. I could also give you another instantiation which was I just told you oil right. So, this is how much oil can I get, and here my factors were salinity, density and so on. There were many other factors.

So, this was my x, again x belongs to Rn, where n is some number integer. And another example could be say fraud detection. So, I have a customer, I am a bank; I have a customer who has bought some credit card, and I want to predict whether he or she would commit a fraud. And I would look at factors like what is his occupation. Maybe

salary, maybe family size and so on, there could be various factors which I could look at. So, here again this becomes an x ok, and you could think of various such examples, right, where you are given an x and you are given a y ok. So, this is the data that you have.

Now, what is machine learning, where does machine learning fit into this. So, we know that there is some relation which exists between y and x. In each of these cases all of us are convinced that there is some relation. So, whether a person would commit a fraud would depend on these factors. It is reasonable to assume, that it is not a very wild assumption. Whether you would find oil at a location would depend on some of these factors and it is related, and similarly for the movie case.

So, there exists some true relation between x and y, such that if I plug this value of x into the relation, it would give me the value of y. There exist a true relation, this true relation could be governed by various things right? It could be governed by physical laws example in the oil mining case, it could be even governed by biological laws; again the marine life in that location and so on. It could be governed by economic law's, it could be covered by psychology right? We do not know why a person cheats what is his function that he is using when he cheats and so on right? So, these could depend on various factors.

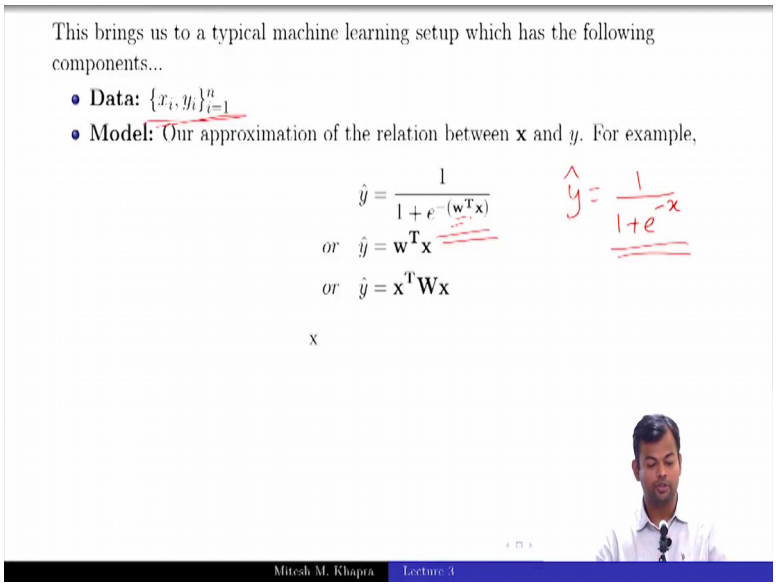
But we all agree that some function exists, hence we get these values for this particular input. For every input we get a certain value. So, there is some function which takes us from the input to the output, we do not know what this function is. We never know in practice it is a very, very complex function is all that we know, we do not know this exact function. If you knew this exact function then there is no problem to solve. We just use that function and you can predict how much oil and all of us can become billionaires.

So, that is not the case. We do not know what this function is. So, then what do we do in machine learn? We make an assumption, ok, we make an assumption that there is some function which takes x to y, and this function is governed by some parameters. And this is our approximation of how the real world works. And now under this assumption, we want to predict the parameters of this model, given the data.

Now, let us take a very simple case where we could assume that y is equal to wx plus b, I am taking this in the scalar single dimensional case. Now how would you estimate the

values of w and b, oh come on. If I give you 2 data points, you can estimate the value, or should I write it that would jog your memory, right, this is how we all learn right. So, m and c you can estimate if I give you 2 data points. So, that is the simplest case, now we will make similar assumptions, but more complex functions and just as we could estimate m and c from the data, we would expect to estimate w's also from the data. So, that is what the machine learning setup is so, let us see.

(Refer Slide Time: 07:48)



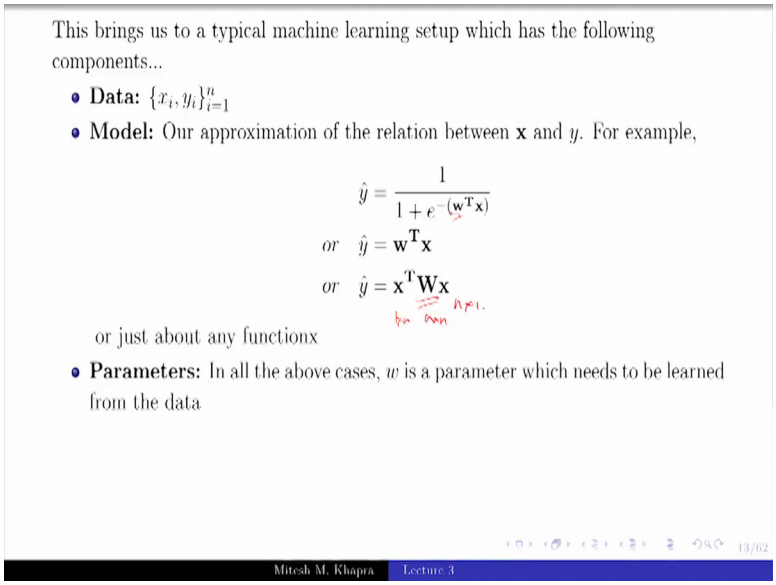
So, the model when we talk about a machine learning model, it is our approximation of the relation between x and y. And we are free to make any such approximation. So, I could say that this is what I think is the relation between y and x, and which is governed by some parameters w. Do you know what is this function? Have you seen this before? No, not sigmoid.

Which model is this? logistic regression. Ok, but I could also have made a different assumption, I could have made this assumption. What do I get? Linear regression, ok, please note that this error on the slide, ok, and I could make some other assumption; I could assume that y is actually a quadratic function of x. I am free to make any assumptions, the only thing I need to ensure is there is some parameter involved. What is wrong with making this assumption? If this is valid, is this also valid? If not, why? There are no parameters.

So, no not for any x we will get the we will it will depend still depend on the value of x. If I plug in different values of x, I will still get a different output, there is nothing to learn. What do I do with all the data that I have. There is absolutely nothing I can use it to learn, I have just said that y is equal to 1 over 1 plus e raised to minus x, I can ignore all the data that you had given me, whenever you give me a new x, I will just plug it into this formula and tell you the answer. And that is bound to be wrong because I have not adjusted this formula.

Now, once I put in the w's, it gives me this degree of freedom where I can now adjust the formula, I can learn the w's in such a way that my predicted y's are very close to the actual y's. So, that is why we need always need a parametric form. Of course, there is nonparametric learning also, but I am just saying in this supervised setup, we are thinking of models whether you have parameters. So, you have the data, you have the model, the model always has some parameters.

(Refer Slide Time: 09:50)



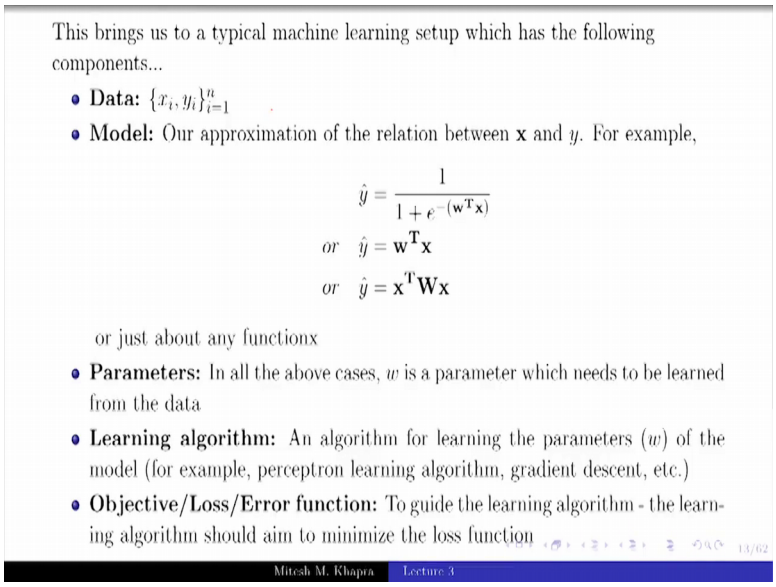
In all of the above cases, w is a parameter right, either the small w which is a vector, or the capital W which is a matrix right. So, notice that this is a matrix, this is one cross n, n cross n and n cross 1.

Now, how do we learn these parameters? That is the question that we need to answer. How do we learn these parameters? We are convinced about 2 things that we never know the true function. So, we come up with an approximate function, and we have to insert

some parameters in that function, so far good. Now I have to be able to learn these parameters.

Now for learning these parameters, we have something known as an learning algorithm. So, did you see any learning algorithm so far? Perceptron learning algorithm right. So, you already saw the perceptron learning algorithm, and it was able to learn the weights for a perceptron.

(Refer Slide Time: 10:43)



There are various such algorithms, today we are going to learn one such algorithm which is gradient descent.

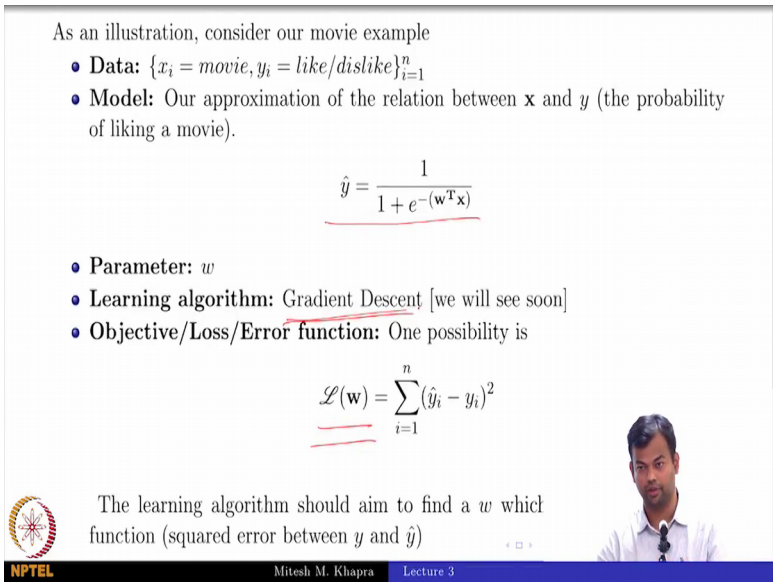
Now, any kind of learning what is it driven by? Learning is driven by errors, objective function. And so, the analogy which I like to give is, suppose you are trying to learn trigonometry. you have a chapter that is your training data, the chapter has a lot of formulae, that is your training data. Now what is your objective? There are 2 objectives, actually I will tell you the easy objective, the training time objective is that once you read to the chapter a few times, at least whatever formulae are given in the chapter, you should be able to produce the correct output for that. So, if I ask you what is sine square theta plus cos square theta, you should be able to answers them. And you should be able to give me the correct answer.

So, in other words you are trying to minimize the error on the training data. Whatever training data you have which is the chapter content, you want to make 0 errors in anything which is given in the chapter. That is your training error. Of course, there is also something as known as a test error; because after you have learned the chapter. I will give you an independent set of exercises, which might contain questions which are not seen in the textbook earlier. So, you would have seen sin square theta plus cos square theta.

But now I could ask you some other formula, which you should be able to give me answers, if you have learned properly right. So now, right now we are just talking about the training error; that means, getting all the formula in the chapter correctly and our chapter is actually the training data which is given to you, this is what we are reading. So, this always going to be driven by an objective function and our goal is just as we wanted to minimize the errors that we make on the formula given in the chapter.

We want to minimize the errors on the training data; is this set up absolutely clear to everyone? Anyone who does not understand this has any doubts? So, this is something this is the same framework that we will use again in lecture 18, 19, 20 and so on to explain some more complex models. So, you have to absolutely make sure that you understand this. It is not very difficult, but just make sure you understand this fine. So, let us concrete at this a bit more.

(Refer Slide Time: 12:44)



And we will consider our movie example and try to fit that into this framework. So, what is our training data there? They are given movie comma like dislike. And when I say movie, I am just using a shortcut it is actually all the details of the movie. Genre, actor, director, critics rating and so on, that is our input, and our output is the like, dislike value.

What is a model that I chose? What is a model that I chose? I do not know what is my true relation between when I like a movie or not, but I made this approximation, that this is how y depends on x, and I made sure I introduced some parameters there. I could have chosen some other functions also, but I chose this. Now the parameter is w this should be bold W, the learning algorithm that we are going to use is gradient descent which we will be seeing soon.

And what is an objective function here? Can you tell me a formula? So, we have been talking about it in terms of English, that I should be able to get predictions which are as close to the true prediction. Can you put it into a formula, y i minus

y i hat where hat is the prediction. This is the prediction. So, that is y i minus y i hat that should be minimized, is that fine? Whole square of that. So, why do you square it? So, that is correct.

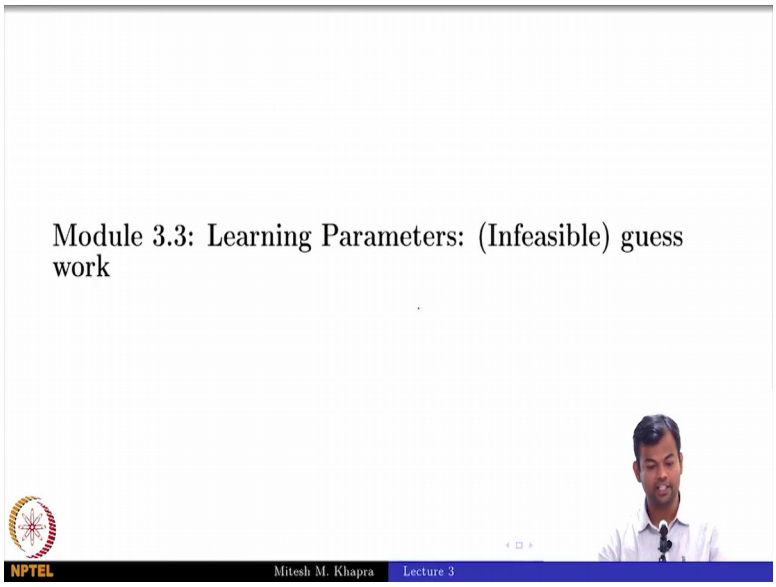
So, for all the training points, n training points, I want to minimize the square difference between y i the true prediction, and the prediction, sorry the true value and the predicted value, is that fine? And why do I use squares, differentiable is 1, the other thing?

The positive errors and the negative errors should not cancel. So, it would be happen that on some movies, I make a positive error of 0.5 right? That means, the actual label should have been 0.5 and I gave 1. On some movies I make a negative error of 0.5, and these would cancel each other, and I will get the false impression that I am making 0 errors. But once I square the values the negative values also become positive: so, this cannot happen right. So, that is why we always use the squared error function, and also this is differentiable which is more important.

Now, the learning algorithm should try to minimize this particular quantity, ok? So, this is a typical machine learning setup, almost any supervised learning problem that you see you could cast it in this framework, change the y hat function appropriately, change the parameters appropriately, maybe use a different learning algorithm depending on the

problem that you are trying to tackle. And you should be able to fit it into the same thing, is that fine? Ok, at least for this course everything that we do we will largely be able to fit it into this framework.

(Refer Slide Time: 15:28)

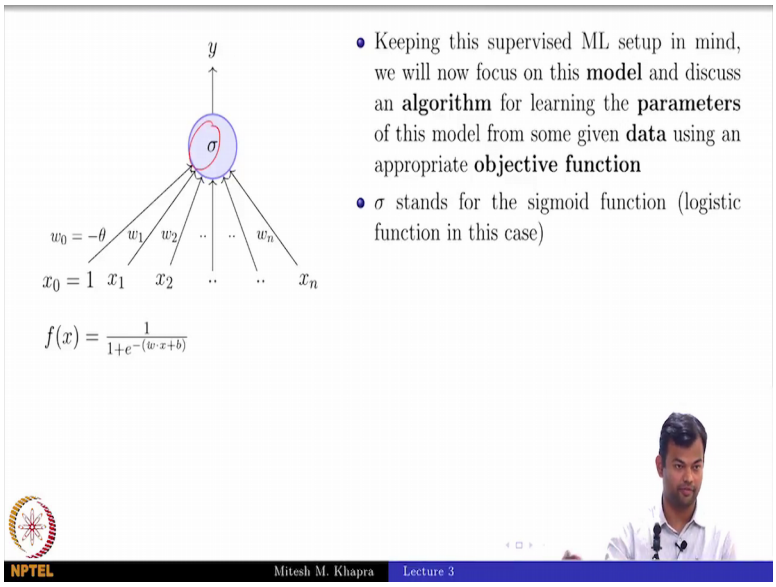


Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 3.3
Learning Parameters: (Infeasible) guess work
Lecture – 03

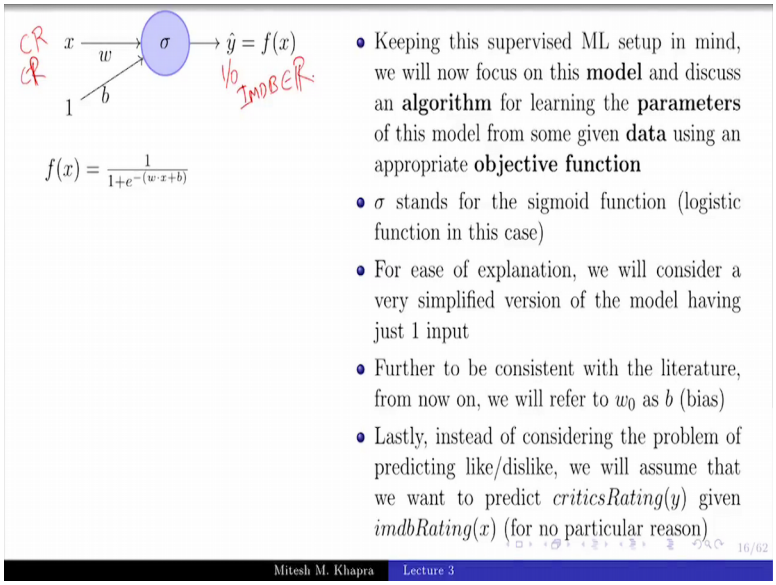
In this module we will try to learn these parameters, and initially we will try to learn them by guesswork, and I will show that that is actually infeasible; that is why we need a more principled approach.

(Refer Slide Time: 00:26)



So, we will keep the supervised machine learning setup in mind and now we will focus on this model, and discuss an algorithm for learning the parameters which are w and b. Given some data using a giving appropriate function objective function. So, that is what we are going to focus on.

(Refer Slide Time: 00:47)

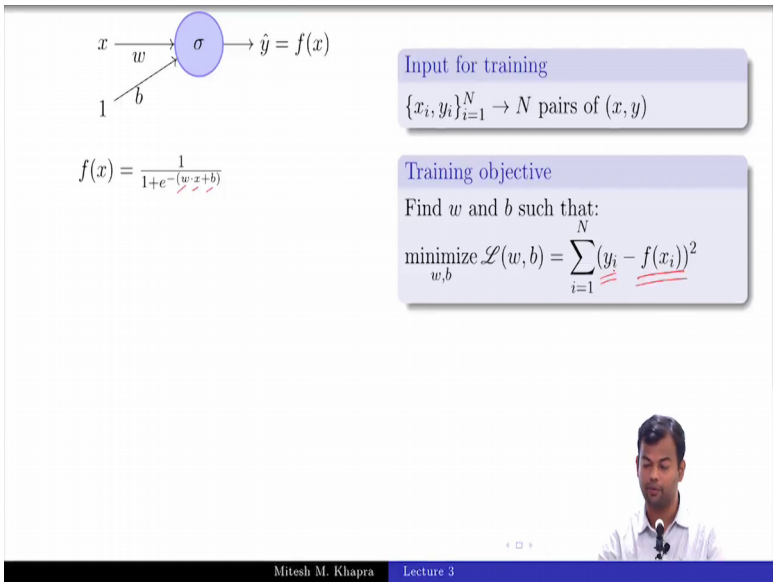


Now, sigma here stands for the sigmoid function, the logistic function in this case when this sigma is actually the logistic function. And now I am going to simplify this further. So, that it helps us to do a better analysis, I am just going to consider the case; where I am just in one input and the bias, ok. And also following the normal terminology in the literature, this w naught from now on I am going to call it b. Because that is the normal convention b stands for bias.

So, I have two parameters w and b, which I need to estimate ok, and this is my model for the movie example. And the other change which I am going to make is instead of deciding whether I like or dislike which is 1, 0; the setup that I am going to work with is that I am giving the critics rating, and I want to predict the IMDB rating. So, I am given a real value, and I also want to predict a real value.

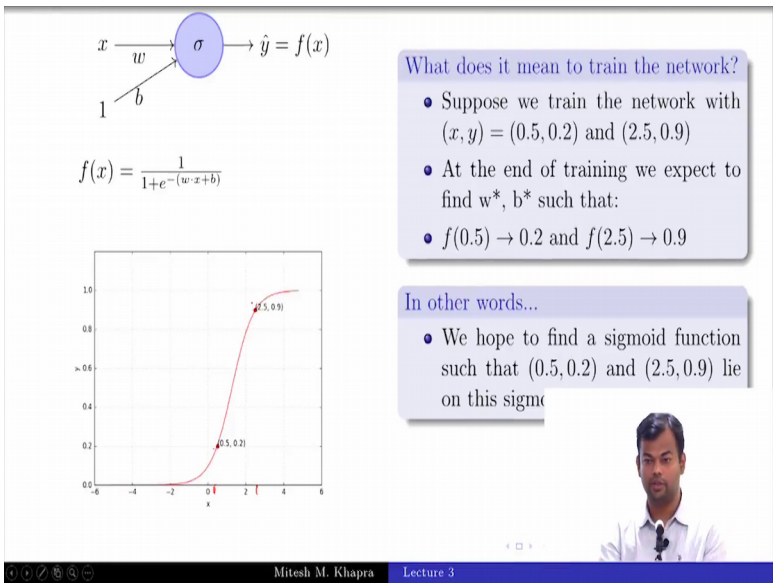
For no particular reason this just makes life easier for me for explaining a few things, but the same thing or the same algorithm would also hold if you add a binary output right, and you will see that later on in the course. So, here is a setup, clear? We just have two parameters w and b, and we are going to assume that y belongs to real numbers it is a IMDB rating and x also belongs to real number; it is a critics rating.

(Refer Slide Time: 02:05)



Now, let us see what we are given as training is a set of points, we are given some n training pairs, and now we understand what this means; that means, for a lot of movie I am giving the critics rating and I am also given the true IMDB rating for them. Of course, in the two variable case this does not make much sense, but just bear with me. And now the training objective is such that whatever my function predicts which is a function of w, x and b that should be very close to the true output that I know, this is the function that I want to optimize. Now let me ask you this.

(Refer Slide Time: 02:37)



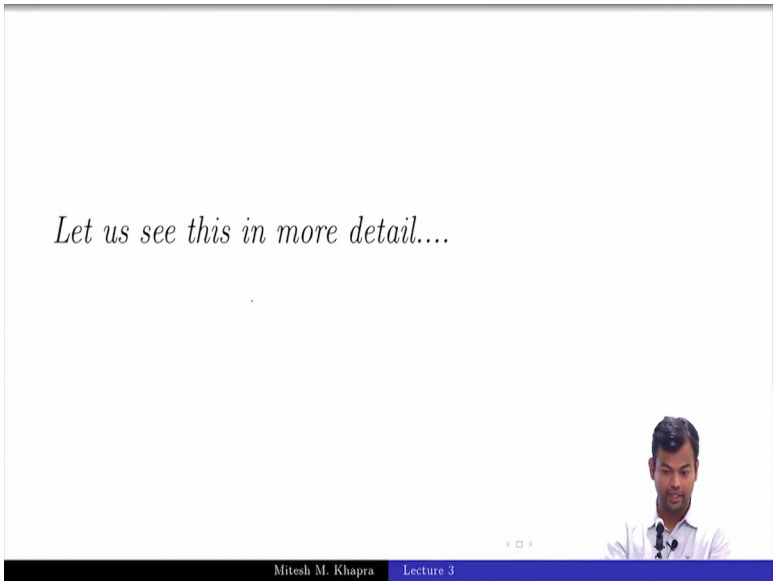
I am trying to tell you that I am going to give you an algorithm for training this network. Suppose I have trained this, with 2 data points, 0.5 comma 0.2 and 2.5 comma 0.9, right. At the end of training, I will give you some values of w and b. Let us call them w star and b star; these are the final values of w which I have given w and b. What do you expect from these values?

What do you expect at the end of training if I say, now, the network has learned, what do you expect? You are still going to the test case I am just talking about the training still. We expect such that what happens if I plug in at the end of training, if I plug in the value 0.5 here what should happen?

0.9. So, this is what you expect at the end of training, if you plug in the value 0.5 it should be very close to 0.2 the output and if you plug in the value 2.5, it should be very close to 0.9.

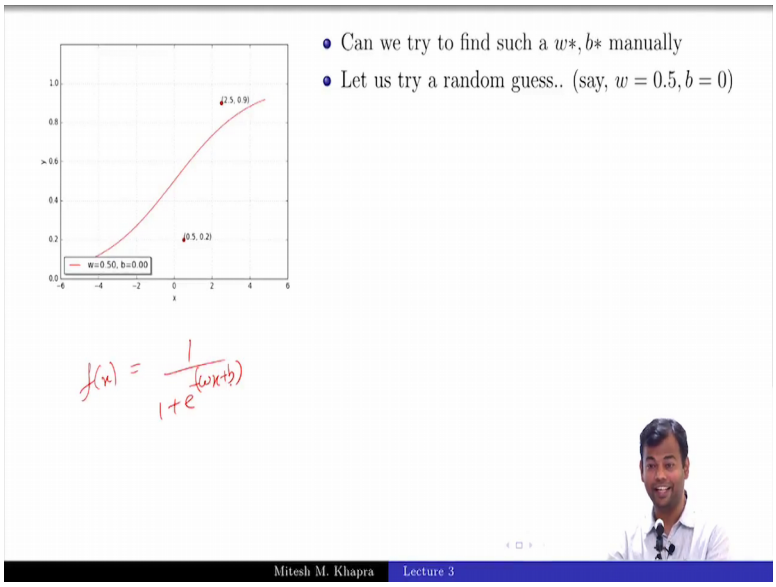
This is exactly what you expect and this is what training means ok, fine. In other words we hope to find a sigmoid function such that these two points lie on that function. Can you imagine a geometric picture for this? What would happen actually? How many if we can imagine it? Ok, how many of you get it now? This is what will happen right. So, you will get a sigmoid function such that these two points lie on that fair ok, and that exactly means that when I plug in this value, I will get this value and when I plug in this value I will get this value right. So, that is what it means.

(Refer Slide Time: 04:09)



So, let us see this in more detail.

(Refer Slide Time: 04:10)

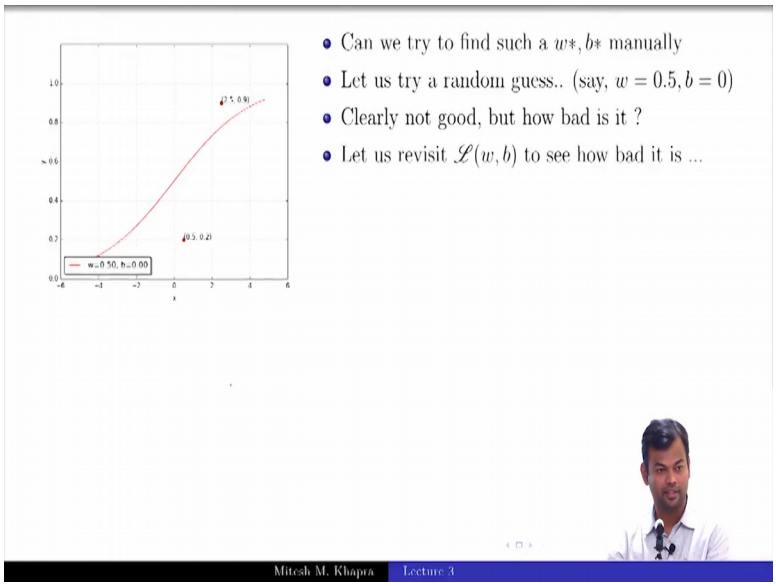


And now what we will do is our quest is for this w star and b star, I will try to find this manually, I will do some random guesswork and try to find this because I do not have any clear principle algorithm for finding it as of now. So, I will just use some guesswork. So, I will give my initial guesswork as w equal to 0.5, b equal to 0 for no reason, I just picked up some values right. And this is what the function that I got, what does this mean? This function an error; so the sigmoid formula should be here we should have this sigmoid formula here.

So, is this a good are you happy with the solution; if I give you?. Are you happy with this solution? Is this good, bad, ugly? Has to be something, bad, we will not call it ugly, ok. So, why is it bad? It is not passing through those points, I will ask you a question how bad is it? Can you assign a number to it? We are always good at qualitative stuff, but quantitatively can you tell me a number, how bad is this? Can you tell me a way of finding how bad this is? I already told you in detail how to find that how bad it is.

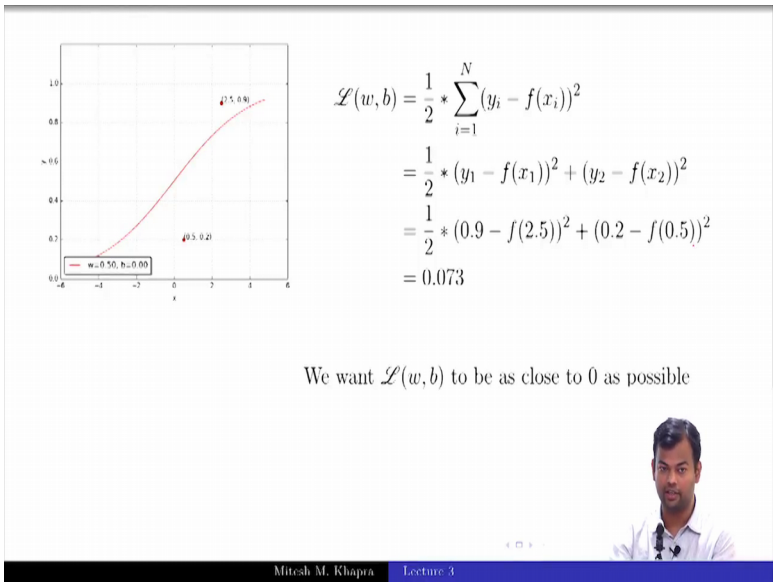
The loss function; right?

(Refer Slide Time: 05:27)



We have the loss function, let us see that again and see if we can find out how bad this is.

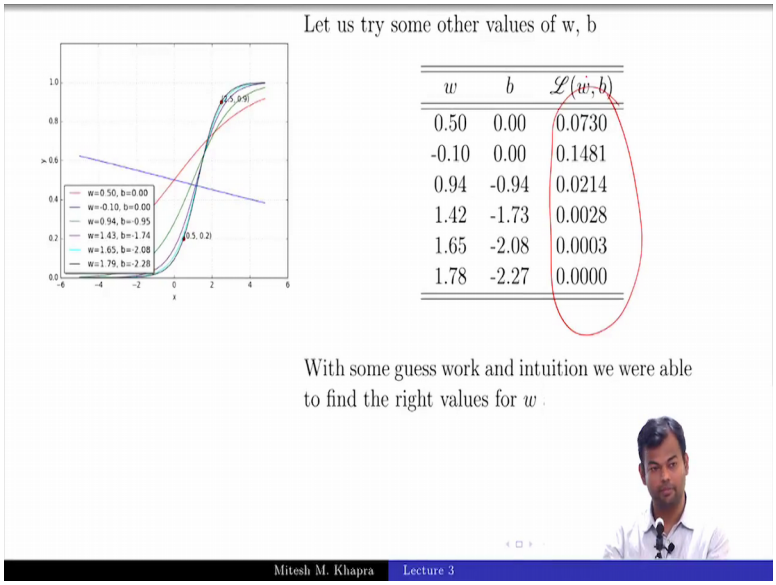
(Refer Slide Time: 05:32)



So, this is what my loss function is, ok and I have 2 data points, I will just expand it out, fine? Now I will plug in the values I know this is 0.9, and I will compute the value of f 2.5, I will plug in this and I will plug in this ok and this is what I get. So, this is how bad it is. What did we actually expect it to be in the good case? 0. So, this is not 0, this is 0.073; so now we have a quantitative handle on how bad this is, ok. So, let us keep this

in mind and let us try to continue guessing. So, we want the loss function to be asked close to 0 as possible; we are not there yet.

(Refer Slide Time: 06:14)



So, then I make a different guess, I say let me try minus 0.10, 0.00 what happened now? Is it now good, bad, ugly?

Now let us call it ugly, right? So, it is worse and how do I know it is worse? Because I plugged it in to the loss function, and I got a value which is greater than the value at which I was. So, I clearly know this is bad.

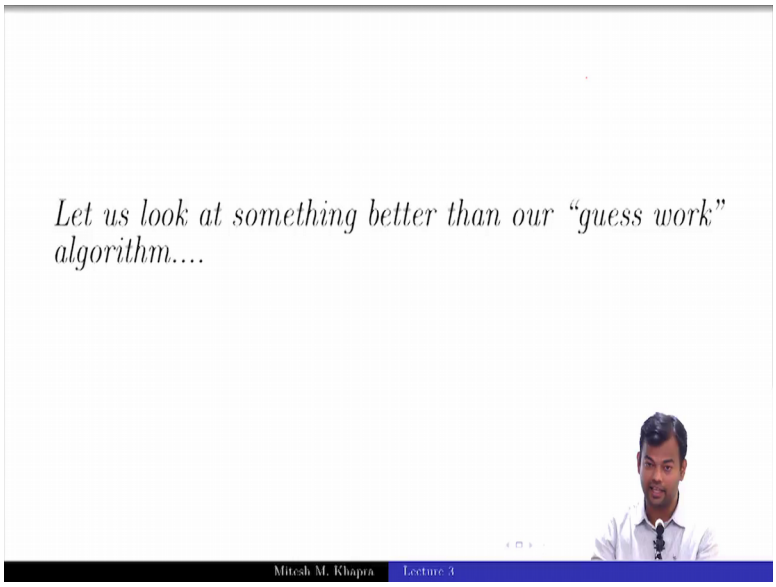
So now this is how my mind is working, right oh I as far as w was positive things looked at least I was close to 0 in the first decimal. Now when I made it negative that does not look good; so, let me just keep it positive and keep increasing it right. So, I saw 0.94 and I also tweaked the b of it I have done complete random guesswork, right? Now what happened? Good, bad, ugly?

Better ok, now what will you do? What would your next case would be? Make w even more positive; perhaps that would help, and be even more negative and so on.

I can continue in this manner and actually get close very close to the solution. So, I can do this guesswork and find these values. But it is still an educated guess, right I am not guessing in the dark, this is what is helping me drive towards those guesses, and I am just looking at these values and making an educated guess right? And that is the educated

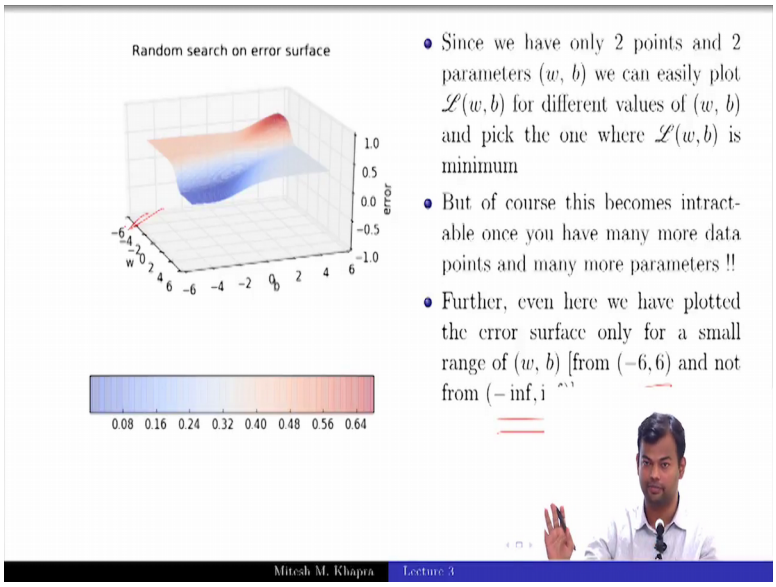
guess which I took that probably making w even more positive would help, but this is still brute force in a sense, right this is not something that you would want to do when you have 100, 1000 parameters and so on right? and 1 million data points and so on.

(Refer Slide Time: 07:42)



So let us look at something which is better than our guesswork algorithm ah. So, we are not there yet actually on the next slide I am still going to talk about the guesswork algorithm.

(Refer Slide Time: 07:53)



And eventually we will get to something which is better than the guesswork that, ok.

So, since we have only two points and two parameters, what I can do is I can take all possible values of w and b, right that is what I was trying; I was picking up some values of w and b. Why just pick some values of w and b? I will pick all possible values of w comma b, right? And I will fix the range, I cannot fix pick it from minus infinity to infinity, but I will pick a range I will say from minus 6 to 6, let me try all values of w comma b compute the loss and plot it, right? Let me tell something about this error function, because this is going to stay with us for quite some time.

So, what you see here is something like a flying carpet? This is colour coded, red is bad, red are the places where the error is high. Blue is good, blue are the places where the error is low, darker the shade of blue, lower the error, darker the shade of red, higher the error. So, in particular if I look at this point, what has happened is, I have taken the corresponding value of w comma b right, which is say minus 4 comma minus 1, right something like that.

I have plugged that value into my loss function, and I got this as the loss function. This has the loss value, and that is what I have plotted for all values between minus 6 to plus 6 and minus 6 to plus 6, for w and b. So, everyone understands how I have constructed this error surface.

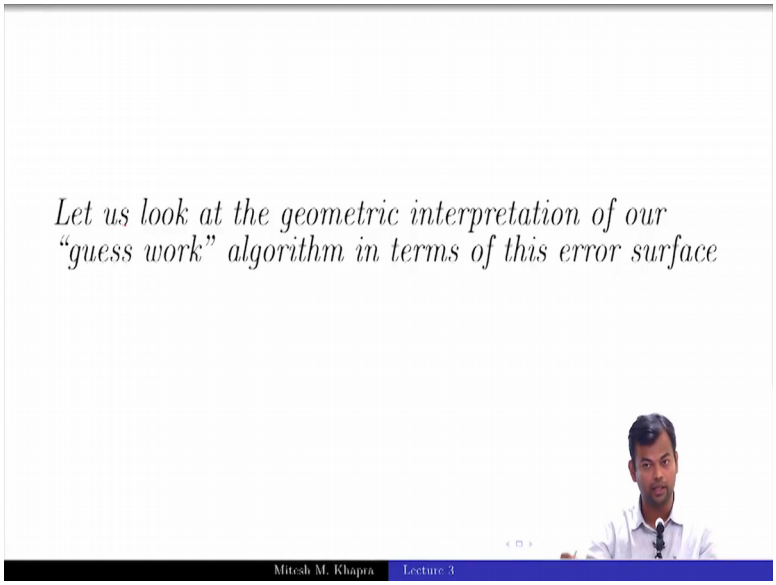
Now, this of course, becomes and now what I can do is once I see this error surface, I know how good this is the point where I need to be, this is the darkest ah shade, and this is where the error is the lowest. So, I can just pick a w comma b value which lies there. This is fine, for this toy example where you just have two parameters. But this becomes untractable, once you have more data points and many more parameters.

And that is what happens in most real world applications, right. So, this is not a feasible way of going about things, right? And here again note that I have only taken the range from minus 6 to 6, I do not even know what will happen if I have to look at all values of w comma b, right maybe there was something outside here, right which was even more lower error or something, right. So, I do not really know that.

So, I cannot really use this. So, I need something better than this plotting the error everywhere and finding it order; that is pure brute force. Or surrogate to this was the guesswork algorithm, but which is again something we cannot do for if you have large number of parameters.

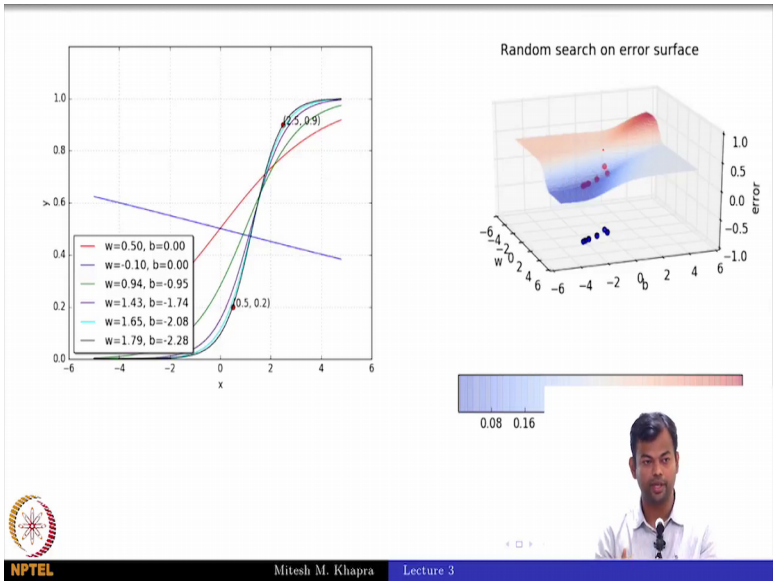
So, everyone gets this? That this is a way of finding the solution, but this is not feasible, right that is the only point I am trying to make.

(Refer Slide Time: 10:24)



And we look at the geometric interpretation of what was actually happening in the case of the guesswork algorithm with respect to the error surface.

(Refer Slide Time: 10:33)



So, I had chosen some values of w comma b, the first value that I chose actually gave me an error of if you remember it was some 0.073 or something like that right. So, that is the point then I decided to take a very random guess, and my error actually increased. So,

you see that I am actually climbing up on this error surface, I have gone from a slightly darker shade of blue to a lighter shade of blue right. And then I corrected myself and then kept moving in a direction where I was going towards the darker and darker shades of blue.

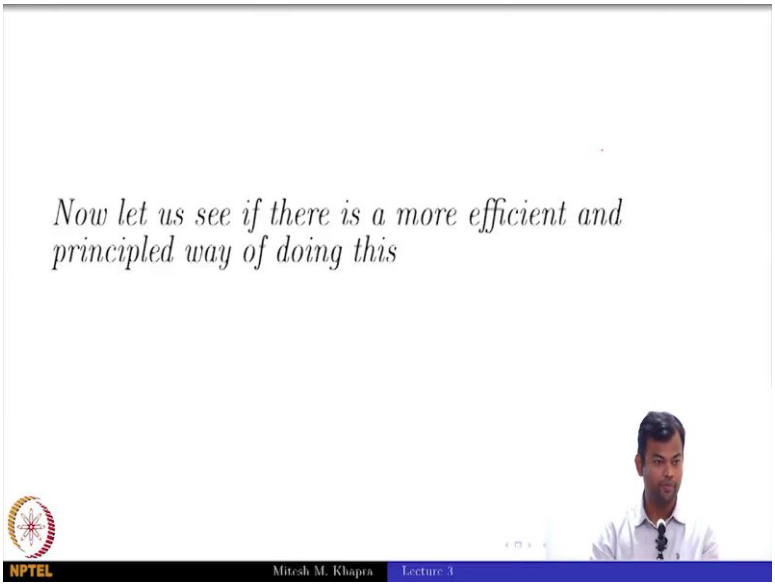
So, what I was actually doing is; I was trying to traverse the error surface and land up in the good regions which were the dark blue regions. Now, what I want to do is I want an algorithm which will allow me to do this in a principled manner which is neither brute force nor guesswork. So, that is what we learn in that module.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module - 3.4
Lecture - 03
Learning Parameter: Gradient Descent

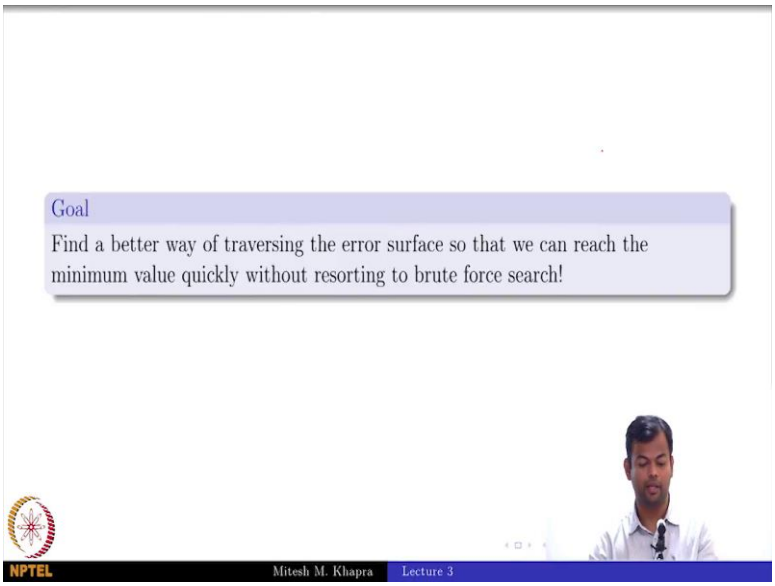
In this module, we will talk about Gradient Descent.

(Refer Slide Time: 00:22)



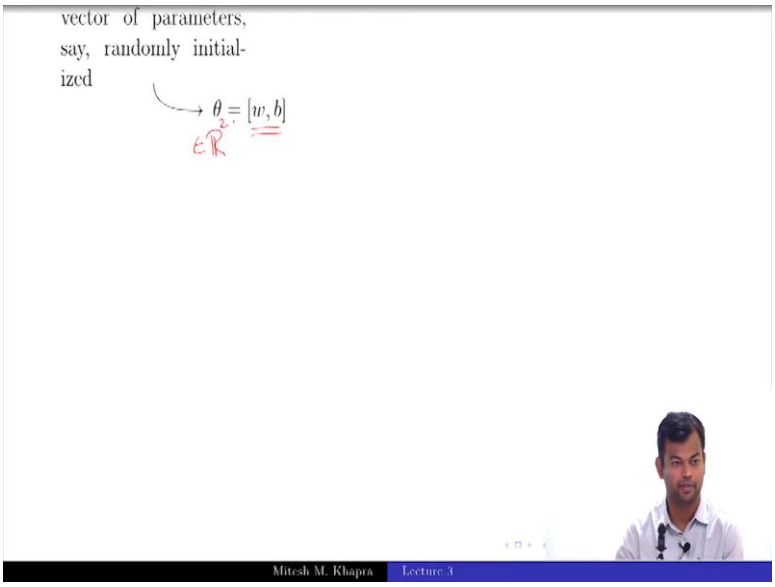
So, what we want to do is find a more efficient and principled way of navigating the error surface.

(Refer Slide Time: 00:30)



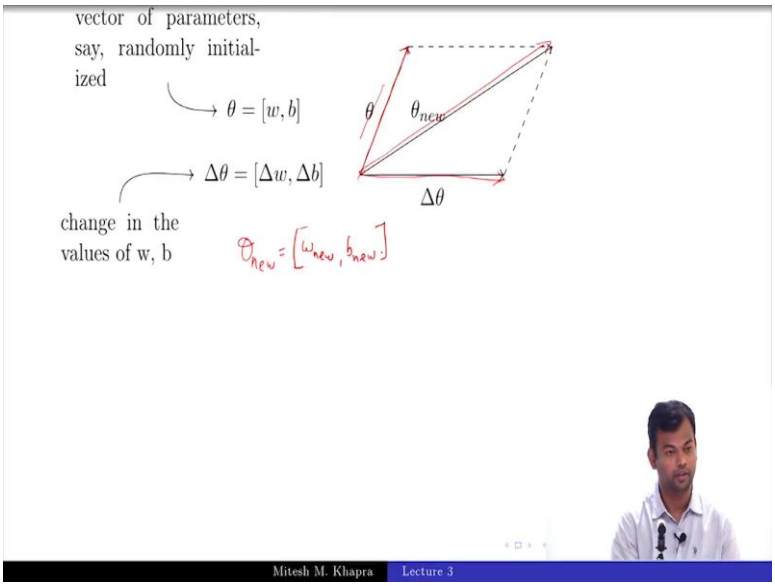
And, the goal is to find a better way of doing this.

(Refer Slide Time: 00:35)



So, let us start by setting up things we will define some notations and some parameters and so on and from there on we will try to come to the algorithm, ok. So, my parameters in this case were w comma b, what I am going to do is I am going to put them into an array or a vector, right and call that vector as theta. So, theta is the vector of parameters and theta belongs to R; R what? R2, right? There are two parameters here. So, it is a two dimensional vector.

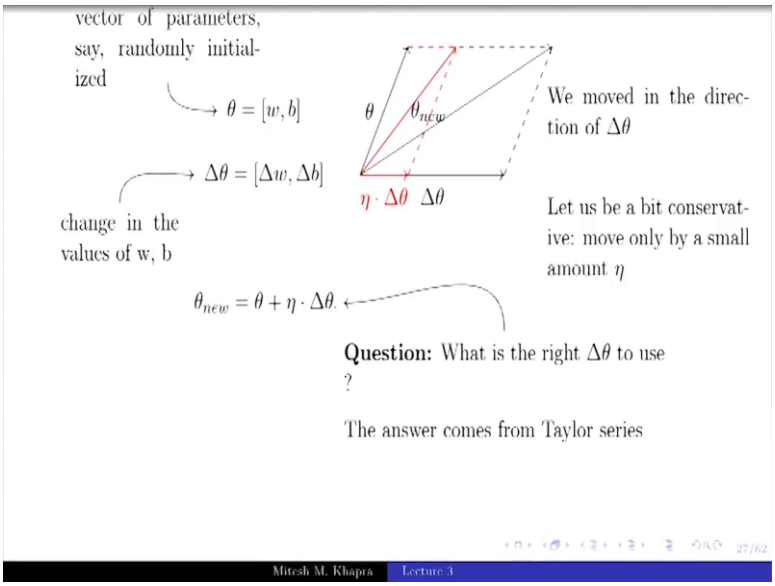
(Refer Slide Time: 01:08)



Now, what I want is again what I will do is, I do not know what the value of w comma b is. So, I started with a random guess. So, that is always going to be my starting point I will always start with a random guess and from there on move on to good values. Now, once I have started with a random guess I want you to tell me some changes that I can make to w and b. So, that I land up in better situations, right that means, I land up in situations where the error is less is that fine.

So, that change in w and b, I am going to call it as delta w and delta b and that again is a vector which is storing these two values. So, this is the picture right I want to take theta and I want to add a small change to it. So, this is my theta, this vector is actually theta right this is the theta vector, I want to add a small change to it which is again a vector, this is delta w comma delta b such that I will get a new value for theta new. So, theta new would be what actually theta new is equal to w new comma b new. Is that fine that is what theta new means.

(Refer Slide Time: 02:17)



Now, what has happened is actually when I have added delta theta to theta I have moved in the direction of delta theta, I have come from here to here. Now, I am going to be a bit conservative and I am going to say that while I am ok in moving in the direction of delta theta I do not want to make a giant stride what I will do is I will just move by a small quantity in that direction.

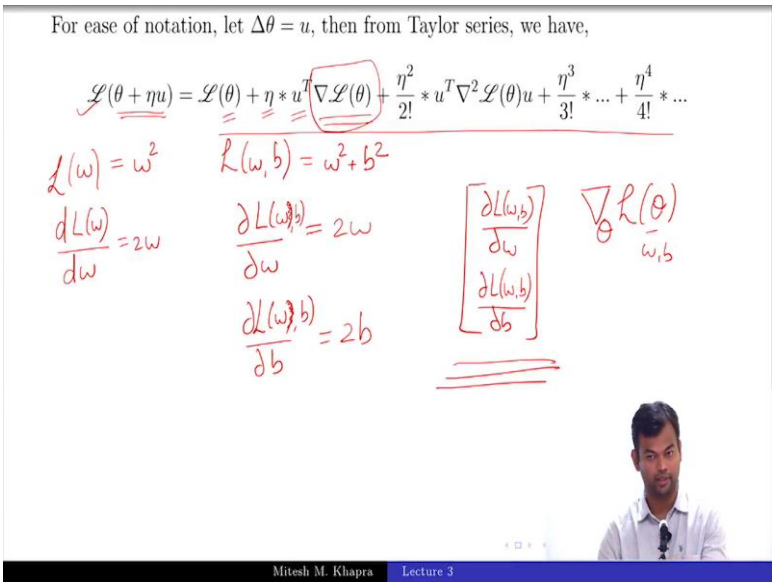
So, this delta theta is this large magnitude. So, all I am saying is that I will not I move in that direction I am fine with that, but I do not want to make a giant stride I will just take a small stride in that direction. So, eta is a scalar which actually scales down delta theta. So, now, if I am going to take only a small step in that direction instead of this large change I will just get a smaller change theta new. So, red the red vector is actually going to be the movement which I make, that is the new value of theta. So, theta new is equal to the original theta plus a small step in the direction of delta theta.

So, everything is clear? You are done we are done with gradient descent? What is missing? What is delta theta, right? I am telling you I want to move in a certain direction, but, what is the right delta theta to use? How many of you know the answer to this? What is the answer? Move in the direction.

Opposite to the gradient, why, where does that answer come from? Not the ML class folks. How many of you know why we need to move in the direction opposite to the gradient? Why, ok. We will see, ok. So, that is the question that we need to answer. If I

give you an answer to this question then what is it I am doing I am giving you a principled way such that you start from a random value of theta move in certain direction and you will ensure that your loss has decreased and then you have to keep doing this right. So, that is the set up and the answer to this comes from Taylor series.

(Refer Slide Time: 04:11)



So, now what I am going to do is I am going to give you the right direction delta theta; fine and for ease of notation I am going to call it as u. So, remember what this delta theta is, what is it? Change in w comma change in b. So, it is a vector in R2, remember that, ok I am just going to call it as u. Now, this is what Taylor series tells me what it tells me is that if I am at a certain value of theta and if I want to change that value a bit then what is going to be the new value of the loss function or any function for that point and this is the formula for that, ok. Now, what is let us see; what are some quantities here what is this quantity scalar, vector matrix?

Scalar, this?

Vector, we just did that right, it is a vector. What about this?

What is this quantity, actually?

Gradient: what is the gradient? What is the gradient? No, you are telling me how to use the gradient, I am asking you what is the gradient you are giving me absolutely correct and absolutely useless definitions.

That is a very good answer, ok. So, now what I am going to do is I am going to digress a bit, and I am going to tell you something about derivatives partial, derivatives and gradients and then we will come back to this, ok. So, now, suppose you have a function l this is L in my handwriting this function of w, and say this function is w square, ok. Now, what is what is this called? A derivative of the function with respect to w, this is the derivative and you know this is 2w, ok. Now, suppose I have a function b square, now what is this quantity?

Is a partial derivative of the function with respect to w? Why partial?

Because, it is considering b as a constant and taking the derivative with respect to only one of the variables, right this happens to be and what is this quantity, oh sorry. So, is w comma b, right this is the partial derivative with respect to b, ok.

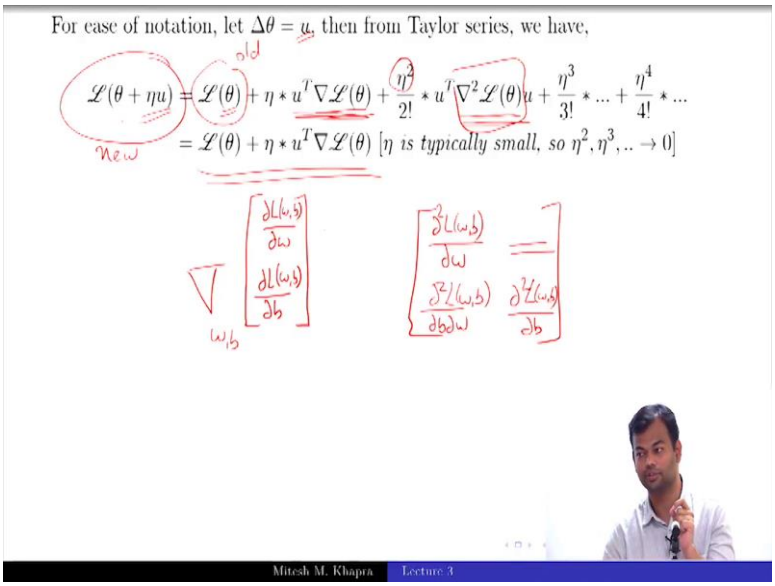
Now, can you tell me what is a gradient? The gradient is nothing but it is just these two partial derivatives taken together and put into a vector, right? Now, suppose I had a function which depended on hundred variables, what would the gradient be size of the gradient?

R100 it would lie, it would be a hundred dimensional case, ok. So, now can you tell me with this evidence in knowledge, but this primer can you tell me, what this is? This is a gradient vector?

Which is right there in front of you in a red ink.

This is what it is, right, fine. Everyone ok with that? So, actually the right way to write this and probably we need to correct in the slides would be theta. So, remember that theta is equal to w comma b. So, this is the derivative of L theta with respect to theta which is nothing, but the collection of the partial derivatives with respect to the components of theta, is it fine. So, everybody understands what is a derivative partial derivative and gradient, ok, fine. So, now, the gradient is a vector in this case, fine ok?

(Refer Slide Time: 08:25)



So, now what is this quantity? It is a?

No, it is what is this?

The dot product between these two vectors, ok fine. Now, one last thing and many more things actually. So, what is this, square of the gradient?

This is not the square of the gradient. What is this? Hessian, fine, everyone knows the textbook what said can you tell me, what does it is a scalar, vector matrix?

Matrix; what is the size of this matrix?

2 by 2; what are the elements of this matrix?

Second order partial derivatives, right. So, it is the gradient of the gradient, right, is that fine. So, what does that mean you had this gradient, this is the gradient. Now, you want to take the gradient of this again with respect to w comma b, right that is what this means, it is a gradient of the gradient right. So, what that means, is we will take the gradient of the first quantity again with respect to w. So, that would be dou square by dou w what would this quantity be?

What would this be?

Is that fine and you can fill in this quantity, right? So, now, it is clear, what the hessian is? It is the derivative of the derivative and it would be a matrix, ok, is that clear to everyone. So, I have a habit of doing a lot of these basic stuff I know that the top 20 percent of the class gets really pissed off when I do this, but as a philosophy I teach for the bottom 30 percent of the class.

So, I do not mind that and the other thing is I use slides. So, I do not write a lot of math so, I can cover a lot of material despite doing all this basic stuff, right. So, I am going to stick to that what I am trying to say is that write this in the feedback that you do not like this basic stuff, but it is just that I am going to ignore that feedback, I mean just being honest, right. So, I like doing this because it just takes me 10 minutes to do this and for the rest of the class I do not have to look at blank faces afterwards, right. So, it really helps me a lot fine. So, is that all clear all the quantities here are clear?

So, now, so, this is the gradient this is the Hessian and now eta, remember what did we say about eta?

It is a small quantity and what do we do with small quantities always in maths?

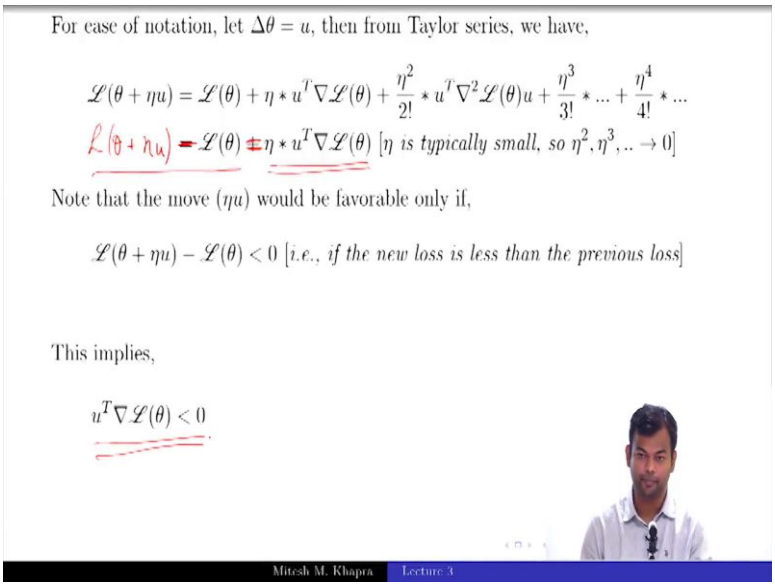
We ignore them. So, once we take their powers you are always ignore them. Whether it is correct or not who cares I mean someone has told it, it is good to ignore. So, we will ignore it, right. So, now, all these higher order terms we can ignore right; that means, I will only consider this, fine.

So, let us again look at what the setup is? The setup is that I have some value of theta I want to move away from that value such that what do you say about this loss compared to this loss? I will call this the new loss and I will call this the old loss. What is the relation between them?

The new loss should be?

Less than so if I or someone gives you a u. I am not getting ok, someone gives you this u then what does what when would you say it is a good u?

(Refer Slide Time: 12:28)

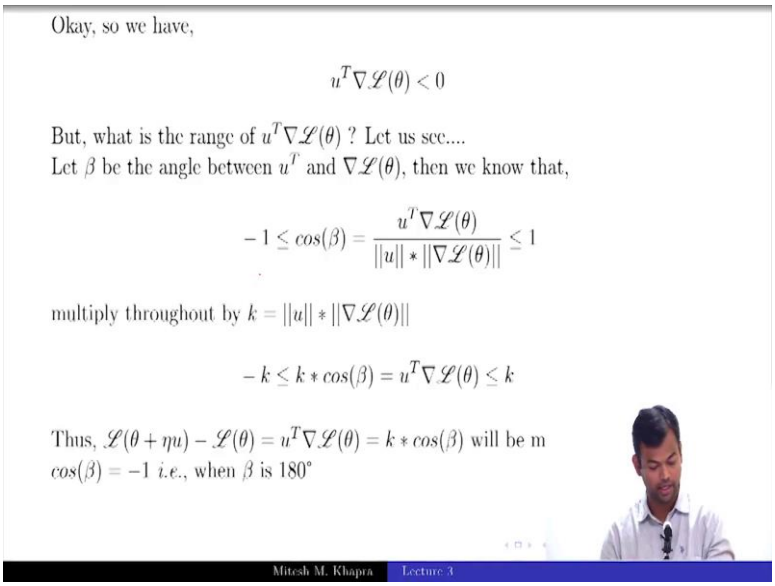


If this condition holds everyone agrees with that, right. So, I have found a good direction to move in if this condition holds. Now, this condition actually implies that this condition should hold right this is L theta plus eta u, right. So, if I just do minus here I get this, right. So, this quantity which should be less than equal to 0, implies that this quantity should be less than equal to 0 and remember eta is a positive constant, ok. Why cannot it be negative?

Why? Because you wanted to take a small step in that direction, if we make it negative we will do what?

We will reverse the direction; we do not want that as of now, right? So, eta is that for a positive quantity. So, that means, this quantity should be less than 0, is it fine with everyone?

(Refer Slide Time: 13:29)



So, so far after all this story what we are left with is this condition should hold for the u that I am trying to choose, so that I can be sure that I have chosen the correct u, right and the definition of correct u is that the loss at the previous step the loss of the new step should be less than the loss at the previous step, is that fine? So, that is what we have arrived at.

Now, what is the range of this quantity, that is why I asked you what is this? This is a?

Dot product, I will leave it at that. So, now, you tell me what is the range of this, people from the ML class cannot answer. Did I cover this in the ML class? No? Ok, fine. What is the range of this? Not a very hard question.

Plus or minus?

Student: (Refer Time: 14:25) mod of u (Refer Time: 14:26).

Very good; how many of you understood that answer? He said plus or minus mod of u into mod of gradient the gradient vector, right why is it so, easy.

Let beta be the angle between u T and this between sorry it should not be u transpose between u and the gradient then we know that this condition holds, cos beta is given by this quantity and we know that cos beta lies between minus 1 and 1, ok. Now, if I just say that this quantity is equal to k then I can just get this condition.

Now, let us see what are we trying to do? We are trying to find a u such that this quantity is negative. We are trying to find the use such that this quantity is negative, now I just stop at negative we would like to make it as negative as possible, right because the more than negative it is the more will be the decrease in my loss function right because this quantity tell me tells me how much my loss decreases. So, the more the negative it is the more the loss will decrease. So, let me make it as negative as possible.

Now, what is that value? When will that happen? When alpha is you know the answer you started with the answer.

Student: (Refer Time: 15:54).

No, what is that one phrase which you have marked up move in the direction?

Student: (Refer Time: 16:07).

Ok now, think of that.

Student: (Refer Time: 16:09).

What would happen when this is the most negative it can be, what would the angle be?

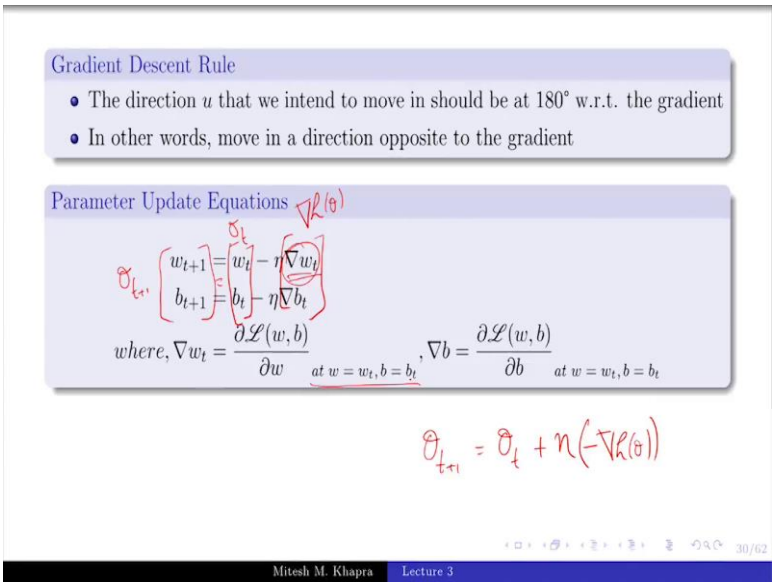
Student: (Refer Time: 16:19).

180 degrees; how many of you get that because when this is the most negative; that means, the cos beta is actually minus 1 and when is cos beta minus 1 when the angle is 180 degrees; that means, u should be such that it is at 180 degrees to the gradient, hence repeat the phrase.

Student: (Refer Time: 16:42).

Move in a direction opposite to the gradient. Is that fine? Everyone gets it now? Why you need to move in the direction opposite to the gradient.

(Refer Slide Time: 16:53)



So, this is what the gradient descent rule is. You are at a particular value of theta you want to move to a new value of theta such that your new loss is less than the current loss, what gradient descent tells you is move in a direction opposite to the gradient. So, are you fine with this? Now, with gradients I have come to scalars, but I will just explain what I have written here.

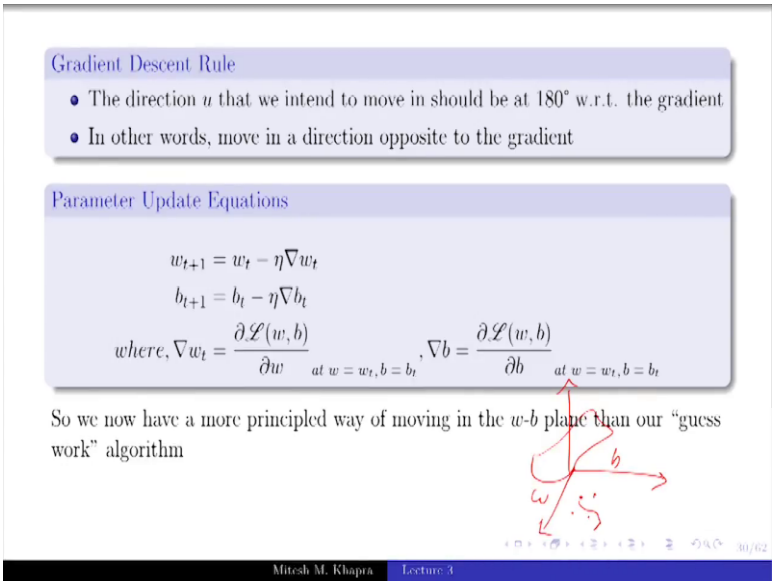
So, this quantity is nothing, but theta t plus 1, right, is equal to theta t, right and what is this, right. So, the new theta is equal to the current theta minus, why because we want to move in the direction opposite. So, it is basically theta t plus 1 is equal to theta t plus eta into a negative direction, right the direction negative to the gradient hence you get that minus 1.

Now, what are these quantities let me just take that carefully. So, this quantity is gradient of the loss function with respect to w, sorry the partial derivative of the loss function with respect to w evaluated at w is equal to wt and b equal to bt, what does that mean. So, remember when you are dealing with derivatives as always a formula and then a value add that at a particular value. So, what is the derivative of x square with respect to what does not matter 2x.

So, derivative of x square with respect to x is 2x. What is the value of this derivative at x equal to 1? 2, right. So, you see the difference you have a formula which is 2x, now you substitute in a particular value and you get the value at that particular value, ok. So, that

is what this means because you are already at w t comma b t, now you cannot subtract a formula from here right you have to put subtract a value. So, you know what the formula is you plug in the values of wt comma bt get that value and subtract it from your current wt, is that fine? So, everyone completely understands what is the gradient descent rule is, fine.

(Refer Slide Time: 19:14)



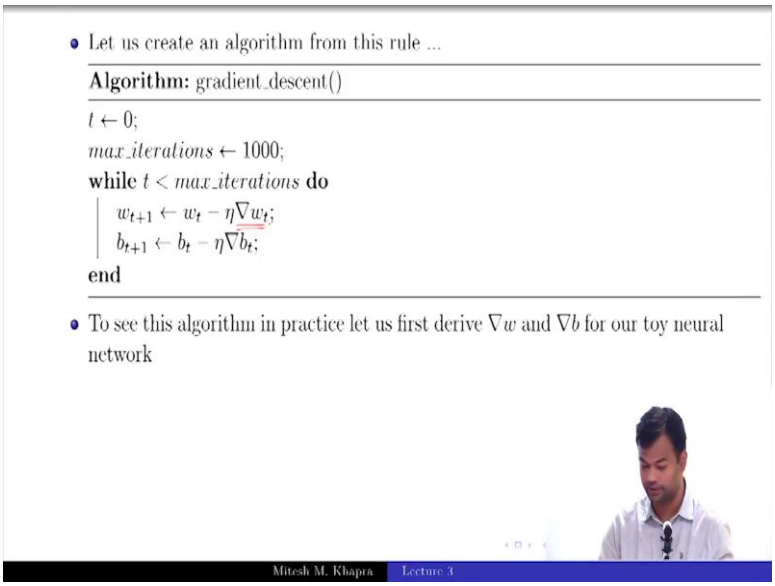
So, now we have a more principled way of moving in the w b plane. What do I mean by that? Remember this was our w b plane, this was our error, this is something what our error surface looked like, it was this flying carpet. I was randomly moving on the w b plane earlier, right and trying to guess what the errors or trying to compute the error and then settle for a particular value. Now, I have a more principled way of moving in the w b plane. I know what is the next step based on the current step I just need to move in the direction opposite to the gradient.

So, let us try to. So, this is what it tells me for one step, but I need to keep doing this till, what is that golden word?

Student: (Refer Time: 19:58).

Convergence, right? I have to keep doing this till convergence, ok.

(Refer Slide Time: 20:02)

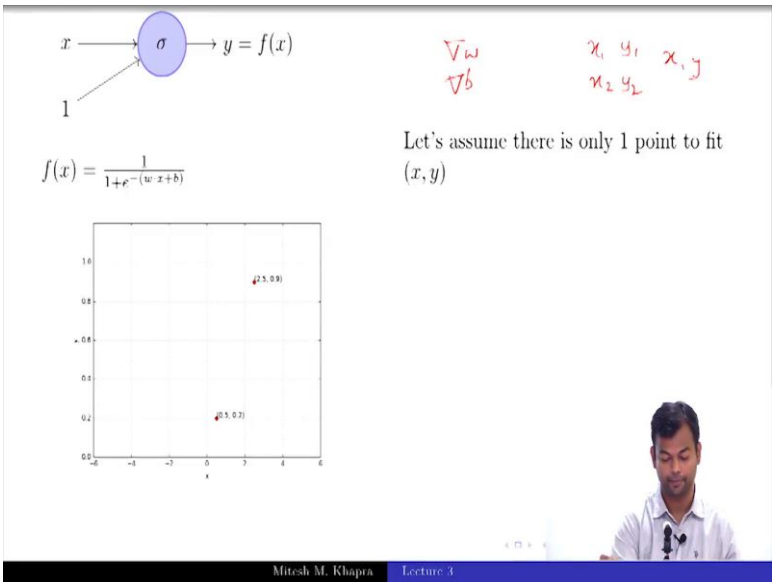


So, let us create an algorithm out of this rule I will start a time step 0, I will do this for some max iterations; instead of saying till convergence I will do it for some iterations. At every iteration I will this is how I will update my weights. I will take the current weights, subtract the gradient from that and get the new weights I mean not subtract the gradient subtract this quantity and get the new weights. So, now, is everything clear? Is the gradient descent algorithm done? Can you do it for the toy network which I had? Is there something still missing?

Student: (Refer Time: 20:38).

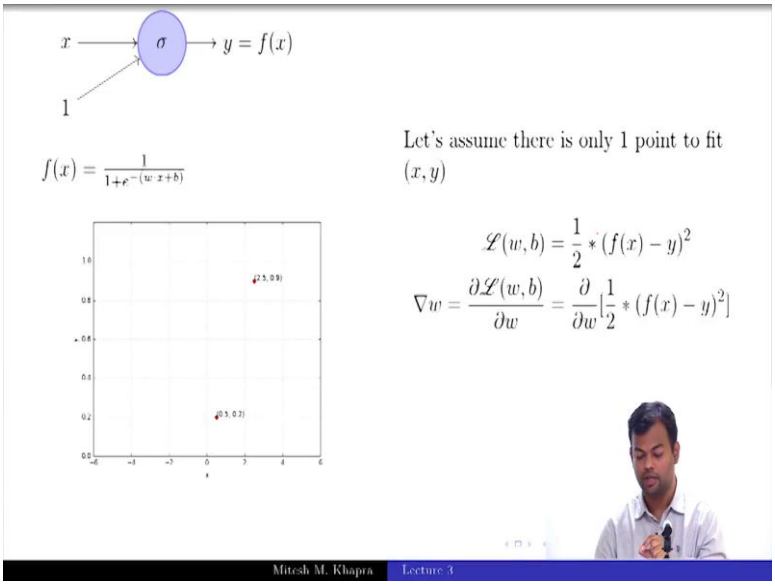
Eta is fine we will take a small value 0.01 or something. Actually, not told you what these are, right I means to write it you know these are derivatives, but what is this actually, ok. So, let us see that now. So, that is what we are going to see next.

(Refer Slide Time: 21:00)



So, now we want to find out we are in the car quest is for this delta sorry the partial derivative with respect to w and partial derivative with respect to b, that is the thing which we had plugged in the formula, but we do not know what that is, right. So, we need to find that out. So, now, for simplicity let us assume there is only one point of it which is x comma y. So, earlier we had this x1 y1 and x2 y2. Now, I am just assuming there is only one point which is x comma y.

(Refer Slide Time: 21:50)



So, now, what is a loss function earlier I had this summation over I equal to 1 to 2, but I have just one x comma y. So, I will just use that this is what my loss function and what are the quantities that I am interested in finding? One is this the partial derivative of this loss function with respect to w.

(Refer Slide Time: 22:11)

$$\nabla w = \frac{\partial}{\partial w}[\frac{1}{2} * (f(x) - y)^2]$$
$$= \frac{1}{2} * [2 * (f(x) - y) * \frac{\partial}{\partial w}(f(x) - y)]$$
$$= (f(x) - y) * \frac{\partial}{\partial w}(f(x))$$
$$= (f(x) - y) * \frac{\partial}{\partial w}\left(\frac{1}{1 + e^{-(wx+b)}}\right)$$
$$= (f(x) - y) * f(x) * (1 - f(x)) * x$$

$$\frac{\partial}{\partial w}\left(\frac{1}{1 + e^{-(wx+b)}}\right)$$
$$= \frac{-1}{(1 + e^{-(wx+b)})^2}\frac{\partial}{\partial w}(e^{-(wx+b)}))$$
$$= \frac{-1}{(1 + e^{-(wx+b)})^2} * (e^{-(wx+b)})\frac{\partial}{\partial w}(-(wx + b)))$$
$$= \frac{-1}{(1 + e^{-(wx+b)})} * \frac{e^{-(wx+b)}}{(1 + e^{-(wx+b)})} * (-x)$$
$$= \frac{1}{(1 + e^{-(wx+b)})} * \frac{e^{-(wx+b)}}{(1 + e^{-(wx+b)})} * (x)$$
$$= f(x) * (1 - f(x)) * x$$

Mitesh M. Khapra Lecture 3

So, let us do this lets actually derive this. So, this is what it looks like now you have to help me in deriving this what will I do first?

Student: (Refer Time: 22:18).

Tell me the next step.

Student: (Refer Time: 22:23).

2 into f of x minus y and push the gradient inside, of course the derivative is that fine? Anyone who has a problem with this? Next, y is a constant this is the true I remember. So, that is why this is a constant is not the predicted y.

Now, this quantity, what is f of x actually?

Student: (Refer Time: 22:51).

Sigmoid function, right; so I will just write it. Now, this is the quantity that I need the derivative for. So, I will just write it here. What is the next step? This is of the form 1 over x. So, what will it be?

Student: (Refer Time: 23:04).

Minus 1 over x square and then you put the derivative and say it. Is that fine? Now, the quantity inside is of the form e raise to x. So, the derivative is e raise to x and you push it inside, is this fine? So, this on slide should come both these are coming together. So, is that fine? Now, what is this actually?

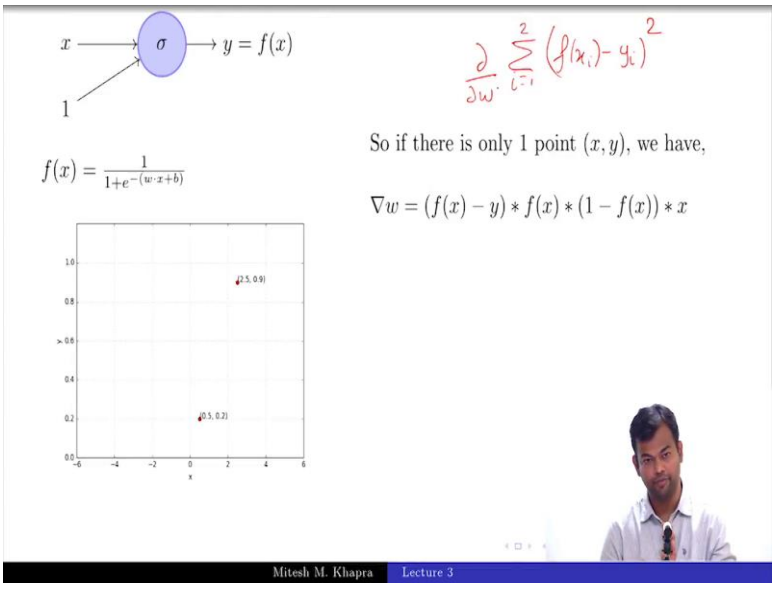
Student: f of x.

f of x; what is this?

Student: (Refer Time: 23:32).

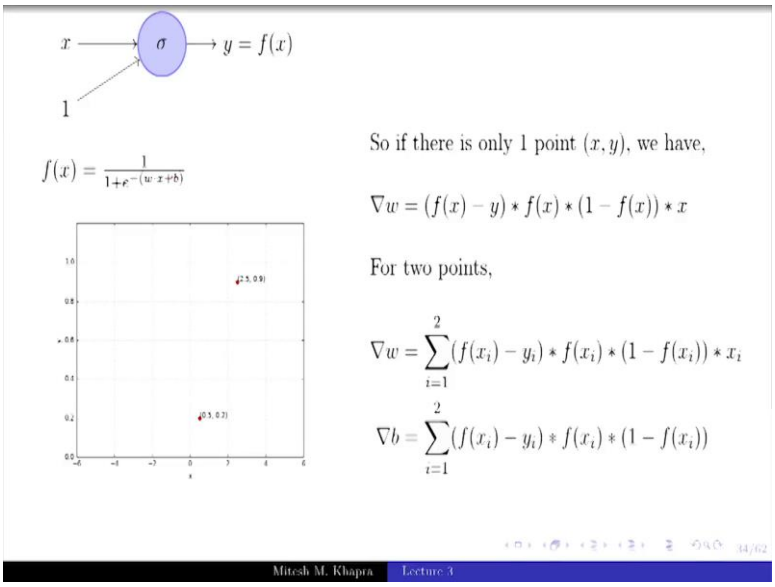
This is actually 1 minus f of x, just take my word for it for now, you can go home and work it out, right. So, this actually if you do 1 minus this and do some trickery you will get to this quantity, right. So, what you get is a very simple formula f of x into 1 minus f of x into x, I am going to substitute back here. So, now I exactly know what the partial derivative of w is.

(Refer Slide Time: 23:57)



So, there is only one point, then this is what the partial derivative with respect to w is going to be of the loss function, right. If there were two points what would happen? If there were two points my loss function was this is a sum of two elements. And I am taking some derivative of a sum, I will get a sum of derivatives, right.

(Refer Slide Time: 24:23)



So, how many of you will not cringe if I say this is the answer? Anyone who has a problem with this? You get this, how many if you do not get this? How many of you get this? Good, fine. Now, can you do a similar thing for b? Can you tell me the answer without actually deriving it?

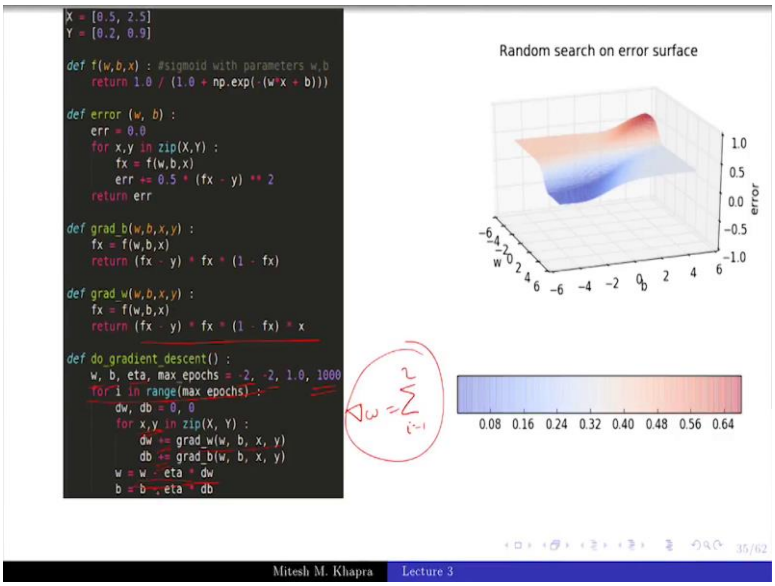
Student: (Refer Time: 24:41).

I can perfectly understand what you are saying.

Student: (Refer Time: 24:48).

x would not be there, right because this last x that you see here came because w into x was there, but b we are not multiplying x. So, what we will get is this. You can go home and check.

(Refer Slide Time: 25:12)



So, now we have everything that we need. Now, we actually have everything that we need, ok, no more trick questions. So, now, we will write code to do this, ok, we will actually implement the code and see what happens. So, these are the two data points that I had 0.5 comma 0.2 and 2.5 comma 0.9. The first thing which I need is something which can implement the sigmoid function. So, this is 1 over one plus e raise to minus w x plus b is that fine.

Now, I need something which can compute the error. So, this is summation of half into f of x minus y the whole square, I go over all the data points summation of half into f of x minus y the whole square, is that fine? Now, what I will do is I will take this try out a lot of values of w comma b and plot the error surface ok, but this is only for illustration, in practice I will not do this. We just know that this error surface exist I just want to verify that whatever algorithm I come up with does not efficient navigation of this error surface, that is what I want to verify, that is why I am plotting this.

Next time you need a function which can compute grad of b. We just saw this on the previous slide this is f of x minus y into f of x into 1 minus f of x, right, simple. Everyone is fine with this? Then I need a function which can compute the grad with respect to w same thing except that I have this x at the end. So, I have all the ingredients in place. Now, what would I do, what is the next thing that I will write? The main loop right, I will write the main loop now.

So, this is what the main loop look like looks like. I start with some random initialize for w comma b. Remember that our initial theta which is composed of w comma b is going to be some random guess. So, I started with the random guess which is minus 2 comma 2, I have chosen eta to be 1; that means, I am not going to be conservative I am going to move in the direction of the gradient. If I chosen at 0.01 and 0.001 I would have been conservative and I am going to run this till 1000 epochs which is my notion of conversions.

Now, in each epoch what I am doing is for every data point, so, remember that this gradient with respect to w was a summation of i equal to 1 to 2 and that formula, right. So, for each data point I am computing the grad adding it, right. So, that is the summation part similar thing I am doing for b. Once I have computed the gradient which is the summation quantity I am just moving in the direction of the gradient, is that fine? Everyone understands the code, it is simple python code and it does exactly what I had shown in the pseudo code.

Now, let us execute this code and see what happens. So, I will start with my random point which was minus 2 comma 2 and now, I am going to actually run this code and keep plotting what happens on the figure. So, just pay attention fine. So, now, here is how the code is running. See what is happening, what is happening actually? So, at every point I am changing my w, so that I am moving in the direction of the gradient I keep doing that as I keep doing that my error keeps decreasing, why? Because that is exactly what we got from Taylor series that if we do this the error is bound to decrease, right and then we keep doing this and after a few iterations we will actually reach almost the value which is the zero error, right and this same thing would happen if you start from anywhere else it will keep moving in a principled way and reach the low error configuration.

Now, some of you would say that maybe this was the shortest path, right. It could have just rolled over from there, but that is not a principled way of doing that right we the principled way of doing it is to move in the direction of the gradient. You might take a longer route, but reach your destination, taking shortcuts is always risky, in life as well as here. So, so, do not please this is an advice for error assignments and so on. So, this is the more principled way and we will reach the solution. So, that is what is happening

here. So, we have actually derived everything that we needed and this is all you need to write for gradient descent for this toy example that you had.

Now, answer this question. Now, suppose I had hundred such variables, instead of w comma b, I had hundred such variables. What would happen? You do not have to visualize it.

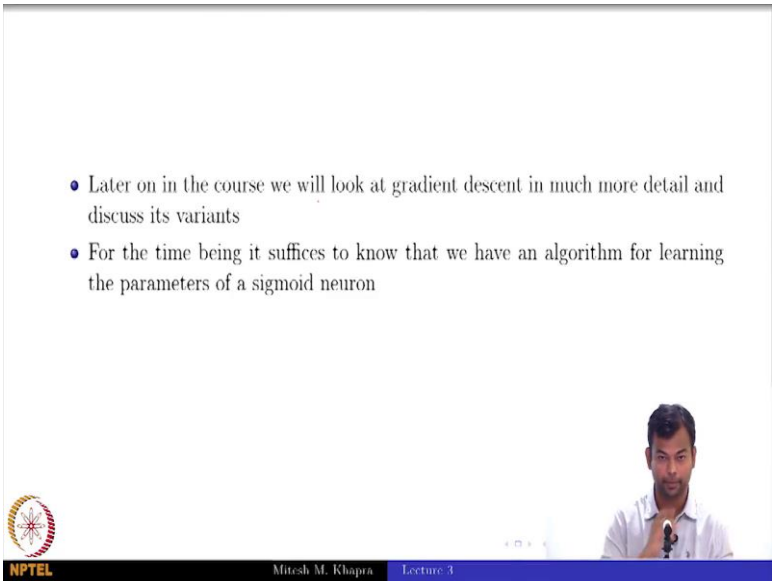
Student: (Refer Time: 29:48).

In terms of the code.

Student: (Refer Time: 29:52).

I will just need to have these functions for all of those. I will have to calculate it by hand, but still doable it is just a lot of tedious work of course, later on we will see a more refined way of doing this where we can do a lot of these computations at one go. So, we can directly start operating in vectors as opposed to scalars here. I am treating w and b separately here I could have actually had a function which tells me grad of theta directly, right. And later on we will see something like this ok, but for now the code is still running here.

(Refer Slide Time: 30:24)



Now, it suffices. So, later on we will see gradient descent in more detail in the course and we will also see a lot of variants of gradient descent, but for now it suffices that we

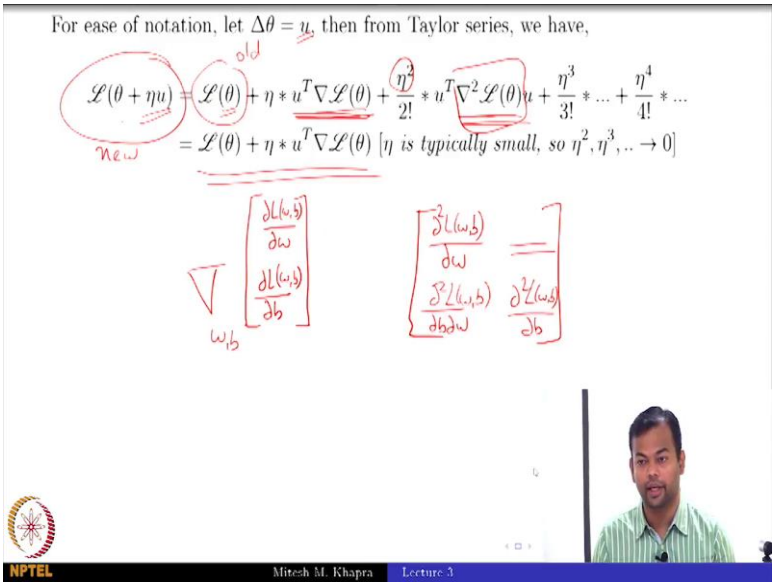
have an algorithm which can learn the parameters of a sigmoid neuron. So, just as we had the perceptron learning algorithm, we have the gradient descent learning algorithm which can help us learn the parameters of the sigmoid neurons starting from random values. And it gives a principled approach for doing that.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module - 3.5
Lecture - 03
Representation Power of a Multilayer Network of Sigmoid Neurons

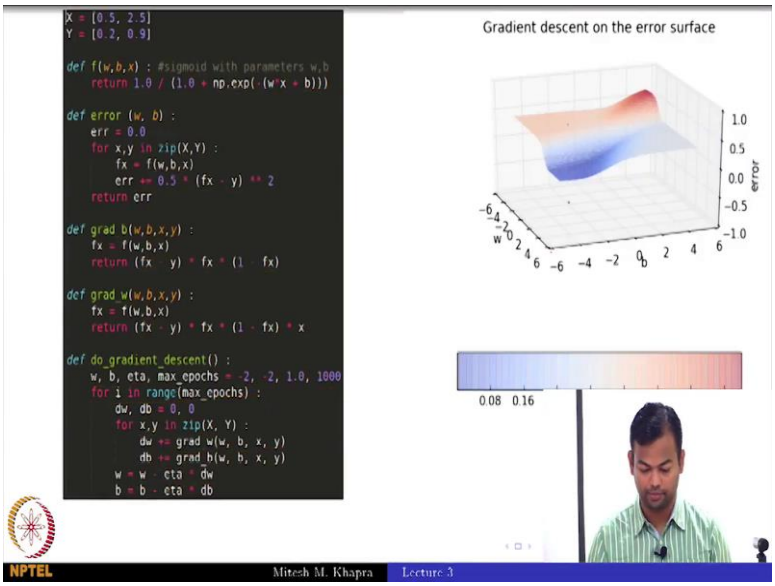
Before we move on to the next modulate some small corrections from yesterday's class.

(Refer Slide Time: 00:13)



So, one was this partial derivative it should have been dou w square. So, we already taken one derivative with respect to w and now you are taking another derivative it is the gradient of the gradient. And similarly should this should have been dou b square, and this should have been dou w dou b.

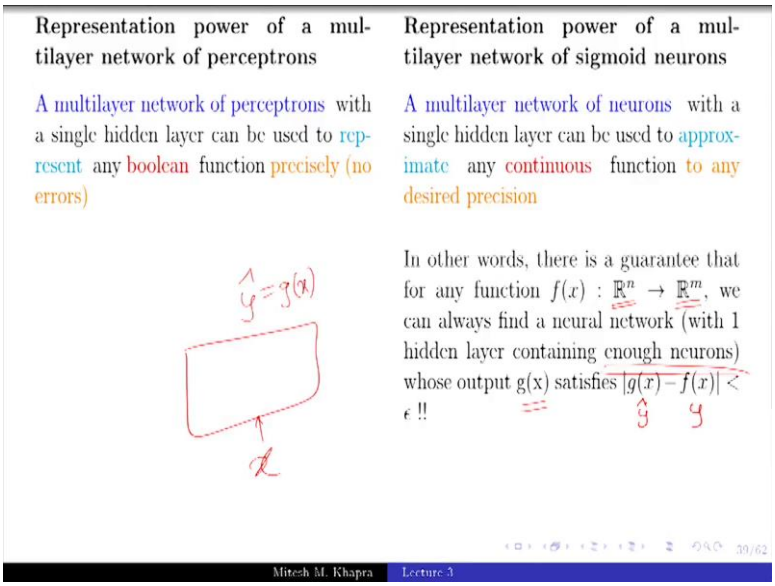
(Refer Slide Time: 00:43)



The other small thing which I wanted to say was so, when I was executing this algorithm, right. So, I forgot to mention that just notice what is happening is the black dot that you see the black dots that you see, right and which are very close to each other. Actually, because you are just making small movements those are the changes in the w comma b values and the red dots are the corresponding loss to that w comma b values right, just to clarify.

So, that is why you see a movement on the w b plane which is this movement and as you keep changing that your loss function changes and it becomes better and better right; that means, it goes closer to 0.

(Refer Slide Time: 01:25)



So, in this module we are going to talk about the representation power of a multilayer network of sigmoid neurons, right. So, I am going to compare these two things which are written in the title. So, first tell me; what was the representation power of a multilayer network of perceptrons, ok? I roughly hear what you are saying and basically what you are telling me is that a multilayer of network of perceptrons with a single hidden layer can be used to represent any Boolean function precisely, right? No, errors that's what we saw with that illustrative proof where we actually constructed once its network.

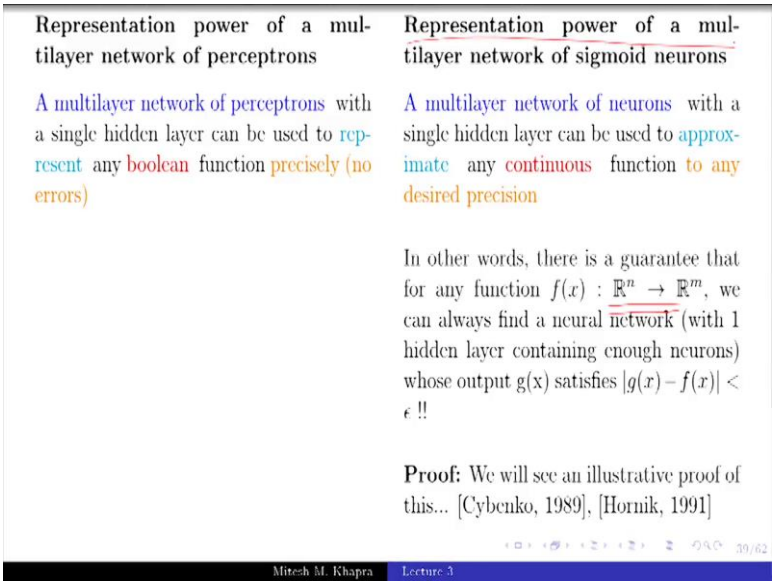
Now, what is the representation power of a multilayer network of sigmoid neurons? So, multilayer network of neurons with a single hidden layer can be used to approximate, ok. So, just see the difference in the language. So, this was a represent; that means, exactly this is approximate; that means I will tolerate some error, any continuous function instead of Boolean function to any desired precision. So, this was not this was precisely with no errors this is up to any arbitrary desired precision.

So, what does this mean, actually what is the meaning of this? So, there is a guarantee that for any function, ok which takes our original x from R n to R m what is the m that we have been considering in all our examples one right we just care about one output, but it can be R m also. We can always find a neural network with one hidden layer containing enough neurons. So, that is the operating trace here enough neurons whose output g of x, so, that means, you would have a network it would take as input n x it

would produce some y hat and that is what I am calling as g of x, right, that g of x would be very close to the true function f of x.

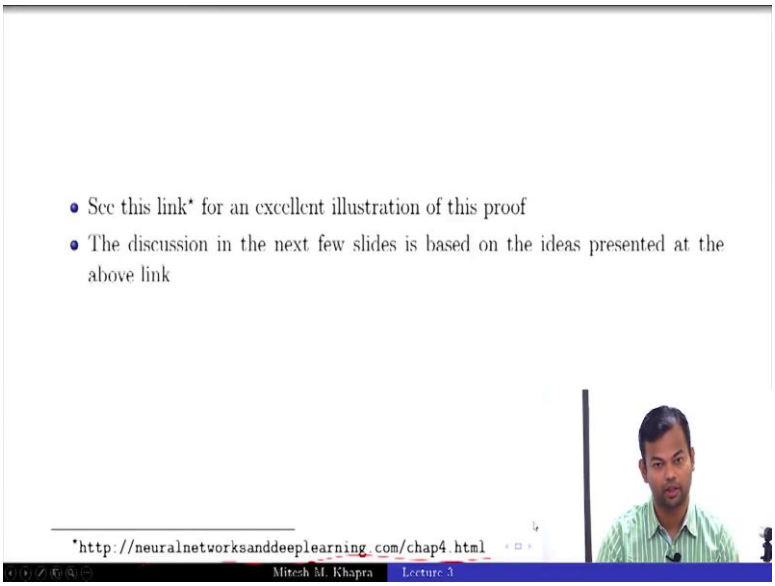
So, remember that we said that there is this true function f of x which gives us the true y's and we are trying to predict this y hat. So, the true why I am calling by f of x and the y hat I am calling by g of x and you can come up with a neural network which can give you values which can predict values which are very close to your true values. Does that make sense? Do you see the value of this theorem, what is it trying to tell me? Tell me, can you can you give me an interpretation of this? Why is this so useful? Do you know what this theorem is called universal approximations here and we did that in the history, right?

(Refer Slide Time: 03:57)



So, this was 1989, ok. What is the significance of this? Why do we care about such arbitrary functions and what does this theorem telling us actually? It is of course, telling us something about the representation power of a multilayer network of sigmoid neurons, but why is this important. So, we will see that.

(Refer Slide Time: 04:26)



So, this the remainder of the lecture I have borrowed ideas from this URL you should actually read this it is a very interesting book it is available online for free very illustrative. So, please take a look at it, ok.

(Refer Slide Time: 04:41)

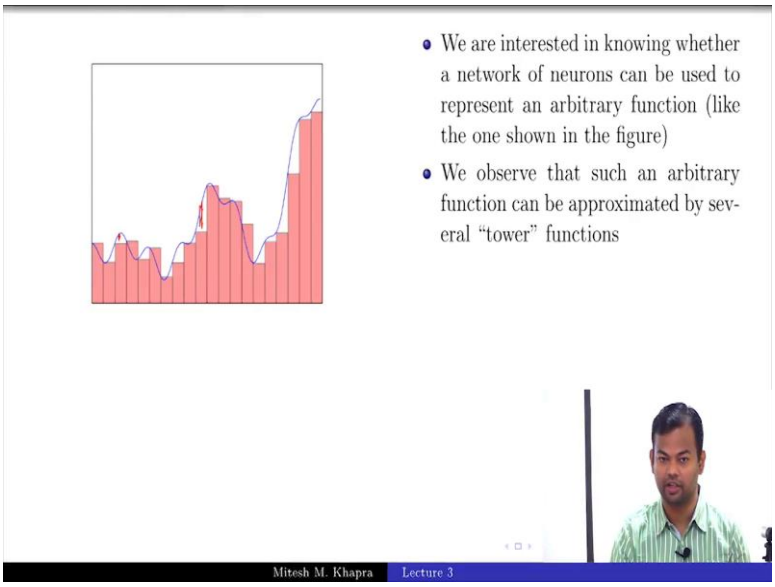


So, now actually what we are interested in is we are interested in knowing whether a network of neurons can be used to represent any arbitrary function like the one shown in the figure, ok. So, let me put some labels on this, so, they understand what I am trying to say. Suppose, this is salinity, again I go back to my oil mining example and I say that my

decisions are based only on a single variable which is salinity and this is actually how the amount of oil varies right as the salinity increase. It is a very arbitrary function, it is definitely not a linear function, it is not even a quadratic function, it is not an exponential function, it is just some arbitrary function, but a mathematical function this is possible. It is quite likely that salinities has this influence or in oil production or maybe it does not, but I am just taking that as an example.

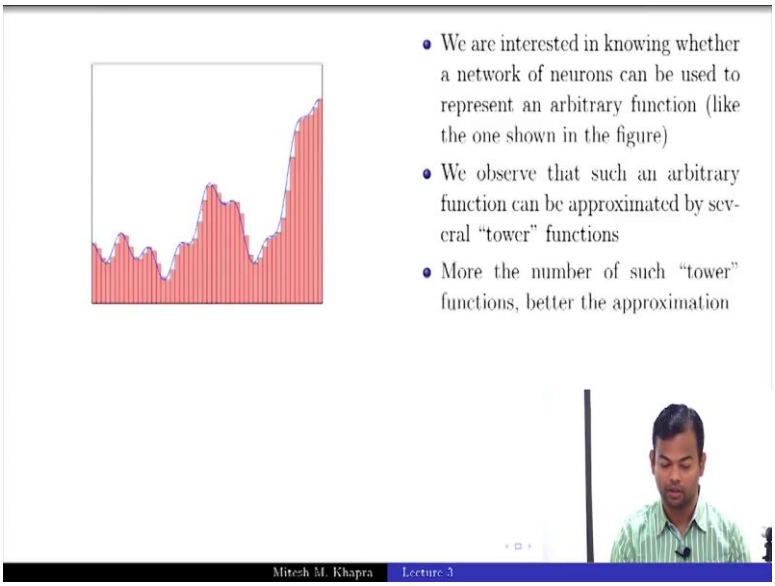
Now, what do we want the network to learn? If I take some data and train the network at the end of training, what do I want? So, if I feed at this point after training, what should happen? It should give me this value, right that is what training means and that means, I should be able to approximate this curve, if I do that that means, I have learned from the training data. So, let us see.

(Refer Slide Time: 06:00)



Now, we make an observation that such an arbitrary function can actually be approximated by a lot of something that we call as tower functions. These are all single I mean pulse functions which you have many of these, and you could have an approximation right and you can see that this approximation is bad at many places, but still it is an approximation it largely gives you the same shape as the original curve. What would happen if I increase the number of such tower functions?

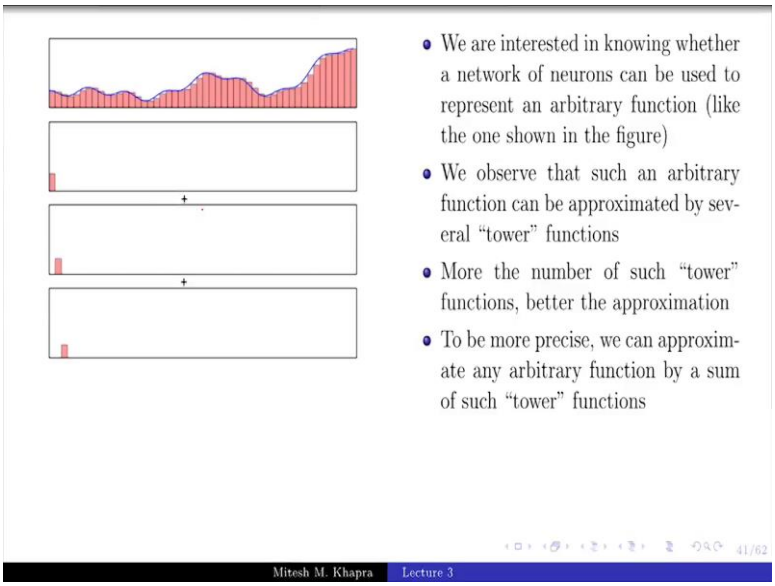
(Refer Slide Time: 06:31)



Student: (Refer Time: 06:29).

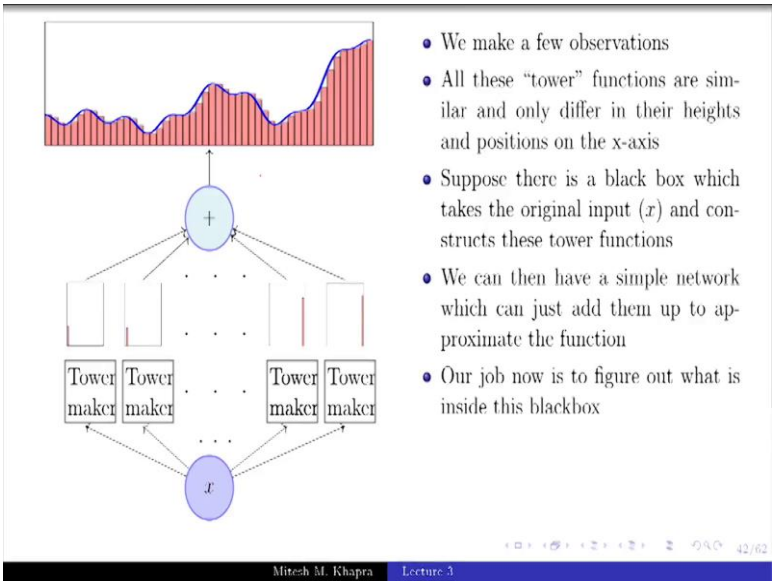
The approximation would improve, right. If I keep increasing it the approximation would go more and more better, right. So, now, just try to keep things in mind whether I write in the theorem, right you can make it arbitrarily close to the actual value; that means, you can keep doing something. So, that your approximation becomes better and better and you already see something of that sort. This is still in the sense of a figure, we need to relate this back to a neural network, but you see that as I am increasing these tower functions I become approx arbitrarily close to the actual function.

(Refer Slide Time: 07:05)



Now, this is what is actually happening right I have multiple such tower functions I am adding them up all of them are shifted in time. So, this tower function is actually this one this tower function is actually this one and so on, right and I have not drawn the remaining ones I am taking all of these tower functions adding them up and getting my original function, right and the more such tower functions have the better is the approximation.

(Refer Slide Time: 07:35)

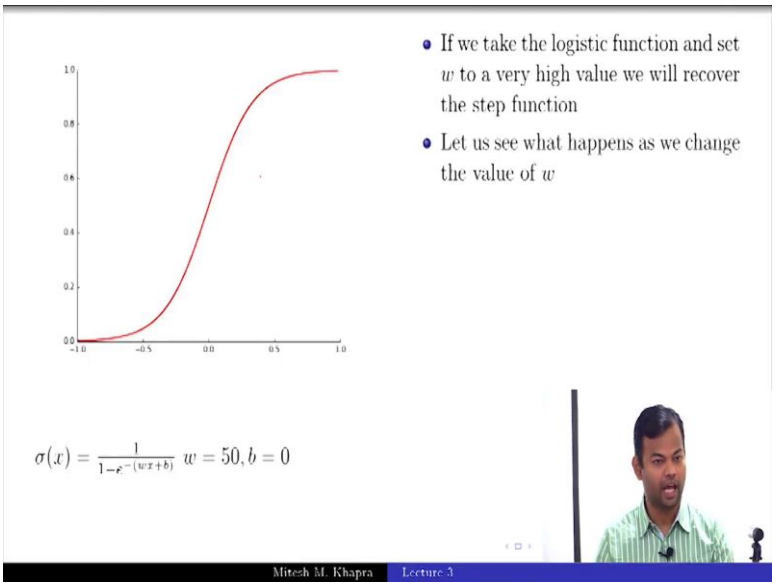


Now, you make a few observations right all these tower functions are actually the same what is the only difference they just shifted and their magnitude changes right, but they are all tower function right. So, let us think of this that if I know how to make a rectangle then I can make any rectangle, right? I just need to change the size of the rectangle and maybe shift it or oriented differently or something, right? So, they are all similar I just need to learn how to draw a tower right? that is what my quest is.

Now, if I take the original input salinity pass it through multiple such blocks each block is capable of making a tower function and each of these would give me one of these towers that I am looking for and I am looking for so, many of these, right. If I have as many such tower makers then I could get these towers I could just add them up, and then get the original function back. And the more these I have the better is my approximation, right? So, I am taking as input the salinity and trying to predict the oil. Does this make sense. Still we have not figured out a neural network way of doing this we are still building intuitions of how to do this.

Now, our job now is to figure out what goes in this black box that is the tower maker and how does it connect to neural networks, if you figure that out then our story is complete. Then we know that a neural network can actually do this and that precisely proves the statement which I had made that it can it can represent arbitrary functions. So, we will figure this out over the next few slides.

(Refer Slide Time: 09:12)

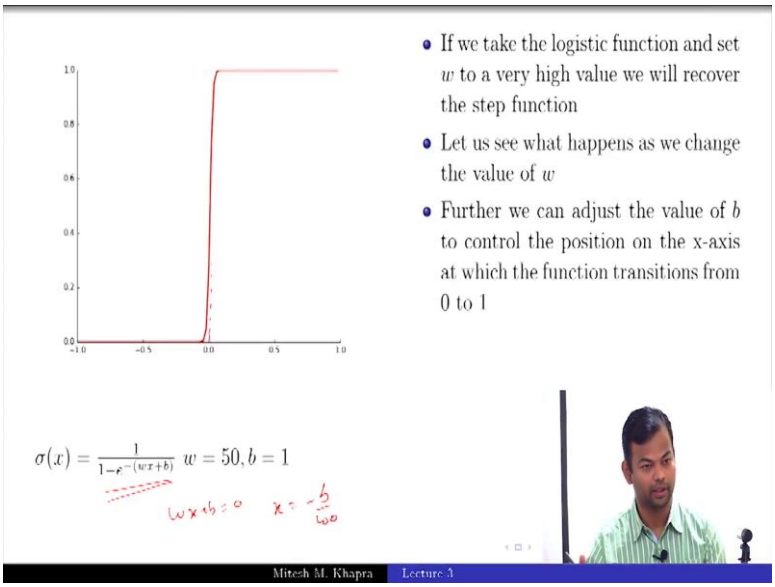


Now, if you take the logistic function and set w to w to a very high value, what will we get? Just try to think about it. The answer is already written, but I want you to imagine it. W covers what?

Student: (Refer Time: 09:31).

The slope, right; as I make w very high what will happen is I will get the. So, let us try changing the value of w, ok. I just increase the value of w and see what happens to the sigmoid curve.

(Refer Slide Time: 09:47)



Some error here, actually there is some problem the w value should have increased and that is how the sigmoid slope increases not the b value the b value comes later on. So, actually sorry about this, the w value as I keep increasing. So, do not think that b is increasing think that the w is increasing. It will become sharper and sharper and it will come very close to the step function, right. It will not become exactly the step function that will only become in the limit, but if I keep increasing I will get very close to the step function everyone agrees with this.

Now, what happens if I increase the value of b, it will shift everyone is confident about that, can you tell me why?

Student: (Refer Time: 10:34).

What will shift actually; the point at which the transition happens, right. So, what is this point actually?

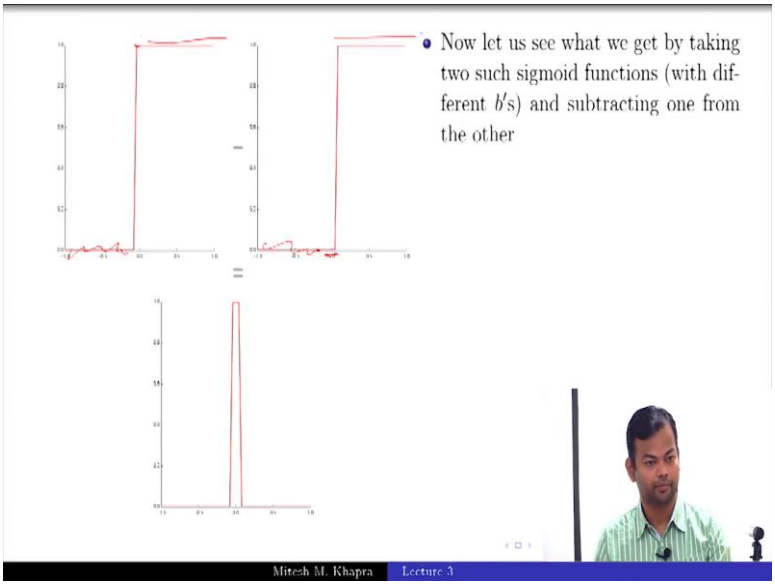
Student: (Refer Time: 10:43).

This is the point at which I get that half value, right and let us look at our function, this is a function. When will I get that half value? When w x plus b is?

Student: 0.

0, right. So, that means, x equal to minus b by w, that is why it is proportional to b. So, as I keep increasing the value of b, this will keep shifting, ok. Is that fine everyone ok with this?

(Refer Slide Time: 11:16)



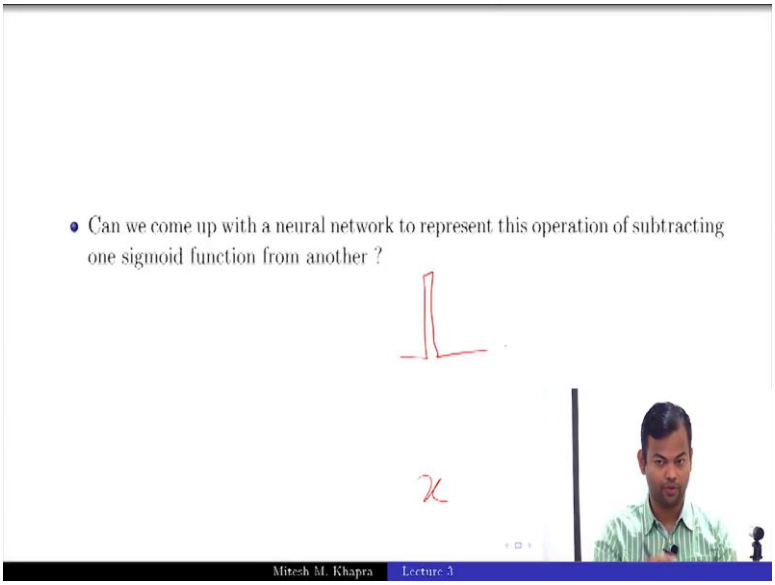
Now, what if I take two such modified sigmoid functions which are shifted differently and both are very close to the step function, right. So, here is where one threshold is, here is where the other threshold is and now I subtract this one function from the other, what will I get?

You know the term.

You will get a tower, right. Is that fine, everyone gets this, right? So, these places up to this point both are 0. So, 0 minus 0 will be 0, at this for this small range this is 1 and this

is 0. So, that 1 minus 0 and then afterwards both are 1. So, 1 minus 1 would be 0. So, you get that tower function. So, now I have my tower maker.

(Refer Slide Time: 12:05)

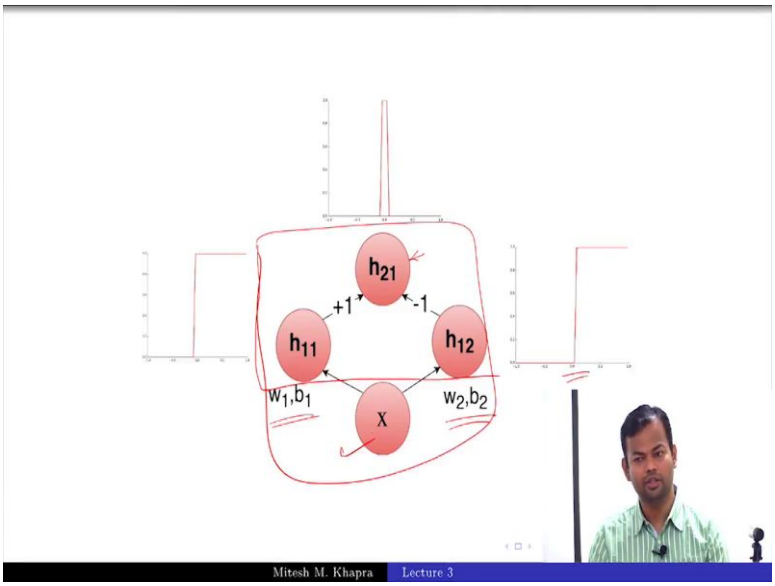


Now, can we come up with a neural network to represent this operation? I want a sigmoid neuron, I was working with a sigmoid neuron with some arbitrary weights, right, so that I recover that step function, can you imagine? Now, given x, I want this tower function and that is exactly what one of the blocks was, right. So, what I am asking you is oh god. So, I am asking you to give me a neural network for this. Can you think of it? Can you try imagining it?

Two neurons in the hidden layer; how many of you agree with that? Ok, can you can you take some more time to imagine what it would be?

And I have already, ok right.

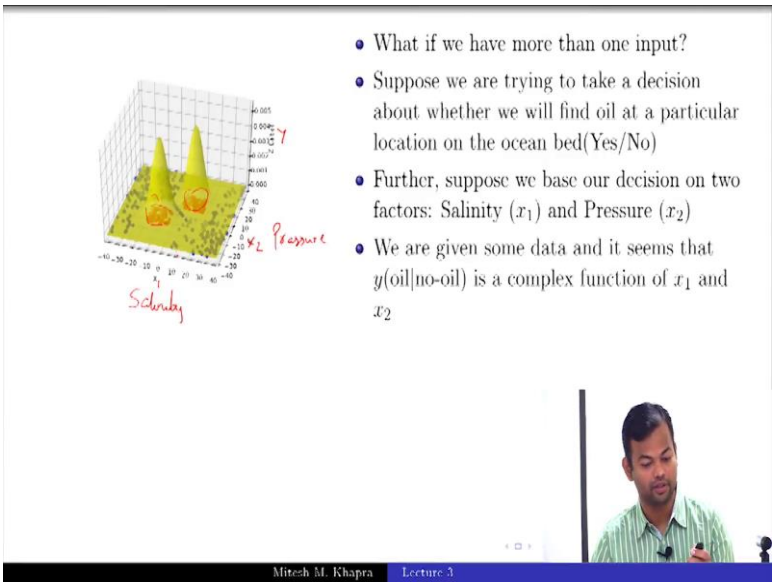
(Refer Slide Time: 13:07)



So, this w 1, b 1 if I set it appropriately I will get this step function. If this w 2 b 2, I set it appropriately I will get this step function. Now, I needed to subtract one from the other, right. So, I will do plus 1 minus 1, this is just a simple addition and I will get this, is that fine? Everyone agrees with this? This is just a adder, right, this is just an aggregator everyone gets this? So, now, I have given you the tower maker? If you put enough of these tower makers and learn the w's appropriately what will you get?

That function that we were needed. So, you can approximate it arbitrarily to any precision that you want as long as you keep increasing the number of these units, right. So, these units actually give you one tower more of these units that if you have actually this much this is the input ring. The more such tower makers that you have the more is the bars that you will get and then you can approximate everyone gets the intuition behind this? Fine, ok. This all is always good in one dimension.

(Refer Slide Time: 14:21)



Now, what will happen in two dimensions? What if we have more than one input? What is the tower there? Do you do you guys all do all know what is the tower there?

If you say no, I will give you a zero on the assignment. Remember the last question of the assignment? Did you all make a tower? Did you all make a 2D tower? Did you all copy that? No. So, what if we have more than one inputs suppose you had again trying to take a decision about whether we will find oil at a particular location of the ocean, right and suppose, now we base it on 2 two, right. So, say this is salinity, this should be x 1, should be x 2 should be y, and this is pressure.

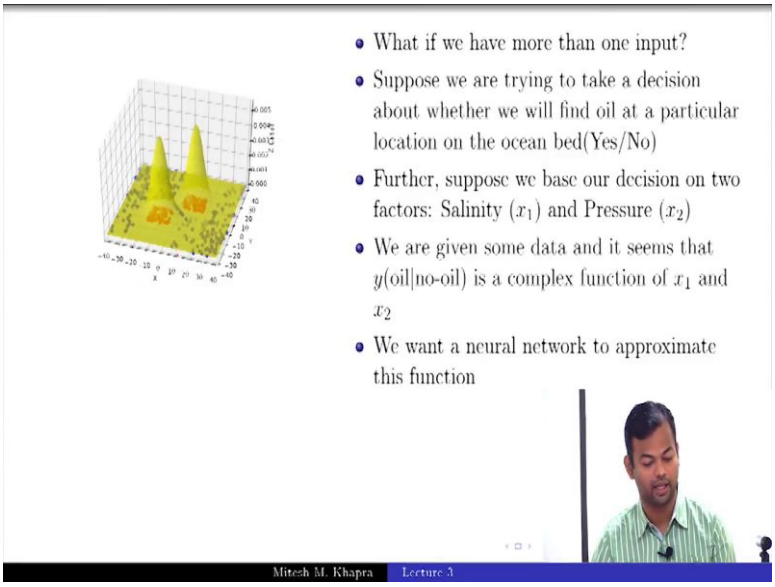
Now, just observe about the red and blue points. So, the red points are where you will not, for those configurations of salinity comma pressure you will not find oil and the blue points are for which you will find oil. What is the one thing that you can tell me about the red points and the blue points? Not linearly separable, right, but we still want to be able to learn this, is that fine, a single perceptron cannot do it I will also make a case for a single sigmoid neuron cannot do it and then I will show you that. In fact, first I will show you that with a network of neurons we can do it and then I will show that: with a signal single sigmoid neuron you cannot not actually do that.

So, now this is again a valid problem you could have we could imagine that you will get this kind of data where you have two factors and your function is some arbitrary function of these two factors. It is not a need linear boundary between the blue and red points.

Everyone sees that the blue and red points are not linearly separable you cannot draw a plane such that all your red points lie on one side and the blue points lie on the other side, everyone sees that ok, but the solution which I have plotted here that is a good solution. It makes sure that all the red points are in this region and the blue points are outside.

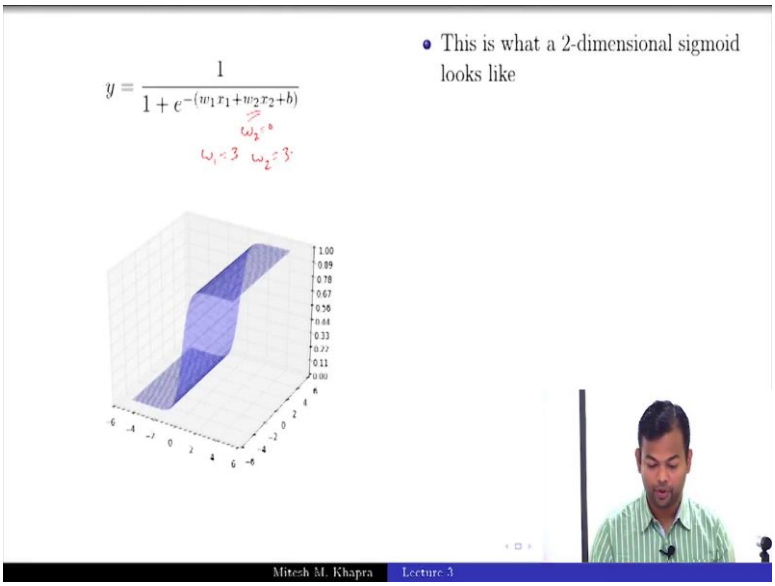
So, it will predict a high value for these red points and a zero value everywhere for the blue points, is that obvious? How many of you understand that figure? Ok, good.

(Refer Slide Time: 16:46)



So, now I want to show that even in 2-dimensions I could come up with arbitrary. I could come up with a neural network which could actually approximate this and again what will I look for? A tower maker, right? I just want something which can make towers and approximate it.

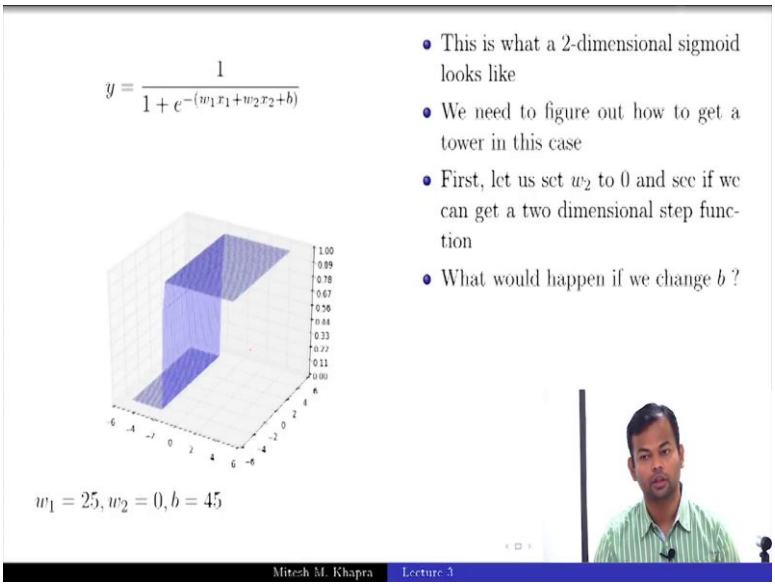
(Refer Slide Time: 16:48)



So, this is what a 2-dimensional sigmoid looks like slightly incorrect because I have what I have done is I have actually said w 2 to 0. So, if you actually I would want you to do this go back and plot this for w 1 equal to 3 and w 2 equal to 3.

Just go back and plot this and see what. You get you will not get such a smooth such a nice looking S, but you will still get something which looks like looks like a snakes hood, right? So, in still get that S shaped function it just that it would be bent at some points and it be thinner at some points and broader at the other points. So, just go back and see and then you will realize what is happening, right?

(Refer Slide Time: 17:29)

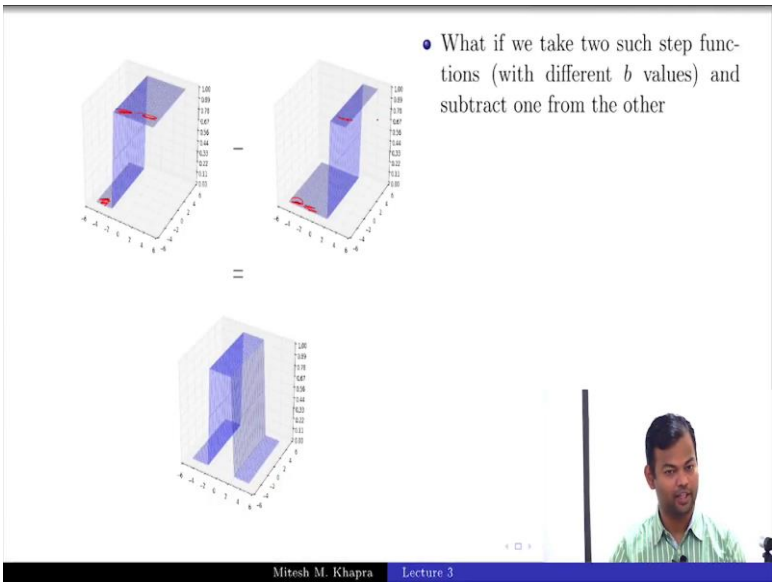


So, here again what we want to figure out is from the single sigmoid I was able to take you to a tower function, right? From a 2-dimensional sigmoid what does a tower mean here and how do I take you to the tower? So, that is what I want to do. So, I have said w 2 equal to 0 and I will it will become obvious why I have done that. So, just understand what the figure is doing, right? So, this if you just look at this is like the cross cut, right? so, you are looking at the front view of this figure and that is just the sigmoid function without the w 2 right? and now, since I have said w 2 equal to 0, no matter what I set x 2 to the same function will get repeated throughout, that axis, do you get that?

So, that is why this entire function is just getting repeated throughout this axis and then you just get a similar S shape function, everyone gets that? How many of you do not get that? How many of you get that? So, this if you look at the front view this is the sigmoid of one variable, but since I have said w 2 to 0, no matter what I change x 2 to, the function is going to remain the same. So, it will just get copied throughout the x 2 axis, is that, fine with you.

Now, what will happen if I increase w 2, sorry w 1? Same thing right, it will just keep shifting till it becomes almost like a 2D step function, ok. Now, what will happen if I increase b? Shift. I can do the same thing here also; same logic applies here also, ok.

(Refer Slide Time: 19:12)

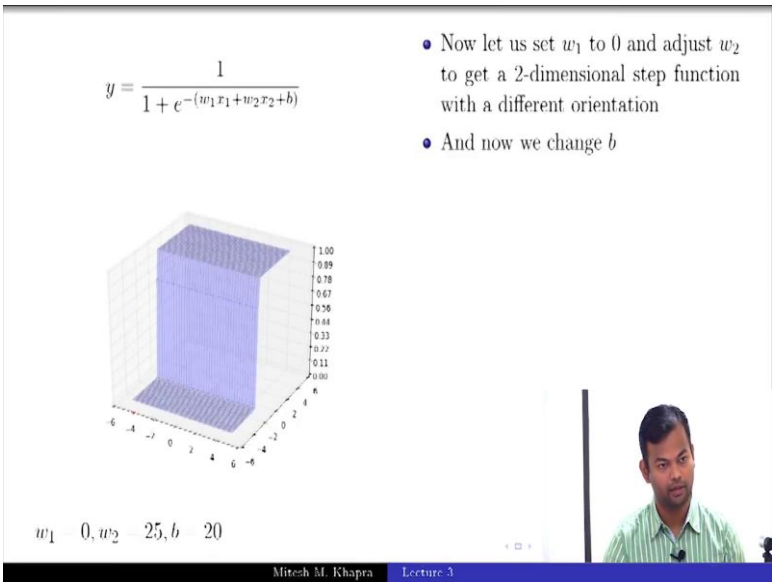


Now, what is the next step that I am going to do? Take two of these which are shifted by some point and, then subtract what will I get? Everyone had this figure in mind? So, just see right? So, this portion both are 0, so, 0 minus 0 would be 0 this portion this is 1, but this is 0. So, that would be 1 minus 0 and again in this portion both of them are 1. So, 1 minus 1 would be 0. So, you will get this kind of function would you like to live in such a tower? I am very serious, yes or no? No? Why? It is open from two sides, right? You cannot live in this tower. So, you want something which is a closed tower, right. So, how will you do that give me an intuition?

We will do the reverse thing. What will be set to 0?

W 1, ok.

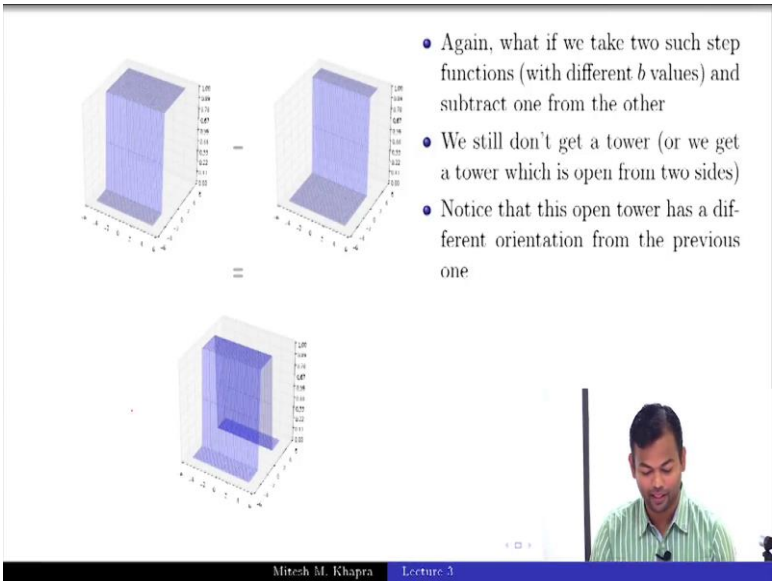
(Refer Slide Time: 19:58)



And, this is how it would look the orientation would change and again so, notice that this is your sigmoid function and since I have set x 1 to 0, no matter what I change along the x 1 axis the same function gets copied and you get a nice looking a sigmoid function.

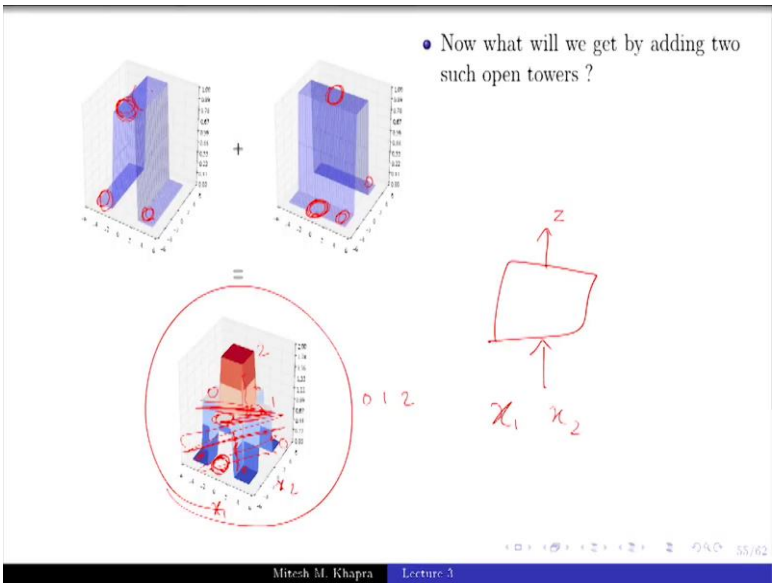
Now, again I will do the same thing I will increase the w, I will get a close to a step function I will increase the b. I will move along this axis.

(Refer Slide Time: 20:24)



Next step, take two of these subtract get what? Another tower function, this is also not a tower that you like to stay in. So, what do I do now? Add them, sure? Add this tower to the other tower.

(Refer Slide Time: 20:44)



So, now what will happen if I add these two, will you get a tower function? What will you get?

You will get a tower function with a parking floor, right? Is that what you will get? Everyone understands why this is? So, these portions both are 0. So, you get 0 same logic applies for all the four corners, right, is that fine; now, for this portion or rather this area, right. So, this guy is 0, this guy is 1. So, you will get a 1, the same logic applies for all these four corners in the centre both are actually 1. So, 1 plus 1 would give you 2. So, this is 2, this is level 2, this is level 1, this is level 0, is that fine?

So, what am I done so far? I have taken my x 1, x 2 passed it through some transformations, right this what are these transformations we will see, but transform it through these multiple hoops, right where I adjusted a w's and b's and I have got some z right and this is how that z behaves. For different values of x 1 comma x 2, I will get these different values and these values range from 0 to 1 to 2, is this pictured clear? I have taken my original x 1 comma x 2, passed it to some of these transformations and irrespective of what my x 1 to x 2 is this tells me the entire range of values that I will get. For some combination of x 1 comma x 2 I will get 0, for some combinations I will get 1,

for some combinations I will get 2 and some combinations also between 1 and 2, right. So, these places where it transitions, is that clear, is that picture clear to everyone?

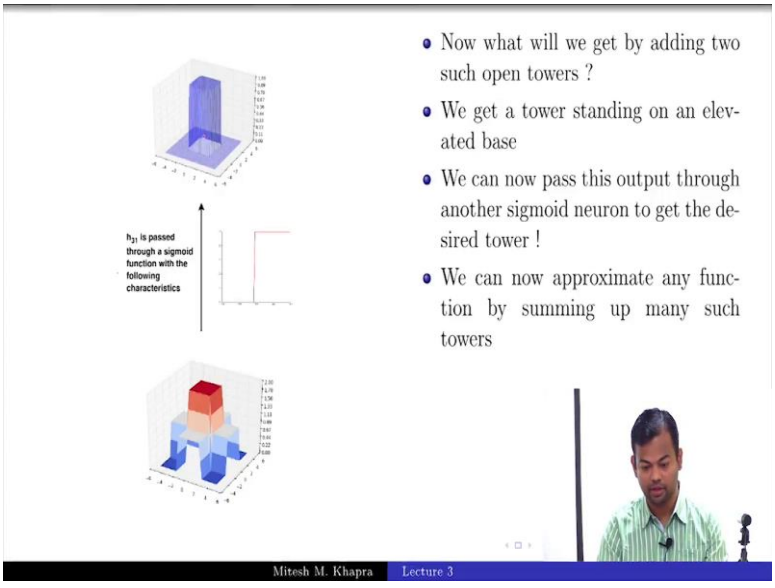
So, now I can treat this as the output z, ok. Now, from here how do I go to a tower?

Threshold it; how will you threshold it? What do you want? You only want this much part to exist, right this without the parking floor. How will you do it? Any output which is greater than equal to 2, you want to keep it any output which is less than 2, you want to make it 0. If I do this will I get a tower? Right, sorry, greater than equal to 1 any value which is greater than equal to 1, you want to keep it, anything which is less than 1 you want to make it 0. So, this entire thing will get demolished. How do you do this? This is an if, else, ok.

If else if something is greater than equal to 0, do something else do something else, what is that?

Perceptron, right, but we do not want to use perceptron. We want to use sigmoid neurons. Have you learned an approximation from a sigmoid neuron to a perceptron. Very high w, right; you get the intuition. Let us see what we do on the next slide.

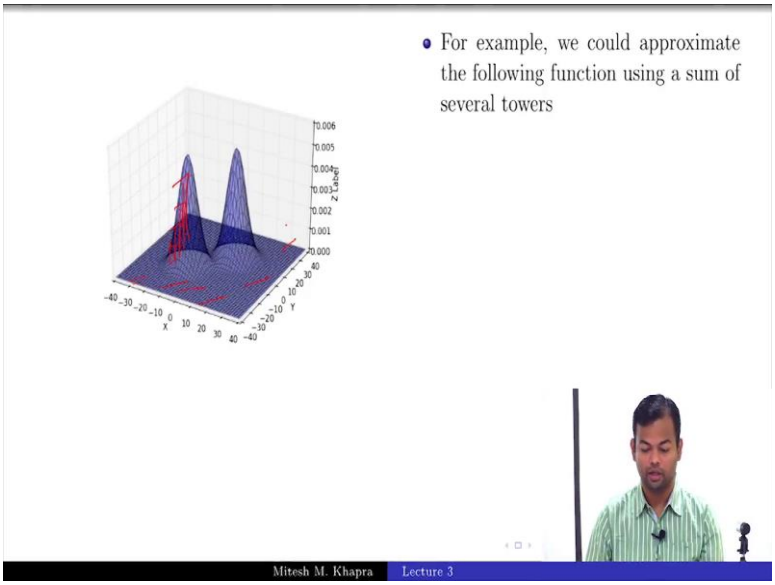
(Refer Slide Time: 24:07)



So, I take this any z which comes from here I will pass it through a sigmoid neuron which are very high slope such that the threshold is at 1, anything which is greater than 1 will pass out as 1; anything which is less than 1 will go to 0. So, everyone sees how we

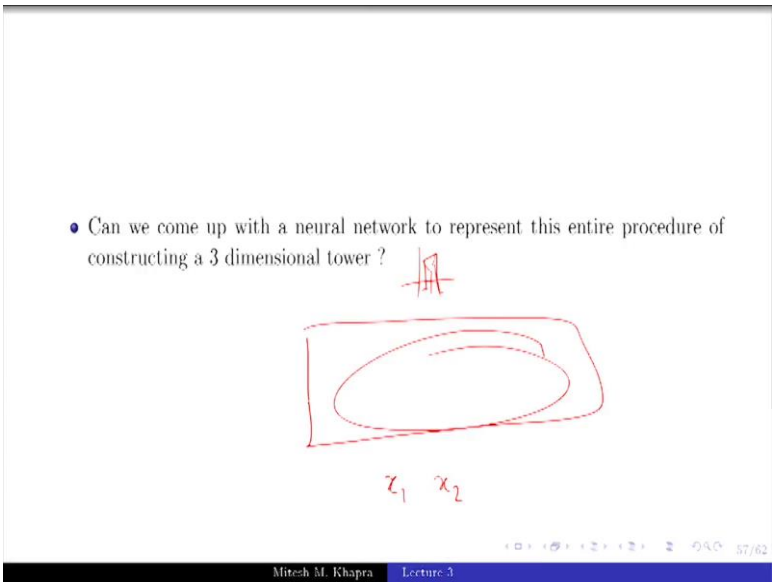
took this structure and converted it to a tower. We have this tower now, now what do I do with this?

(Refer Slide Time: 24:34)



I lead multiple such towers and I can approximate this I could put a tower here, here, here and so on. I could have these multiple towers and here of course, all my towers would be of 0 height right in this region, right. So now, I can cover the entire 2D space with a lot of tower functions and approximate this exactly, that is a very weird statement, approximate this exactly I mean approximate this to arbitrary precision, everyone gets this? Do you see why we constructed these tower functions and now we can put them inside this cone and approximate it.

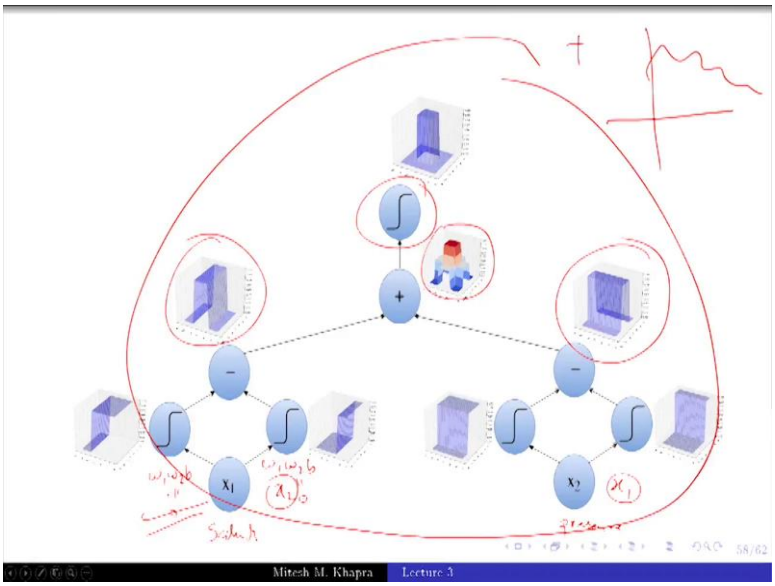
(Refer Slide Time: 25:09)



Now, all this is fine I was making some towers there. So, can you now give me a complete neural network which does this? I want you to imagine that. Remember you are taking what I am asking you to do is this x 1, x 2 give me this such that I get this 2-dimensional tower I do not know how to draw it, something like this maybe whatever. So, I want this 2-dimensional tower what is this network of perceptrons going to look like? Just go back to all the operations that we did and try to imagine in your mind.

No, we will not use perceptron because we can always use a sigmoid neuron instead of a perceptron with the high w. I do not expect you to answer this I just want you to imagine; right, we just try with a there is something known as a pen, there is something on a paper, ok. So, here is the solution.

(Refer Slide Time: 26:08)



So, what is happening here you have this salinity and oh actually this is slightly wrong I do not know why you guys saying it is correct. Actually, at both places I need both the inputs it is just that in one case I do not care about that input because I have said w 2 to 0. So, I learn these weights w1 w2 b, w1 w2 b of course, here the network should learn that w 2 is equal to 0, right and then you get this one tower do not needs this to be modified, this figure is incorrect.

So, we need x 1 x 2 both as inputs we need to label it with w 1 w 2 equal to 0 and b and so on it. So, we will discuss this later, anyways, but you get the idea right that you take these two inputs make one tower, take the inputs again make another tower, add them up to get this function, pass it through this step neuron function step sigmoid function, so that you get the tower. So, this is one block. You will have many such blocks each of which will learn different w's and b's. So, that they get shifted and then you will place them all together you have an aggregator on top of this which will combine them. Just a minute, how many of you get this? ok good.

Yes. So, that is a good question, I am going to come to that, right. So, I have very conveniently given you a solution where I have what is the bad thing that I have done? I have hand coded these things, right. I have hand coded w ones w twos and b's, is that fine in practice? No, I mean that is where we started off and we do not want to hand code these, right?

So, now you know a learning algorithm for a single sigmoid neuron. Now, what you have is a network of neurons right for this network of neurons, I need to give you a learning algorithm driven by the objective function that whatever output it gives would be very close to that arbitrary function that you are trying to model.

If I give you a learning algorithm then you would be convinced that if this has to be minimized and the weight configuration which need it needs to arrive at as w 2 is equal to 0, then the algorithm should be able to do that, right. Because, we saw we have some faith in these algorithms in the case of a signal sigmoid neuron that with the right objective function it will give me a principled way of reaching that objective function. In this big network my objective function is to arbitrarily to approximate this of this true function, right.

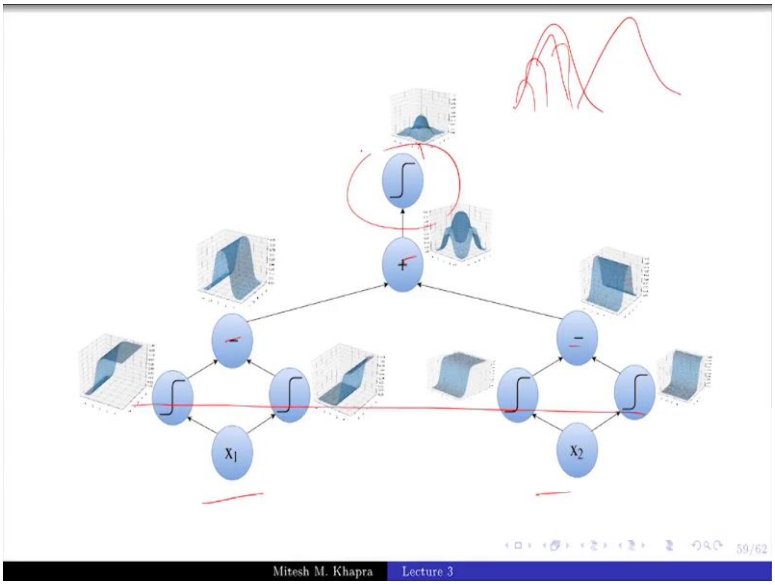
So, now if I give you that as the objective that whatever outputs the network generates; so, the network might generate something like this. So, that has to be very close to the true output that is the objective function that I am going to use in that learning algorithm and if that learning algorithm works which will prove then you should be able to arrive at the necessary weights to make this approximation, right? Is that clear and in fact, there might you might not even have to do these multiple towers in practice. All I am trying to prove is that there is one solution which exists.

If there is one solution which exists I can say that locate the network can learn, that is the only claim I make. I am not saying this is the only solution, right? Same as in the case of the Boolean functions, where I said that one solution exists where you have to raise to n neurons of the hidden layer that was a sufficient solution, that was not a necessary solution for the AND function we were actually able to do it with a single sigma neuron, right? So, just keep that in mind I am just giving you a sufficient solution.

And, the network could actually learn something better than this all right this is again a very bulky solution, why? It scales with the number of neurones' proportional to number of input variables that you have. So, that is for a sufficient solution, but you would want something better than that. All I am trying to say is that it can approximate I am just telling you the representation power and just as we had the catch there that the hidden layer is very large the same catch applies here also, is this story clear to everyone?

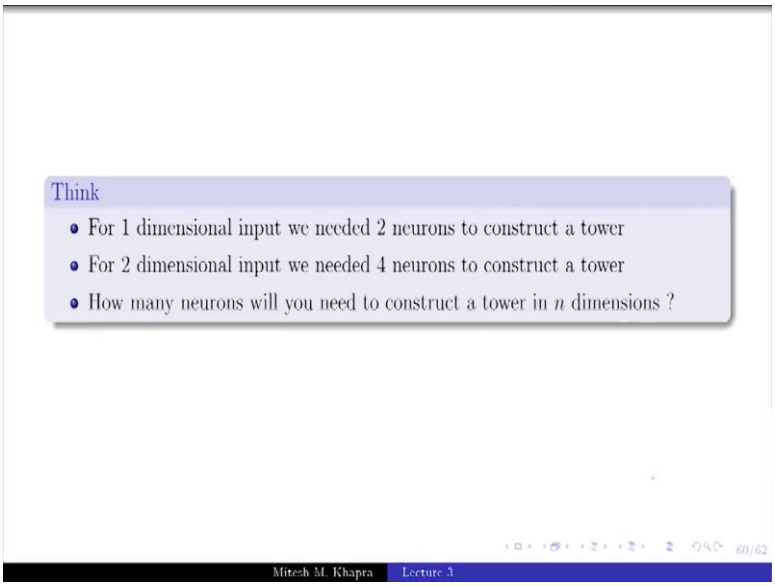
So, I have given you a solution I have not told you how to learn the weights I have given you a network. Now, later on we will discuss a learning algorithm for this network. And we will have some confidence that given a particular objective function that learning algorithm can strive to go to minimum error or minimize the quantity of that objective function that is going to come in two lectures from now, is that fine?

(Refer Slide Time: 30:32)



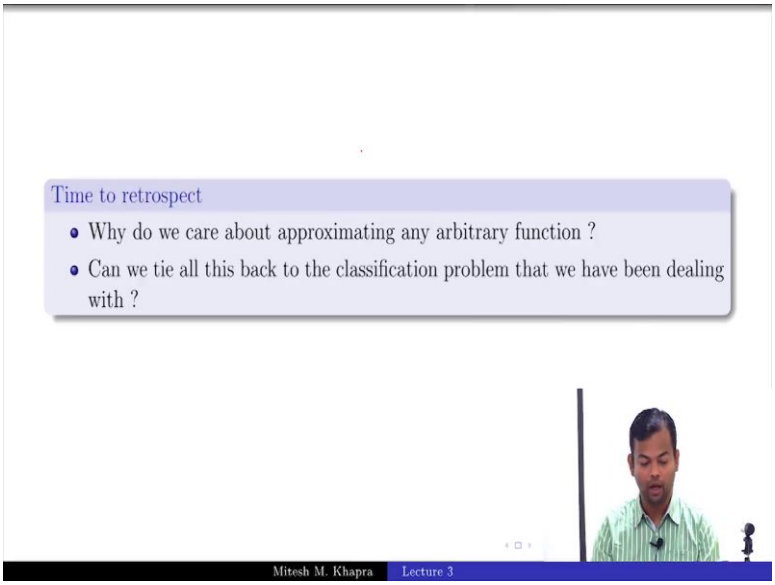
And, that was for the tower function now; I could have actually directly done this right. So, I wanted to approximate these functions. So, I could have placed a lot of these kinds of things here and approximated it, right? So, that instead of that very high slope sigmoid function I could just use a normal sigmoid function also. Ok, and again there is a error here, but I hope you get the picture, it is just that you feed both the inputs to them.

(Refer Slide Time: 30:56)



So, for 1 dimensional input we needed 2 neurons to construct a tower. For 2 dimensional input how many neurons did we need? I am just counting these because these are simple aggregators, right and this is one constant at the end. So, how many did we need actually o of 2 n, I mean o of I mean. So, for n how many would we need? Let us try to work that out, ok. So, I will ask you that in the quiz how many do we need for n dimensions.

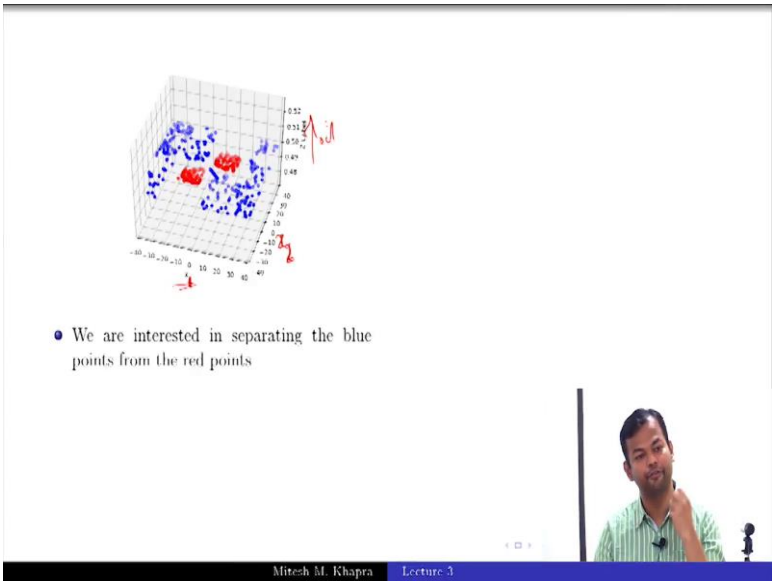
(Refer Slide Time: 31:30)



Now, why do we care about approximating any arbitrary function? We will again try to close the loop now, we saw that we can arbitrarily we can approximate any arbitrary

function. But now again I want to come back to the point why do we want to do this and can we tie this back to the classification problem that we were dealing with.

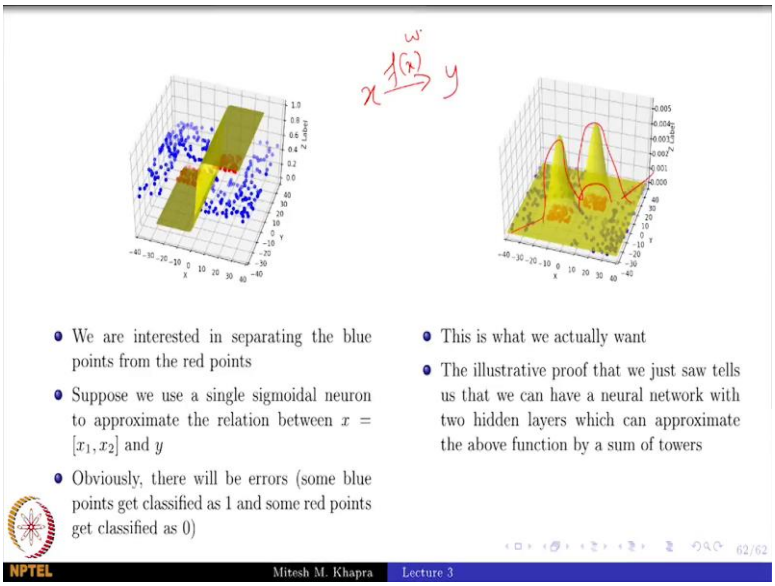
(Refer Slide Time: 31:47)



And, this is the data which I had given you which was there were some points some values of x and y sorry; this should be x 1 and x 2 it is, where this is pressure and salinity or salinity tendency and this is the output which is oil.

Now, there was this is what the function actually looks like now what would have happened if I had used a single sigmoid neuron to try to approximate this function try to represent this function and sigmoid neuron in 2 dimensions, right, so, the 2 dimensional sigma what would have happened? Can you give me one solution for this? Remember earlier I had said that perceptron cannot handle data which is not linearly separable, but then I anyways used it for data which was not linearly separable. And we got some line such that we got some errors the red points and the blue points are not clearly separated. So, I am asking you for a similar thing here, I force you to use a sigmoid neuron, what would you give me?

(Refer Slide Time: 32:47)



Is this fine? This is one of the possibilities of course, it could have been oriented differently and several things. What is happening here is that for these blue points it is acting correctly, but for these red points it is not acting correctly. I am assuming red is positive and blue is negative, I think that should have been the other way round, but let us assume red is positive and blue is negative. Again, now for these red points this part is working fine, but it is misclassifying all these blue points.

So, all these bad locations is actually saying that you can find oil and for all these good locations here it is saying that you cannot find oil, that is what a sigmoid neuron would do and you could have multiple solutions are possible here right, but all of them would have this problem that will make errors on some red points and some blue points right, but the true solution that we wanted is something like this. Again there are multiple solutions possible, right. You could have anything there are you could have even finer one side you could just have this much there many things possible this is one such solution. What the illustrative proof told you is that, you can actually use a network of perceptrons and approximate this arbitrary function which exists between the input variables and the output variable.

So, if this is the function which exists between the input variables and the output variables now, you could take these multiple 2 dimensional tower functions and approximate it. With the catch that you might need many of these in the hidden layer, but

you can still do that, ok. So, that is why this in theorem important because now any problem that you take right any problem that you will have in machine learning would always want you to take an x learn a function of x which takes you to y this function will be have some the function will have some?

Parameters, right and now what this theorem is saying is that you could adjust these parameters such that you can arbitrarily come close to the true function, right. So, that is the significance of this. Any machine learning problem that you can think of in the sense of classification or regression you would find that this is useful and I am giving you a very powerful tool to do that. Of course, with the catch that I am not giving you any bound on the number of neurons that you will need, I am just saying use as many as you want.